Byzantine-tolerant detection of causality: there is no holy grail

Mechanisation in Isabelle/HOL of the impossibility results of Misra and Kshemkalyani (Parallel Computing 124, 2025)

May 19, 2026

Abstract

We mechanise in Isabelle/HOL the impossibility results of Misra and Kshemkalyani, *Byzantine-tolerant detection of causality: there is no holy grail* (Parallel Computing 124 (2025), article 103136). The causality determination problem CD(E, F, e_i^*) asks an asynchronous distributed algorithm to reconstruct the global execution history E at a correct process p_i, with respect to a target event e_i^*, such that the happened-before relation evaluated against the algorithm's collected F agrees with the truth in E. In the presence of even a single Byzantine process this is impossible.

We formalise:

- Theorems 1 and 2 (paper Section 4.1): no algorithm can prevent false negatives in the Byzantine model, and for internal events no algorithm can prevent both false negatives and false positives. Both proofs are constructive: given any candidate algorithm we exhibit a Byzantine adversary using a fresh natural number as event identifier.
- Theorems 3, 4, 5 (paper Section 4.2): the headline impossibility results for unicast, broadcast, and multicast communication. The proofs route directly through Theorem 1 (`CD_FN_unavoidable`) – bypassing the paper's Consensus $\preceq$ Black_Box $\preceq$ CD + FLP chain at the critical-path level. The paper's chain is preserved as a fully-proven alternative derivation (`Reductions.thy` + `BlackBox_Unsolvable.thy` + `FLP_Consensus.thy`), with the FLP impossibility imported from the AFP entry *FLP* (Bisping et al., 2025) as a proven theorem (`flp_consensus_unsolvable`, discharged against `flpPseudoConsensus.ConsensusFails`).
- Theorems 6, 7, 8 (paper Section 4.3): the Byzantine-happened-before extension CD_B. T6/T7 solvable under unicast/broadcast at three layers (abstract `naive_cd_B_alg` under `correct_reporting`; operational reduction to a structural delivery property; concrete operational construction via an inductive `run_step` with explicit in-flight buffer). T8 impossible under multicast.
- Theorems 9 through 14 (paper Section 4.4 + 4.5): cryptography variants. Impossibilities (T9, T10, T11) as direct corollaries of T3/T4/T5 – the Theorem-1 FN attack is crypto-independent. Possibilities (T12, T13) as direct corollaries of T6/T7. T14 (the genuinely new content: multicast `+` crypto possible, whereas T8 said impossible without crypto) via the naive algorithm under `correct_reporting`. Cryptography is treated at the same abstraction the paper itself takes when citing Bracha 1987 for BRB.
- Theorems 15 and 16 (paper Section 5.1): in the Byzantine model CD is harder than Consensus, directly from CD unsolvability and the abstract Consensus

witness; in the crash-failure model, the asymmetry reverses (Consensus is unsolvable by FLP, CD is solvable via the `cd_alg_with_recv` signature with $F := recv$ whenever reports are faithful to the true execution).

- Theorems 17 and 18 (paper Section 5.2): CD and CO are interreducible in the Byzantine model (T17), and CO is subject to FN and FN-or-FP for internal-event witnesses (T18). CO is mechanised as a strict restriction of CD to receive-event targets; T17 forward direction is constructive (a CD solver is automatically a CO solver), T18 is proven directly via a two-message fresh-id construction, and the reverse direction of T17 is proven under Byzantine premises by routing through the two independently-established impossibility chains.

The development sits on top of the AFP entry *FLP*. The headline impossibility theorems (3, 4, 5) take exactly one mild finiteness side hypothesis `fin_cd` — shape-compatible with the `fin_F` of Theorem 1 (`CD_FN_unavoidable`) — and no other meta-level assumptions. No HOL axioms, no locale axioms on the impossibility chain.

The proof routes directly through Theorem 1: `CD_solvable` extracts a `produces_valid_F` witness, which Theorem 1 defeats with an admissible Byzantine adversary. The paper's "Consensus $\preceq$ Black_Box $\preceq$ CD + FLP" chain is preserved as paper-faithful documentation (R1, R2 in `Reductions.thy`; the BlackBox impossibility `BlackBox_unsolvable` in `BlackBox_Unsolvable.thy`; and the FLP impossibility `flp_consensus_unsolvable` in `FLP_Consensus.thy`, proven against the AFP entry's `ConsensusFails`) but is not on the critical path of the headline theorems.

Contents

Reading order

The theories are intended to be read in the following order:

1. `ByzantineSystem`: the process partition (correct $\uplus$ Byzantine) and the abstract Consensus signature.
2. `Events`: events, per-process and global histories, program order, message order, happened-before relation (paper Definitions 1, 2).
3. `CD`: the Causality Determination problem, $valid(F)$, false positives / negatives, the adversary model, the CD-solver signature (paper Definition 5).
4. `BlackBox`: the Black_Box problem of paper Section 4.2 with its piecewise broadcast value w.
5. `Reductions`: the two reductions of paper Section 4.2 – R1 (Consensus $\preceq$ BB) constructive in declarative Isar, R2 (BB $\preceq$ CD) constructive modulo the paper's named meta-level step `cd_can_identify_correct`.
6. `Theorems_1_2`: paper Section 4.1, Theorems 1 and 2, with explicit Byzantine adversary constructions using fresh natural numbers.
7. `BlackBox_Unsolvable`: a proof of $\neg$ `BlackBox_solvable procs correct` by direct reduction to Theorem 1 via the BB-to-CD projection. Discharges what was previously a meta-level hypothesis on Theorems 3/4/5.
8. `FLP_Consensus`: the FLP-style consensus predicate with the impossibility proven against the AFP entry's `ConsensusFails`. Retained as the AFP-FLP citation motivating the paper's chosen chain; not on the impossibility path of this development.
9. `Impossibility`: paper Section 4.2, Theorems 3, 4, 5 – the headline results.
10. `Foundation_Vacuity`: a regression diagnostic documenting why the abstract `solves_Consensus` predicate is too weak to express FLP, and motivating the FLP-style chain used by `Impossibility`.
11. `CD_vs_Consensus`: paper Section 5.1, Theorems 15 and 16 (Byzantine: CD harder than Consensus; crash: Consensus half of T16 only).
12. `BHB`, `CD_B_Algorithm`, `Delivery`, `Execution_Model`: paper Section 4.3. The B-happened-before relation; the CD_B problem; abstract and operational solvability under unicast/broadcast (Theorems 6, 7); impossibility under multicast (Theorem 8); inductive execution model with deadlock freedom.
13. `Liveness`: real-world fairness on the inductive execution model. Infinite executions modelled as `nat => 'p config` with a side condition that each adjacent pair satisfies `run_step`; the fairness predicate `fair_run` states that every buffered triple eventually leaves the buffer; the headline theorem `fair_run_delivers` says every correct-to- correct `Send` in a fair infinite run has a matching `Receive` at some later step.

14. `Primitives`: Byzantine Reliable Unicast (BRU) and Byzantine Causal Broadcast over Byzantine Reliable Broadcast (BCB-over-BRB) named explicitly at the event level. `bru_satisfied`, `bcb_causal_order`, and `bcb_over_brb_satisfied` predicates; operational discharge of BRU from the run model; end-to-end composition theorems giving operational T6 / T7 directly from a fair drained run.

15. `T6_Concrete`: a fully-concrete worked example of T6. A two-process scenario with an explicit three-step run (`step_send`, `step_recv`, `step_internal`), composed with the operational T6 chain to demonstrate that the existential statement `CD_B_solvable_with_recv Unicast correct` is non- vacuously satisfiable – there is an actual adversary, an actual run, and an actual algorithm output that witness it.

16. `T6_Multihop`: a three-process two-hop variant of the T6 demo. Five `run_step` transitions cover a bhb chain crossing three processes (p_b sends to p_a, p_a relays to p_c, p_c's target internal event); `multi_bhb_chain` proves the four-edge bhb path, `T6_multihop_demo` and `T6_multihop_witnessed` mirror the 1-message demo at the bigger scale.

17. `T6_With_Byzantine`: a T6 demo with a Byzantine bystander. Two correct processes plus a Byzantine p_c performing an arbitrary local internal event (exercises `step_byzantine`, unused in the other demos); `byzantine_event_not_on_bhb_chain_*` proves the Byzantine's event is excluded from every bhb chain by `bhb_proc_of_endpoints`; `T6_with_byzantine_demo` and `T6_with_byzantine_witnessed` demonstrate T6's robustness to live Byzantine activity.

18. `CO`: paper Section 5.2, Theorems 17 and 18. The causal-ordering problem CO as a strict restriction of CD to receive-event targets; the forward reduction CD $\Longrightarrow$ CO; T18 false-negative and false-negative-or- false-positive constructions; the impossibility of CO under all three communication modes; the interreducibility statement under Byzantine premises.

19. `CD_with_Crypto`: paper Section 4.4 + 4.5, Theorems 9 through 14. The six cryptography theorems, all as corollaries of the corresponding non-crypto results plus T14 as the new multicast-with-crypto-possibility case.

```
theory ByzantineSystem
  imports
    FLP.AsynchronousSystem
    FLP.Execution
    FLP.FLPTheorem
begin
```

1 Process identifiers, correct vs. Byzantine partition

Paper, Section 2:

> "The distributed system is modeled as an undirected graph `G = (P, C)`. Here `P` is the set of processes communicating asynchronously in the distributed system. Let `|P| = n`."

The paper does not formalise the correct/Byzantine partition in the model itself; it talks about "a correct process `p_c`" and "a Byzantine process `p_b`" in prose. We make the partition explicit: `procs` is the set `P`, `correct` is the set of non-Byzantine processes, `byzantine` is the rest. Finiteness and non-emptiness of `P` are taken from the paper's tacit assumptions.

```
locale process_partition =
  fixes
    procs     :: "'p set" and
    correct   :: "'p set" and
    byzantine :: "'p set"
  assumes
    finite_procs:     "finite procs" and
    nonempty_procs:   "procs ≠ {}" and
    partition_union:  "procs = correct ∪ byzantine" and
    partition_disj:   "correct ∩ byzantine = {}" and
    correct_subset:   "correct ⊆ procs" and
    byzantine_subset: "byzantine ⊆ procs"

context process_partition
begin

lemma correct_finite [simp]: "finite correct"
  using finite_procs correct_subset by (rule rev_finite_subset)

lemma byzantine_finite [simp]: "finite byzantine"
  using finite_procs byzantine_subset by (rule rev_finite_subset)

lemma correct_byzantine_disjoint:
  assumes "p ∈ correct" shows "p ∉ byzantine"
  using assms partition_disj by blast

lemma proc_correct_or_byz:
  assumes "p ∈ procs" shows "p ∈ correct ∨ p ∈ byzantine"
  using assms partition_union by blast
```

end — context `process_partition`

2 Abstract Consensus solver signature

Paper, Section 4.2:

> "In the Consensus problem, each process has an initial value and all correct processes must agree on a single value. The solution needs to satisfy the following three conditions.
>
> - Agreement: All non-faulty processes must agree on the same single value.
> - Validity: If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value.
> - Termination: Each non-faulty process must eventually decide on a value.
>
> "

We model a Consensus algorithm at the level of its observable input/output behaviour: it takes an initial-value vector `V` (boolean per process) and produces a decision (boolean per process).

Major deviation (load-bearing). At this level of abstraction the predicate `solves_Consensus` below only enforces Agreement and Validity, with Termination trivially holding by totality of the function type. The FLP impossibility result relies crucially on Termination-under-failure in an asynchronous distributed *protocol*, which a pure HOL function does not model; this makes `solves_Consensus` too weak for FLP to bite on, and is the reason the impossibility chain in `Impossibility.thy` routes through the FLP-style predicate in `FLP_Consensus.thy` instead. See `Foundation_Vacuity.thy` for a machine-checked counter-example to the naive "no abstract consensus solver exists" claim.

```
type_synonym 'p consensus_alg = "('p ⇒ bool) ⇒ 'p ⇒ bool"

definition consensus_agreement ::
  "'p set ⇒ 'p consensus_alg ⇒ ('p ⇒ bool) ⇒ bool" where
  "consensus_agreement C alg V ⟷ (∀p ∈ C. ∀q ∈ C. alg V p = alg V q)"

definition consensus_validity ::
  "'p set ⇒ 'p consensus_alg ⇒ ('p ⇒ bool) ⇒ bool" where
  "consensus_validity C alg V ⟷
     ((∀p ∈ C. V p) ⟶ (∀p ∈ C. alg V p)) ∧
     ((∀p ∈ C. ¬ V p) ⟶ (∀p ∈ C. ¬ alg V p))"

definition solves_Consensus ::
  "'p set ⇒ 'p consensus_alg ⇒ bool" where
  "solves_Consensus C alg ⟷
     (∀V. consensus_agreement C alg V ∧ consensus_validity C alg V)"
```

3 The Byzantine system locale

We bundle the process partition into a locale named `byzantineSystem`. The locale has no extra assumptions beyond those of `process_partition`: it merely fixes the partition `procs = correct ∪ byzantine`.

Historical note. Earlier versions of this development added a locale axiom `flp_consensus_impossibility` that claimed the FLP impossibility directly on `solves_Consensus`, but that axiom turned out to be unsatisfiable in HOL (the pure-HOL function `simple_alg C V p ≡ ∃q∈C. V q` satisfies `solves_Consensus`, so denying it collapses to `byzantine = {}`). See `Foundation_Vacuity.thy` for the machine-checked counter-example. The FLP impossibility is now proven (not axiomatised) against the AFP entry's `ConsensusFails` theorem in `FLP_Consensus.thy`. The headline theorems 3/4/5 in `Impossibility.thy` no longer rely on any FLP-derived axiom or hypothesis at all – they route directly through Theorem 1 (`CD_FN_unavoidable`) under a mild `fin_cd` side condition. `flp_consensus_unsolvable` is retained as the AFP-FLP citation that motivates the paper's chosen chain `Consensus ≼ BlackBox ≼ CD + FLP`, but that chain is preserved as documentation rather than load-bearing.

```
locale byzantineSystem = process_partition procs correct byzantine
  for procs correct byzantine :: "'p set"

end


theory Events
  imports ByzantineSystem
begin
```

4 Events

The paper writes `e_i^x` for the `x`-th event of process `p_i` (Section 2: "Let `e_i^x`, where `x ≥ 1`, denote the `x`th event executed by process `p_i`"). We represent this as a tagged record carrying both the process `p` and the local sequence number `n`, plus – for sends and receives – the peer process and a message identifier that links the matching send and receive.

```
datatype 'p event =
    Internal (proc_of: 'p) (seq_of: nat)
  | Send     (proc_of: 'p) (seq_of: nat) (peer: 'p) (msg_id: nat)
  | Receive  (proc_of: 'p) (seq_of: nat) (peer: 'p) (msg_id: nat)

abbreviation is_send :: "'p event ⇒ bool" where
  "is_send e ≡ (∃p n q m. e = Send p n q m)"

abbreviation is_receive :: "'p event ⇒ bool" where
  "is_receive e ≡ (∃p n q m. e = Receive p n q m)"

abbreviation is_internal :: "'p event ⇒ bool" where
  "is_internal e ≡ (∃p n. e = Internal p n)"
```

A send and the corresponding receive of the same message are identified by a shared `msg_id`. The `matches` predicate also checks that the peer fields are dual (the send's `peer`

is the receiver, the receive's `peer` is the sender).

Deviation: the paper does not bother to give `matches` a separate name; it is implicit in "the corresponding message receive event" (Definition 1). We name it because Isabelle's inductive happened-before definition needs it as a predicate.

```
fun matches :: "'p event ⇒ 'p event ⇒ bool" where
  "matches (Send p n q m) (Receive q' n' p' m') ⟷ (p = p' ∧ q = q' ∧ m = m')"
| "matches _ _ ⟷ False"
```

5 Per-process and global histories

Section 2 of the paper: "The execution history at `p_i` is the finite execution at `p_i` up to the current or most recent or specified local state." Section 3: "Let `E_i` denote the actual execution history at `p_i` ... and let `E = ⋃_i {E_i}`." We represent both `E_i` (per process) and `E` (global) by a single mapping `'p ⇒ 'p event list`: the per-process history is recovered by applying the mapping to a process, and the set of all events `T(E)` is given by `events_of` below.

```
type_synonym 'p history_local = "'p event list"
type_synonym 'p history       = "'p ⇒ 'p history_local"
```

`T(E)` in the paper: the set of all events occurring in `E`, taken process-by-process and unioned.

```
definition events_of :: "'p history ⇒ 'p event set" where
  "events_of H = (⋃p. set (H p))"
```

Well-formedness of a history: each per-process list (a) records only events whose `proc_of` field is that process, and (b) numbers its events 1, 2, 3, ... consecutively, matching the paper's enumeration ⟨`e_i^1, e_i^2, ...`⟩.

```
definition wf_history_local :: "'p ⇒ 'p history_local ⇒ bool" where
  "wf_history_local p es ⟷
     (∀e ∈ set es. proc_of e = p) ∧
     (∀k < length es. seq_of (es ! k) = Suc k)"

definition wf_history :: "'p history ⇒ bool" where
  "wf_history H ⟷ (∀p. wf_history_local p (H p))"

lemma events_of_simp:
  "e ∈ events_of H ⟷ (∃p. e ∈ set (H p))"
  by (auto simp: events_of_def)
```

6 Program order and the happened-before relation

Definition 1, rule 1 (Section 2):

> "Program Order: For the sequence of events ⟨`e_i^1, e_i^2, ...`⟩ executed by process `p_i`, ∀ `x, y` such that `x < y` we have `e_i^x → e_i^y`."

We mechanise this as: `e` precedes `e'` in `H p` iff `e` appears at a strictly earlier list index than `e'` in the same per-process list.

```
definition program_order :: "'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "program_order H e e' ⟷
     (∃p i j. i < j ∧ j < length (H p) ∧ (H p) ! i = e ∧ (H p) ! j = e')"
```

Definition 1, rule 2 (Section 2):

> "Message Order: If event `e_i^x` is a message send event executed at process `p_i` and `e_j^y` is the corresponding message receive event at process `p_j`, then `e_i^x → e_j^y`."

Our `message_order` fires when the send and receive are matched by `matches` (same peer, same message id) and both events are recorded in the history.

Deviation: the paper takes for granted that the send-receive pair really happened; we make this explicit by requiring both endpoints to be in `events_of H`. For the algorithm-collected history `F` this is the right reading – if either endpoint is not in `F`, then "the algorithm doesn't know about it", and this is how the paper itself reasons when discussing false negatives ("Byzantine processes may delete information about `e_h^x` and `m` from `F_h`, leading to a false negative", Section 4.2).

```
definition message_order :: "'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "message_order H e e' ⟷
     is_send e ∧ is_receive e' ∧ matches e e' ∧
     e ∈ events_of H ∧ e' ∈ events_of H"
```

One step of the happened-before relation: either a program-order step or a message-order step.

```
definition hb_step :: "'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "hb_step H e e' ⟷ program_order H e e' ∨ message_order H e e'"
```

Definition 1, rule 3 (Section 2):

> "Transitive Order: If `e → e' ∧ e' → e''` then `e → e''`."

We define `hb` as the transitive closure of `hb_step`. This is the standard Lamport happened-before relation, evaluated against an arbitrary history `H`.

Note: the paper's Definition 3 is the Byzantine-happened-before relation `→_B` that restricts both program order and message order to correct-process chains. We do not formalise `→_B` in this development – the impossibility theorems we cover (T1-T5) work on the plain `→`. The `Send` and `Receive` constructors carry the peer field precisely so that a future development of Theorems 6-8 can define `→_B` by an inductive predicate quantifying over a correct set.

```
definition hb :: "'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "hb H e e' ⟷ (hb_step H)^++ e e'"
```

The paper's `e → e'|_E` notation (Section 3):

> "Let `e_1 → e_2 |_E` and `e_1 → e_2 |_F` be the evaluation (1 or 0) of `e_1 → e_2` using `E` and `F`, respectively. ... If `e_h^x ∉ T(E)` then `e_h^x → e_i^*|_E` evaluates to false; ... If `e_h^x ∉ T(F)` (or `e_i^* ∉ T(F)`) then `e_h^x → e_i^*|_F` evaluates to false."

We mechanise this via a boolean `hb_eval` that requires both endpoints to be in `events_of H` before consulting the transitive closure.

```
definition hb_eval :: "'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "hb_eval H e e' ⟷
     (e ∈ events_of H ∧ e' ∈ events_of H ∧ hb H e e')"
```

7 Causal past

Definition 2 (Section 2): "The causal past of an event `e` is denoted as `CP(e)` and defined as the set of events that causally precede `e` under $\rightarrow$." We parameterise the definition by the history `H` against which the relation is evaluated – the paper writes simply `CP(e)` because in Section 2 only the actual history `E` is in play.

```
definition causal_past :: "'p history ⇒ 'p event ⇒ 'p event set" where
  "causal_past H e = {e'. hb_eval H e' e}"

lemma causal_past_subset_events:
  "causal_past H e ⊆ events_of H"
  by (auto simp: causal_past_def hb_eval_def)
```

8 Algorithm-perceived history

Section 3 of the paper distinguishes the actual global history `E` from the algorithm-collected history `F`:

> "For any causality determination algorithm, let `F_i` be the execution history at `p_i` as perceived and collected by the algorithm and let `F = ⋃_i {F_i}`. ... Byzantine processes may corrupt the collection of `F` to make it different from `E`. ... Therefore it is not sufficient for the correct processes to agree mutually on a `F` that differs from `E` in what happened in `E` at the Byzantine processes; their `F_j` must also agree with `E_j` at all processes `p_j`."

`correct_processes_agree` below is the property an algorithm *would* like to achieve at correct processes. Crucially, we do *not* bake it into admissibility: a Byzantine adversary can cause `F` to disagree with `E` even at correct processes (via contamination), so the impossibility argument needs to keep this as a separate property an algorithm might fail.

```
definition correct_processes_agree ::
  "'p set ⇒ 'p history ⇒ 'p history ⇒ bool" where
  "correct_processes_agree C E F ⟷ (∀p ∈ C. F p = E p)"
```

An admissible pair `(E, F)`: both are well-formed histories. This is the weakest invariant we need; stronger constraints (the formal CD problem itself) appear in `CD.thy` via `valid`.

```
definition admissible_pair ::
  "'p set ⇒ 'p history ⇒ 'p history ⇒ bool" where
  "admissible_pair C E F ⟷ wf_history E ∧ wf_history F"

end
```

theory CD
  imports Events
begin

9 valid(F), false positives, false negatives

Definition 5 of the paper:

> "The causality determination problem `CD (E, F, e_i*)` for any event `e_i* ∈ T E` at a correct process `p_i` is to devise an algorithm to collect the execution history `E` as `F` at `p_i` such that `valid F = 1`, where
>
> $$valid(F) = \begin{cases} 1 & \text{if } \forall e_h^x,\ e_h^x \rightarrow e_i^*|_E = e_h^x \rightarrow e_i^*|_F \\ 0 & \text{otherwise} \end{cases}$$
>
> "

The paper later clarifies (Section 4) that the universal quantifier ranges over `T(E) ∪ T(F)`: "we have to evaluate ... even if `e_h^x ∈ (T(E) ∪ T(F)) \\ T(E)` because such an `e_h^x` is recorded by the algorithm as part of `F`". This is the inclusion of fake events the algorithm may have invented.

Faithfulness: we mechanise exactly this – the quantifier ranges over `events_of E ∪ events_of F`, not just `events_of E`.

```
definition valid :: "'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "valid E F e_star ⟷
     (∀e ∈ events_of E ∪ events_of F.
        hb_eval E e e_star = hb_eval F e e_star)"
```

False negative (paper Section 4):

> "∃ `e_h^x` such that `e_h^x → e_i^*|_E = 1 ∧ e_h^x → e_i^*|_F = 0` (denoting a false negative, abbreviated FN)."

Note: the paper writes the disjunction "either FN or FP" when saying "0 is returned if one of the following two cases holds". We define the two cases separately and prove `valid` iff neither occurs (lemma below).

```
definition false_negative ::
  "'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "false_negative E F e_star ⟷
     (∃e ∈ events_of E ∪ events_of F.
        hb_eval E e e_star ∧ ¬ hb_eval F e e_star)"
```

False positive (paper Section 4):

> "∃ `e_h^x` such that `e_h^x → e_i^*|_E = 0 ∧ e_h^x → e_i^*|_F = 1` (denoting a false positive, abbreviated FP)."

```
definition false_positive ::
```

```
  "'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "false_positive E F e_star ⟷
     (∃e ∈ events_of E ∪ events_of F.
        ¬ hb_eval E e e_star ∧ hb_eval F e e_star)"

lemma valid_iff_no_FP_FN:
  shows "valid E F e_star
           ⟷ ¬ false_negative E F e_star
              ∧ ¬ false_positive E F e_star"
proof -
  have "valid E F e_star
          ⟷ (∀e ∈ events_of E ∪ events_of F.
                hb_eval E e e_star = hb_eval F e e_star)"
    by (simp add: valid_def)
  also have "... ⟷ ¬ false_negative E F e_star
                   ∧ ¬ false_positive E F e_star"
    by (auto simp: false_negative_def false_positive_def)
  finally show ?thesis .
qed
```

10 Adversary model

In our model an adversary fixes the actual execution `E`, the process `i` at which the determination is to be made, and the target event `e_star`. The algorithm-collected `F` is *not* part of the adversary record – it is an output of the algorithm, computed from observable inputs (`i`, `e_star`) under the adversary's chosen Byzantine strategy.

Deviation: the paper's CD problem statement gives `(E, F, e_i^*)` as parameters of the problem. We separate roles so that "algorithm sees only its observable inputs" is an explicit modelling property, not buried in interpretation.

Section 3 of the paper:

> "We assume that a correct process `p_i` needs to determine whether `e_h^x` → `e_i^*` holds and `e_i^*` is an event in `T(E)`."

We mechanise this admissibility as: `E` is well-formed, `i` is correct (i.e. in `C`), `e_star` is an event at `i`, and `e_star` actually occurs in `E`.

```
record 'p adversary =
  adv_E       :: "'p history"
  adv_e_star  :: "'p event"
  adv_i       :: 'p

definition adversary_admissible ::
  "'p set ⇒ 'p adversary ⇒ bool" where
  "adversary_admissible C adv ⟷
     wf_history (adv_E adv) ∧
     adv_i adv ∈ C ∧
     proc_of (adv_e_star adv) = adv_i adv ∧
     adv_e_star adv ∈ events_of (adv_E adv)"
```

11 CD-solver signature

A CD-solver, given the inputs the algorithm at `p_i` can see – namely the process identifier `i` and the target event `e_star` – produces the collected history `F'` it proposes plus a boolean "claim of validity". Paper Section 4:

> "When 1 is returned, the algorithm output matches God's truth and solves CD correctly. Thus, returning 1 indicates that the problem has been solved correctly by the algorithm using `F`."

Deviation: the paper does not pin down whether the algorithm sees `E` during its computation. Our type signature commits to "algorithm sees only (`i`, `e_star`); `F` is its constructed view" – which is the strongest reading consistent with the paper's notion of `F` as "the execution history at `p_i` as perceived and collected by the algorithm" (Section 3).

```
type_synonym 'p cd_solver =
  "'p ⇒ 'p event ⇒ 'p history × bool"
```

Two notions of correctness in increasing strength.

`solves_CD_decision` is the weaker form: the algorithm's boolean output agrees with `valid` *computed against the algorithm's own* `F'`. An algorithm satisfying this is internally consistent – it returns 1 iff its `F'` is a valid reconstruction of *some* execution.

`produces_valid_F` is the paper's intended reading: `F'` actually matches the adversary's true `E` in the sense of Definition 5. The impossibility theorems are stated about this stronger predicate; the lemma below shows it always entails the weaker form (so impossibility of one entails impossibility of the other).

```
definition solves_CD_decision ::
  "'p set ⇒ 'p cd_solver ⇒ bool" where
  "solves_CD_decision C alg ⟷
     (∀adv. adversary_admissible C adv ⟶
        (let (F', b) = alg (adv_i adv) (adv_e_star adv) in
           b = valid (adv_E adv) F' (adv_e_star adv)))"

definition produces_valid_F ::
  "'p set ⇒ 'p cd_solver ⇒ bool" where
  "produces_valid_F C alg ⟷
     (∀adv. adversary_admissible C adv ⟶
        (let (F', _) = alg (adv_i adv) (adv_e_star adv) in
           valid (adv_E adv) F' (adv_e_star adv)))"
```

From a `produces_valid_F` solver we trivially obtain a `solves_CD_decision` solver by always returning the boolean `True`: if `F'` is genuinely valid, then claiming "valid" is correct.

```
lemma produces_valid_F_implies_solves_CD_decision:
  assumes "produces_valid_F C alg"
  shows "∃alg'. solves_CD_decision C alg'"
proof -
  define alg' where
    "alg' ≡ (λi e. (fst (alg i e), True))"
  have "solves_CD_decision C alg'"
```

```
  proof (unfold solves_CD_decision_def, intro allI impI)
    fix adv
    assume A: "adversary_admissible C adv"
    have "fst (alg' (adv_i adv) (adv_e_star adv))
            = fst (alg (adv_i adv) (adv_e_star adv))"
      by (simp add: alg'_def)
    moreover from A assms
    have "valid (adv_E adv) (fst (alg (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
      by (auto simp: produces_valid_F_def Let_def split: prod.split)
    ultimately show
      "(let (F', b) = alg' (adv_i adv) (adv_e_star adv) in
           b = valid (adv_E adv) F' (adv_e_star adv))"
      by (simp add: alg'_def Let_def)
  qed
  thus ?thesis by blast
qed
```

12 Communication mode: unicast / broadcast / multicast

Section 2 of the paper:

> "There are three modes of communication: multicast, unicast, and broadcast. In multicast, a message is sent to a group `G` of processes corresponding to some subset of `P`. A unicast is a multicast where `|G| = 1`. A broadcast is a multicast where `G = P`."

We carry the mode as a datatype tag.

Deviation: our `CD_solvable` predicate is *not* sensitive to the mode tag – it just existentially quantifies over algorithms producing valid `F`, and the `valid` predicate is mode-agnostic at the level of Definition 5. This is a deliberate simplification: at this abstraction the modes only differ in what an adversary may do (the adversary in broadcast mode commits to sending to all processes, etc.), not in the validity criterion itself. The three impossibility theorems are nevertheless stated separately so a richer development that refines the mode (e.g. adding BRB-as-layer for Theorem 4) can specialise the proof without re-stating the theorem.

```
datatype comm_mode = Unicast | Broadcast | Multicast

definition CD_solvable :: "comm_mode ⇒ 'p set ⇒ bool" where
  "CD_solvable m C ⟷ (∃alg. produces_valid_F C alg)"

end


theory BlackBox
  imports CD
begin
```

13 The broadcast value w (paper, Section 4.2)

Paper, Section 4.2, immediately after Theorem 3's proof opens the BB definition:

> "Black_Box(V, E, F, e_i*) executed at p_i takes as input a vector V of initial boolean values, one per process, E, F, and local event e_i* at a process p_i. Black_Box invoked at p_i acts as follows. The correct process p_i broadcasts the value w where:
>
> $$w = \begin{cases} 0 & \text{if each correct } p_j \text{ has } V[j] = 0 \\ 1 & \text{if each correct } p_j \text{ has } V[j] = 1 \\ CD(E, F, e_i^*) & \text{otherwise} \end{cases}$$
>
> and locally returns L, a list of ids of correct processes."

The CD output "valid(F)" is a boolean; we inline "CD(E, F, e_i^*)" as valid applied to the relevant arguments.

```
definition w_value ::
  "'p set ⇒ ('p ⇒ bool) ⇒ 'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "w_value C V E F e_star =
     (if (∀p ∈ C. ¬ V p) then False
      else if (∀p ∈ C. V p) then True
      else valid E F e_star)"
```

Three helper lemmas that decompose w_value by case – used both by the reduction R1 in Reductions.thy and by Validity arguments later.

```
lemma w_value_uniform_false:
  assumes "∀p ∈ C. ¬ V p"
  shows   "w_value C V E F e_star = False"
  using assms unfolding w_value_def by simp

lemma w_value_uniform_true:
  assumes nonempty: "C ≠ {}" and all_true: "∀p ∈ C. V p"
  shows "w_value C V E F e_star = True"
proof -
  from nonempty all_true have "¬ (∀p ∈ C. ¬ V p)" by blast
  with all_true show ?thesis unfolding w_value_def by simp
qed

lemma w_value_mixed:
  assumes notF: "¬ (∀p ∈ C. ¬ V p)" and notT: "¬ (∀p ∈ C. V p)"
  shows "w_value C V E F e_star = valid E F e_star"
proof -
  have step1: "(if (∀p ∈ C. ¬ V p) then False
                else if (∀p ∈ C. V p) then True
                else valid E F e_star)
               = (if (∀p ∈ C. V p) then True else valid E F e_star)"
    using notF by (rule if_not_P)
  have step2: "(if (∀p ∈ C. V p) then True else valid E F e_star)
                 = valid E F e_star"
    using notT by (rule if_not_P)
```

```
  have "w_value C V E F e_star =
          (if (∀p ∈ C. ¬ V p) then False
           else if (∀p ∈ C. V p) then True
           else valid E F e_star)"
    by (simp add: w_value_def)
  also have "... = (if (∀p ∈ C. V p) then True else valid E F e_star)"
    by (rule step1)
  also have "... = valid E F e_star"
    by (rule step2)
  finally show ?thesis .
qed
```

14 Black_Box output

A Black_Box solver at `p_i` outputs three things:

- the collected history `F'` (carried over from the embedded CD sub-invocation: the paper's "else" branch of `w` reads `CD (E, F, e_i*)`);
- the broadcast value `w` (paper: "broadcasts the value `w`");
- the set `L` of correct-process ids (paper: "locally returns `L`, a list of ids of correct processes").

Deviation: the paper uses a list, we use a set. This loses nothing for the impossibility – only the SET of correct ids matters when we later say "`L = correct`" in the correctness condition. For the reduction R1, where the paper writes `min L`, we hard-wire a single `p_star` instead; see `Reductions.thy`.

```
record 'p bb_output =
  bb_F :: "'p history"
  bb_w :: bool
  bb_L :: "'p set"
```

Algorithmic signature: given the BB inputs (`P` the full process set, `V` the initial-value vector, `e_star` the local target event, `i` the invoking process), produce a `bb_output`.

```
type_synonym 'p bb_solver =
  "'p set ⇒ ('p ⇒ bool) ⇒ 'p event ⇒ 'p ⇒ 'p bb_output"
```

15 Correctness of Black_Box

Combining the paper's three demands on a Black_Box output:

1. `bb_F out` is a valid reconstruction of the adversary's `adv_E` (this is the embedded CD-correctness clause: "Solving Black_Box at `p_i` requires ... solving CD", paper Section 4.2);
2. `bb_w out` equals `w_value` as paper-defined, evaluated at the algorithm's own `bb_F out`;
3. `bb_L out` is exactly the set `C` of correct processes (paper: "Solving Black_Box at `p_i` requires identifying the set of correct processes").

```
definition bb_correct_output ::
  "'p set ⇒ 'p set ⇒ ('p ⇒ bool)
     ⇒ 'p history ⇒ 'p event ⇒ 'p
     ⇒ 'p bb_output ⇒ bool" where
  "bb_correct_output P C V E e_star i out ⟷
     valid E (bb_F out) e_star ∧
     bb_w out = w_value C V E (bb_F out) e_star ∧
     bb_L out = C"
```

A BB solver `alg` *solves* the BB problem for `P`, `C` when, for every initial vector `V` and every admissible adversary, the output is correct in the sense above.

```
definition solves_BlackBox ::
  "'p set ⇒ 'p set ⇒ 'p bb_solver ⇒ bool" where
  "solves_BlackBox P C alg ⟷
     (∀V adv. adversary_admissible C adv ⟶
        bb_correct_output P C V (adv_E adv)
          (adv_e_star adv) (adv_i adv)
          (alg P V (adv_e_star adv) (adv_i adv)))"

definition BlackBox_solvable :: "'p set ⇒ 'p set ⇒ bool" where
  "BlackBox_solvable P C ⟷ (∃alg. solves_BlackBox P C alg)"

end


theory Reductions
  imports BlackBox
begin
```

16 The trivial-execution gadget

To invoke the universal quantifier in `solves_BlackBox` and `produces_valid_F` (the validity-condition predicates over all admissible adversaries) we need at least one concrete admissible adversary to instantiate them at. This gadget supplies the simplest such adversary: a one-event history at a designated correct process `p_star`, with that single event playing the role of the target `e_star`.

This is a mechanisation artefact, not a step of the paper. Existing distributed-algorithm literature usually formulates BB as "runs correctly on the actual execution"; we need an "actual" execution the universal quantifier can range over, and the trivial one is sufficient for Validity.

```
definition trivial_history :: "'p ⇒ 'p history" where
  "trivial_history p_star ≡
     (λp. if p = p_star then [Internal p_star 1] else [])"

definition trivial_event :: "'p ⇒ 'p event" where
  "trivial_event p_star ≡ Internal p_star 1"

definition trivial_adversary :: "'p ⇒ 'p adversary" where
  "trivial_adversary p_star ≡
     (| adv_E = trivial_history p_star,
```

```
       adv_e_star = trivial_event p_star,
       adv_i = p_star ⟩"

lemma wf_history_local_one:
  shows "wf_history_local p_star [Internal p_star 1]"
  by (simp add: wf_history_local_def)

lemma wf_history_local_empty:
  shows "wf_history_local p []"
  by (simp add: wf_history_local_def)

lemma wf_history_trivial:
  shows "wf_history (trivial_history p_star)"
proof -
  have "⋀p. wf_history_local p (trivial_history p_star p)"
  proof -
    fix p
    show "wf_history_local p (trivial_history p_star p)"
    proof (cases "p = p_star")
      case True
      hence "trivial_history p_star p = [Internal p_star 1]"
        by (simp add: trivial_history_def)
      thus ?thesis
        using True wf_history_local_one by simp
    next
      case False
      hence "trivial_history p_star p = []"
        by (simp add: trivial_history_def)
      thus ?thesis
        using wf_history_local_empty by simp
    qed
  qed
  thus ?thesis by (simp add: wf_history_def)
qed

lemma events_of_trivial:
  shows "events_of (trivial_history p_star) = {Internal p_star 1}"
proof -
  have "events_of (trivial_history p_star)
          = (⋃p. set (trivial_history p_star p))"
    by (simp add: events_of_def)
  also have "... = {Internal p_star 1}"
    by (auto simp: trivial_history_def)
  finally show ?thesis .
qed

lemma adversary_admissible_trivial:
  assumes "p_star ∈ C"
  shows "adversary_admissible C (trivial_adversary p_star)"
proof -
  have eE: "adv_E (trivial_adversary p_star) = trivial_history p_star"
    by (simp add: trivial_adversary_def)
```

```
  have eI: "adv_i (trivial_adversary p_star) = p_star"
    by (simp add: trivial_adversary_def)
  have eS: "adv_e_star (trivial_adversary p_star) = trivial_event p_star"
    by (simp add: trivial_adversary_def)
  have "wf_history (adv_E (trivial_adversary p_star))"
    using wf_history_trivial eE by simp
  moreover have "adv_i (trivial_adversary p_star) ∈ C"
    using assms eI by simp
  moreover have
    "proc_of (adv_e_star (trivial_adversary p_star))
       = adv_i (trivial_adversary p_star)"
    by (simp add: eS eI trivial_event_def)
  moreover have
    "adv_e_star (trivial_adversary p_star)
       ∈ events_of (adv_E (trivial_adversary p_star))"
    by (simp add: eE eS events_of_trivial trivial_event_def)
  ultimately show ?thesis by (simp add: adversary_admissible_def)
qed
```

17 Reduction R1: Consensus reduces to BlackBox

Paper, Section 4.2 (proof of Theorem 3, step 2):

> "Now we give the reduction from Consensus to Black_Box. To solve Consensus(V) at (a correct process) p_i, we invoke Black_Box(V, E, F, e_i^*) locally (and likewise to solve Consensus(V) at (each process) p_j, invoke Black_Box(V, E, F, e_j^*) at each p_j). Each correct process computes min(L) from the locally returned list L and outputs as its consensus value the broadcast value that it receives from p_min(L) and terminates. The conditions of Consensus – Agreement, Validity, and Termination – can be seen to be satisfied. So Consensus $\preceq$ Black_Box."

Deviation: we do not compute `min(L)`. Since BB's correctness condition guarantees `bb_L = correct` – the set is the same at every correct process – every correct process resolves `p_min(L)` to the same value, so any fixed correct `p_star` serves in place of `min(L)`. We thread `p_star` as a constructor parameter.

```
definition consensus_from_bb ::
  "'p set ⇒ 'p ⇒ 'p bb_solver ⇒ 'p consensus_alg" where
  "consensus_from_bb P p_star bb_alg ≡
     (λV _. bb_w (bb_alg P V (trivial_event p_star) p_star))"

context byzantineSystem
begin
```

17.1 Agreement

Paper's first Consensus property (Section 4.2):

> "Agreement: All non-faulty processes must agree on the same single value."

In our construction every correct process reads the same `bb_w (bb_alg ...)` value (`consensus_from_bb` ignores its second argument). Agreement is immediate from this constancy.

```
lemma consensus_from_bb_agreement:
  fixes V :: "'p ⇒ bool"
  shows "consensus_agreement correct (consensus_from_bb procs p_star bb_alg) V"
proof -
  have const_in_i:
    "⋀p q. consensus_from_bb procs p_star bb_alg V p
              = consensus_from_bb procs p_star bb_alg V q"
    by (simp add: consensus_from_bb_def)
  show ?thesis by (simp add: consensus_agreement_def const_in_i)
qed
```

17.2 Validity

Paper's second Consensus property (Section 4.2):

> "Validity: If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value."

We get Validity by instantiating `solves_BlackBox` at the trivial adversary and reading off the uniform-case branches of `w_value`: if all correct processes have `V[p] = True`, the BB output is the `w_value_uniform_true` branch, which is `True`; similarly for `False`.

```
lemma bb_w_at_trivial:
  assumes bb:       "solves_BlackBox procs correct bb_alg"
      and p_star_C: "p_star ∈ correct"
  shows "bb_w (bb_alg procs V (trivial_event p_star) p_star)
           = w_value correct V
                     (trivial_history p_star)
                     (bb_F (bb_alg procs V (trivial_event p_star) p_star))
                     (trivial_event p_star)"
proof -
  let ?adv = "trivial_adversary p_star"
  have adm: "adversary_admissible correct ?adv"
    using p_star_C by (rule adversary_admissible_trivial)
  from bb have univ:
    "∀V adv. adversary_admissible correct adv ⟶
        bb_correct_output procs correct V
          (adv_E adv) (adv_e_star adv) (adv_i adv)
          (bb_alg procs V (adv_e_star adv) (adv_i adv))"
    by (simp add: solves_BlackBox_def)
  from univ adm have bb_inst:
    "bb_correct_output procs correct V
       (adv_E ?adv) (adv_e_star ?adv) (adv_i ?adv)
       (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv))"
    by blast
  hence
    "bb_w (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv))
       = w_value correct V (adv_E ?adv)
           (bb_F (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv)))
```

```
          (adv_e_star ?adv)"
    by (simp add: bb_correct_output_def)
  thus ?thesis
    by (simp add: trivial_adversary_def)
qed

lemma consensus_from_bb_validity:
  assumes bb:       "solves_BlackBox procs correct bb_alg"
      and p_star_C: "p_star ∈ correct"
  shows "consensus_validity correct (consensus_from_bb procs p_star bb_alg) V"
proof -
  let ?out = "bb_alg procs V (trivial_event p_star) p_star"
  have C_ne: "correct ≠ {}" using p_star_C by blast
  have bb_w_eq:
    "bb_w ?out
       = w_value correct V (trivial_history p_star)
                 (bb_F ?out) (trivial_event p_star)"
    by (rule bb_w_at_trivial[OF bb p_star_C])

  have case_True:
    "(∀p ∈ correct. V p)
       ⟶ (∀p ∈ correct. (consensus_from_bb procs p_star bb_alg) V p)"
  proof
    assume H: "∀p ∈ correct. V p"
    have "bb_w ?out = True"
      using bb_w_eq w_value_uniform_true[OF C_ne H] by simp
    thus "∀p ∈ correct. (consensus_from_bb procs p_star bb_alg) V p"
      by (simp add: consensus_from_bb_def)
  qed

  have case_False:
    "(∀p ∈ correct. ¬ V p)
       ⟶ (∀p ∈ correct. ¬ (consensus_from_bb procs p_star bb_alg) V p)"
  proof
    assume H: "∀p ∈ correct. ¬ V p"
    have "bb_w ?out = False"
      using bb_w_eq w_value_uniform_false[OF H] by simp
    thus "∀p ∈ correct. ¬ (consensus_from_bb procs p_star bb_alg) V p"
      by (simp add: consensus_from_bb_def)
  qed

  from case_True case_False
  show ?thesis by (simp add: consensus_validity_def)
qed
```

17.3 Termination is implicit in totality

Paper's third Consensus property (Section 4.2):

> "Termination: Each non-faulty process must eventually decide on a value."

Deviation (a load-bearing one): our `'p consensus_alg` is a total HOL function, so every correct process *always* returns a decision and Termination holds trivially. This trivi-

alisation of Termination is exactly what makes the abstract `solves_Consensus` predicate too weak to express FLP – see `Foundation_Vacuity.thy` – and is the reason the impossibility chain in `Impossibility.thy` goes through the FLP-style `flp_consensus_solvable` predicate of `FLP_Consensus.thy` rather than directly through `solves_Consensus`.

17.4 Composition: the headline reduction

```
theorem consensus_reduces_to_blackbox:
  assumes p_star_C: "p_star ∈ correct"
      and bb:       "solves_BlackBox procs correct bb_alg"
  shows "solves_Consensus correct (consensus_from_bb procs p_star bb_alg)"
proof -
  have agr: "⋀V. consensus_agreement correct
                   (consensus_from_bb procs p_star bb_alg) V"
    by (rule consensus_from_bb_agreement)
  have val: "⋀V. consensus_validity correct
                   (consensus_from_bb procs p_star bb_alg) V"
    by (rule consensus_from_bb_validity[OF bb p_star_C])
  show ?thesis using agr val by (simp add: solves_Consensus_def)
qed

corollary BlackBox_solvable_imp_Consensus_solvable:
  assumes ne: "correct ≠ {}" and bb: "BlackBox_solvable procs correct"
  shows "∃alg. solves_Consensus correct alg"
proof -
  obtain bb_alg where solver: "solves_BlackBox procs correct bb_alg"
    using bb by (auto simp: BlackBox_solvable_def)
  obtain p_star where pc: "p_star ∈ correct" using ne by blast
  have "solves_Consensus correct (consensus_from_bb procs p_star bb_alg)"
    by (rule consensus_reduces_to_blackbox[OF pc solver])
  thus ?thesis by blast
qed

end — context byzantineSystem
```

18 Reduction R2: BlackBox reduces to CD

Paper, Section 4.2 (proof of Theorem 3, step 1):

> "If there were an algorithm to make `F` match `E`, it requires identifying whether each of the processes that input their execution histories is correct or Byzantine, and tracing and dealing with / resolving the impact of contamination via message passing by the Byzantine processes from and through those Byzantine processes on the execution histories of processes at other processes. Thus, Black_Box $\preceq$ CD."

Deviation – meta-level step. The paper does not exhibit a syntactic construction of a Black_Box solver from a CD solver; the argument is meta-level ("it requires identifying ..."). We capture this faithfully as a single named locale assumption `cd_can_identify_correct` in the sub-locale `byzantineSystem_with_identification`:

> "If there is an algorithm `cd_alg` that produces a valid `F`, then there is an algorithm `cd_alg'` that (i) produces the same valid `F`, (ii) returns the decision `True`, and (iii) also returns the set of correct processes."

This is the positive form of the paper's contrapositive ("producing valid `F` is impossible without internally identifying the correct set"). From this assumption the rest of R2 is fully constructive: the BB solver projects the augmented CD solver's `L` field into `bb_L`, its `F` field into `bb_F`, and applies the piecewise definition of `w_value` to obtain `bb_w`.

```
context byzantineSystem
begin

type_synonym 'q cd_solver_with_L =
  "'q ⇒ 'q event ⇒ 'q history × bool × 'q set"
```

The augmented predicate insists on the three properties Misra–Kshemkalyani actually need from the meta-level step:

1. the collected `F` is valid in the sense of Definition 5;
2. the algorithm's claim `b` is `True`, matching Definition 5's "returning 1 indicates that the problem has been solved correctly";
3. the reported list `L` equals the correct set.

```
definition produces_valid_F_with_L ::
  "'p set ⇒ 'p cd_solver_with_L ⇒ bool" where
  "produces_valid_F_with_L C alg ⟷
     (∀adv. adversary_admissible C adv ⟶
        (let (F', b, L) = alg (adv_i adv) (adv_e_star adv) in
           valid (adv_E adv) F' (adv_e_star adv)
           ∧ b
           ∧ L = C))"

end — context byzantineSystem
```

Role of this locale in the development. The locale `byzantineSystem_with_identification` below extends `byzantineSystem` with the single named meta-level axiom `cd_can_identify_correct` – the positive form of the paper's contrapositive "producing valid F is impossible without identifying the correct set" (paper Section 4.2). Together with the constructive `bb_from_cd_with_L` defined below it gives R2 (BB $\preceq$ CD), which is what the paper actually proves to establish Theorems 3/4/5.

Note on critical path. After the discharge of the BB impossibility via Theorem 1 (`BlackBox_Unsolvable.thy`) and the direct Theorem-1 chain for the headline impossibility theorems (`Impossibility.thy`), the locale and its axiom are no longer on the critical path of Theorems 3/4/5: the headline theorems live in plain `byzantineSystem` and route through Theorem 1 in one step. The locale and R2 are nevertheless retained, as paper-faithful documentation of the ğ4.2 chain and as a fully-proven alternative derivation (composing R2 with `BlackBox_Unsolvable.thy` yields the same headline conclusion via the route the paper actually uses). This is the only locale axiom anywhere in the development; it is explicitly localised in this sub-locale rather than added to the base locale.

```
locale byzantineSystem_with_identification = byzantineSystem +
  assumes cd_can_identify_correct:
    "produces_valid_F correct cd_alg ⟹
       ∃cd_alg'. produces_valid_F_with_L correct cd_alg'
                 ∧ (∀i e. fst (cd_alg' i e) = fst (cd_alg i e))"

context byzantineSystem_with_identification
begin
```

18.1 Constructive part: BB from an augmented CD solver

Given an augmented CD solver (one that, by `produces_valid_F correct ?cd_alg ⟹ ∃cd_alg'. produces_valid_F_with_L correct cd_alg' ∧ (∀i e. fst (cd_alg' i e) = fst (?cd_alg i e))`, also reports the correct set), we build a Black_Box solver by simple projection:

- `bb_F` := the augmented CD solver's collected `F'`;
- `bb_L` := the augmented CD solver's reported `L` (which equals `correct` by assumption);
- `bb_w` := the paper's piecewise `w_value`, computed against `L` and the CD solver's boolean.

This matches the paper's reduction in Section 4.2: "Solving Black_Box at `p_i` requires identifying the set of correct processes and solving CD." The augmented CD solver does both; the projection just exposes its outputs in the BB record shape.

```
definition bb_from_cd_with_L ::
  "'p cd_solver_with_L ⇒ 'p bb_solver" where
  "bb_from_cd_with_L cd_alg' ≡
     (λP V e_star i.
        let (F', b, L) = cd_alg' i e_star in
        ⦇ bb_F = F',
          bb_w = (if (∀p ∈ L. ¬ V p) then False
                  else if (∀p ∈ L. V p) then True
                  else b),
          bb_L = L ⦈)"
```

18.2 Three sub-claims mirroring `bb_correct_output`

```
lemma bb_from_cd_with_L_correct:
  assumes augmented: "produces_valid_F_with_L correct cd_alg'"
  shows "solves_BlackBox procs correct (bb_from_cd_with_L cd_alg')"
proof (unfold solves_BlackBox_def, intro allI impI)
  fix V adv
  assume adm: "adversary_admissible correct adv"

  — Decompose the augmented CD solver's output.
  obtain F' b L where
    decomp: "cd_alg' (adv_i adv) (adv_e_star adv) = (F', b, L)"
    by (cases "cd_alg' (adv_i adv) (adv_e_star adv)") auto
```

```
  from augmented adm decomp
  have valid_F': "valid (adv_E adv) F' (adv_e_star adv)"
    and b_True:  "b"
    and L_eq:    "L = correct"
    by (auto simp: produces_valid_F_with_L_def Let_def)

  let ?out = "bb_from_cd_with_L cd_alg' procs V
                                (adv_e_star adv) (adv_i adv)"

  have F_field: "bb_F ?out = F'"
    by (simp add: bb_from_cd_with_L_def decomp)
  have L_field: "bb_L ?out = correct"
    by (simp add: bb_from_cd_with_L_def decomp L_eq)
  have w_field:
    "bb_w ?out =
       (if (∀p ∈ correct. ¬ V p) then False
        else if (∀p ∈ correct. V p) then True
        else b)"
    by (simp add: bb_from_cd_with_L_def decomp L_eq)
```

— Sub-claim 1: `F` is valid.

```
  have claim_valid:
    "valid (adv_E adv) (bb_F ?out) (adv_e_star adv)"
    using valid_F' F_field by simp
```

— Sub-claim 2: `bb_w` matches `w_value`. Three branches following the piecewise definition (Section 4.2): uniform-false, uniform-true, mixed.

```
  have claim_w:
    "bb_w ?out =
       w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv)"
  proof -
    consider
        (uniform_false) "∀p ∈ correct. ¬ V p"
      | (uniform_true)  "¬ (∀p ∈ correct. ¬ V p)" "∀p ∈ correct. V p"
      | (mixed)         "¬ (∀p ∈ correct. ¬ V p)"
                        "¬ (∀p ∈ correct. V p)"
      by blast
    thus ?thesis
    proof cases
      case uniform_false
      have "bb_w ?out = False"
        by (simp add: w_field uniform_false)
      moreover have
        "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv) = False"
        by (simp add: w_value_uniform_false[OF uniform_false])
      ultimately show ?thesis by simp
    next
      case uniform_true
      have C_ne: "correct ≠ {}" using uniform_true(2) uniform_true(1) by blast
      have inner_reduce:
        "(if (∀p ∈ correct. V p) then True else b) = True"
        using uniform_true(2) by simp
```

```
      have outer_reduce:
        "(if (∀p ∈ correct. ¬ V p) then False
          else if (∀p ∈ correct. V p) then True
          else b)
         = (if (∀p ∈ correct. V p) then True else b)"
        using uniform_true(1) by (rule if_not_P)
      have "bb_w ?out = True"
        using w_field outer_reduce inner_reduce by simp
      moreover have
        "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv) = True"
        by (rule w_value_uniform_true[OF C_ne uniform_true(2)])
      ultimately show ?thesis by simp
    next
      case mixed
      have "bb_w ?out = b"
        by (simp add: w_field mixed)
      hence bb_w_True: "bb_w ?out = True" using b_True by simp
      have
        "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv)
           = valid (adv_E adv) (bb_F ?out) (adv_e_star adv)"
        by (rule w_value_mixed[OF mixed(1) mixed(2)])
      also have
        "... = valid (adv_E adv) F' (adv_e_star adv)"
        by (simp add: F_field)
      also have "... = True" using valid_F' by simp
      finally show ?thesis using bb_w_True by simp
    qed
  qed

  — Sub-claim 3: bb_L equals the correct set.
  have claim_L: "bb_L ?out = correct" by (rule L_field)

  show "bb_correct_output procs correct V
          (adv_E adv) (adv_e_star adv) (adv_i adv) ?out"
    using claim_valid claim_w claim_L
    by (simp add: bb_correct_output_def)
qed

theorem blackbox_reduces_to_cd:
  assumes cd: "produces_valid_F correct cd_alg"
  shows "∃bb_alg. solves_BlackBox procs correct bb_alg"
proof -
  from cd_can_identify_correct[OF cd]
  obtain cd_alg' where
    aug: "produces_valid_F_with_L correct cd_alg'"
    by blast
  have "solves_BlackBox procs correct (bb_from_cd_with_L cd_alg')"
    by (rule bb_from_cd_with_L_correct[OF aug])
  thus ?thesis by blast
qed

corollary CD_solvable_imp_BlackBox_solvable:
```

```
  assumes "CD_solvable m correct"
  shows   "BlackBox_solvable procs correct"
proof -
  obtain cd_alg where "produces_valid_F correct cd_alg"
    using assms by (auto simp: CD_solvable_def)
  thus ?thesis
    using blackbox_reduces_to_cd by (auto simp: BlackBox_solvable_def)
qed

end — context byzantineSystem_with_identification

end


theory Theorems_1_2
  imports CD
begin
```

19 Natural numbers used by an algorithm's output

For a finitely-supported collected history `F`, the set of natural numbers appearing as either a local sequence number or a message identifier of any event in `events_of F` is finite. Hence a fresh natural number, strictly larger than all those that appear, exists.

```
fun nats_of_event :: "'p event ⇒ nat set" where
  "nats_of_event (Internal _ n)     = {n}"
| "nats_of_event (Send _ n _ m)     = {n, m}"
| "nats_of_event (Receive _ n _ m)  = {n, m}"

definition nats_in_F :: "'p history ⇒ nat set" where
  "nats_in_F F ≡ ⋃e∈events_of F. nats_of_event e"

definition fresh_nat :: "'p history ⇒ nat" where
  "fresh_nat F ≡ Suc (Max (insert 0 (nats_in_F F)))"

lemma finite_nats_of_event [simp]: "finite (nats_of_event e)"
  by (cases e) auto

lemma finite_nats_in_F:
  assumes "finite (events_of F)"
  shows   "finite (nats_in_F F)"
  using assms unfolding nats_in_F_def by simp

lemma nats_in_F_le_Max:
  assumes "finite (events_of F)" "k ∈ nats_in_F F"
  shows   "k ≤ Max (insert 0 (nats_in_F F))"
proof -
  have "finite (insert 0 (nats_in_F F))"
    using assms(1) finite_nats_in_F by simp
  thus ?thesis using assms(2) by (intro Max_ge) auto
qed
```

```
lemma fresh_nat_above:
  assumes "finite (events_of F)" "k ∈ nats_in_F F"
  shows   "k < fresh_nat F"
proof -
  have "k ≤ Max (insert 0 (nats_in_F F))"
    using assms by (rule nats_in_F_le_Max)
  thus ?thesis by (simp add: fresh_nat_def)
qed

lemma fresh_nat_positive [simp]: "fresh_nat F > 0"
  by (simp add: fresh_nat_def)
```

Concrete corollaries: any specific event constructed at `fresh_nat F` as either its sequence number or its message identifier is outside `events_of F`.

```
lemma Internal_at_fresh_nat_not_in_F:
  assumes "finite (events_of F)"
  shows   "Internal p (fresh_nat F) ∉ events_of F"
proof
  let ?e = "Internal p (fresh_nat F)"
  assume H: "?e ∈ events_of F"
  have inside: "fresh_nat F ∈ nats_of_event ?e" by simp
  have "fresh_nat F ∈ nats_in_F F"
    using H inside unfolding nats_in_F_def by blast
  with assms fresh_nat_above show False by fastforce
qed

lemma Send_at_fresh_nat_not_in_F:
  assumes "finite (events_of F)"
  shows   "Send p n q (fresh_nat F) ∉ events_of F"
proof
  let ?e = "Send p n q (fresh_nat F)"
  assume H: "?e ∈ events_of F"
  have inside: "fresh_nat F ∈ nats_of_event ?e" by simp
  have "fresh_nat F ∈ nats_in_F F"
    using H inside unfolding nats_in_F_def by blast
  with assms fresh_nat_above show False by fastforce
qed

context process_partition
begin
```

20 Theorem 1: false negatives are unavoidable

Paper, Theorem 1 (Section 4.1):

> "It is impossible to prevent false negatives in solving the causality determination problem (Definition 5) as specified by CD(`E, F, e_i^*`) in an asynchronous unicast/multicast/broadcast-based message passing system with one or more Byzantine processes."

Paper's proof sketch:

> "In the determination of `e_h^x` → `e_i^*`, a false negative may arise when a send-receive event pair (`e_f^u, e_g^v`) in a causal chain from `e_h^x` to `e_i^*` is missing as per `F`. ... A Byzantine `p_g` can suppress letting the rest of the system know of the occurrence of `e_g^v` or swap the order of occurrence of `e_g^v` and `e_g^v'` in what it lets the rest of the system know about the occurrence of the two local events."

Our construction. Given an algorithm `alg`, pick a correct process `p_i` and a Byzantine process `p_b`. Take `e_star = Internal p_i 2`, and let `F = fst (alg p_i e_star)` be the algorithm's collected history on this input. Use a fresh message id `m = fresh_nat F` and build the execution `E`:

- `E p_b = [Send p_b 1 p_i m]`;
- `E p_i = [Receive p_i 1 p_b m, Internal p_i 2]`;
- `E p = []` elsewhere.

The Send event has a hb-chain to `e_star` in `E` (message order plus program order at `p_i`), but it is not in `events_of F` because its message id was chosen fresh. Hence `hb_eval E s e_star` holds while `hb_eval F s e_star` does not – a false negative.

```
definition fn_E :: "'p ⇒ 'p ⇒ nat ⇒ 'p history" where
  "fn_E p_i p_b m ≡
     (λp. if p = p_b then [Send p_b 1 p_i m]
          else if p = p_i then [Receive p_i 1 p_b m, Internal p_i 2]
          else [])"

definition fn_adv :: "'p ⇒ 'p ⇒ nat ⇒ 'p adversary" where
  "fn_adv p_i p_b m ≡
     (| adv_E = fn_E p_i p_b m,
       adv_e_star = Internal p_i 2,
       adv_i = p_i |)"

lemma fn_E_at_pb [simp]:
  assumes "p_b ≠ p_i"
  shows "fn_E p_i p_b m p_b = [Send p_b 1 p_i m]"
  using assms by (simp add: fn_E_def)

lemma fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "fn_E p_i p_b m p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
  using assms by (auto simp: fn_E_def)

lemma fn_E_elsewhere:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "fn_E p_i p_b m p = []"
  using assms by (simp add: fn_E_def)

lemma fn_E_events:
  assumes "p_b ≠ p_i"
  shows "events_of (fn_E p_i p_b m) =
           {Send p_b 1 p_i m, Receive p_i 1 p_b m, Internal p_i 2}"
```

```
proof -
  have "events_of (fn_E p_i p_b m) = (⋃p. set (fn_E p_i p_b m p))"
    by (simp add: events_of_def)
  also have "... = set (fn_E p_i p_b m p_b) ∪ set (fn_E p_i p_b m p_i)
                 ∪ (⋃p ∈ -{p_b, p_i}. set (fn_E p_i p_b m p))"
    by auto
  also have "set (fn_E p_i p_b m p_b) = {Send p_b 1 p_i m}"
    using assms by simp
  moreover have "set (fn_E p_i p_b m p_i)
                   = {Receive p_i 1 p_b m, Internal p_i 2}"
    using assms by (simp add: fn_E_at_pi)
  moreover have "(⋃p ∈ -{p_b, p_i}. set (fn_E p_i p_b m p)) = {}"
    by (auto simp: fn_E_elsewhere)
  ultimately show ?thesis by auto
qed

lemma wf_history_local_pb_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_b (fn_E p_i p_b m p_b)"
proof -
  have list_eq: "fn_E p_i p_b m p_b = [Send p_b 1 p_i m]"
    using assms by simp
  have "∀e ∈ set [Send p_b 1 p_i m]. proc_of e = p_b" by simp
  moreover have "∀k < length [Send p_b 1 p_i m].
                   seq_of ([Send p_b 1 p_i m] ! k) = Suc k"
    by (simp add: less_Suc_eq)
  ultimately show ?thesis
    using list_eq unfolding wf_history_local_def by simp
qed

lemma wf_history_local_pi_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (fn_E p_i p_b m p_i)"
proof -
  let ?l = "[Receive p_i 1 p_b m, Internal p_i 2]"
  have list_eq: "fn_E p_i p_b m p_i = ?l"
    using assms by (rule fn_E_at_pi)
  have proc_ok: "∀e ∈ set ?l. proc_of e = p_i" by simp
  have seq_ok: "∀k < length ?l. seq_of (?l ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?l"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?l ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_fn_E:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "wf_history_local p (fn_E p_i p_b m p)"
  using assms by (simp add: fn_E_elsewhere wf_history_local_def)
```

```
lemma wf_history_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history (fn_E p_i p_b m)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (fn_E p_i p_b m p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_fn_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms wf_history_local_pi_in_fn_E by simp
    next
      case False
      with ‹p ≠ p_b› show ?thesis
        by (rule wf_history_local_elsewhere_in_fn_E)
    qed
  qed
qed

lemma program_order_fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "program_order (fn_E p_i p_b m)
           (Receive p_i 1 p_b m) (Internal p_i 2)"
proof -
  let ?H = "fn_E p_i p_b m"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
    using assms by (rule fn_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Internal p_i 2" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis
    using e0 e1 unfolding program_order_def by blast
qed

lemma message_order_fn_E_send_receive:
  assumes "p_b ≠ p_i"
  shows "message_order (fn_E p_i p_b m)
           (Send p_b 1 p_i m) (Receive p_i 1 p_b m)"
  using assms unfolding message_order_def
  by (simp add: fn_E_events)

lemma hb_send_to_estar_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
proof -
```

```
  let ?H = "fn_E p_i p_b m"
  let ?s = "Send p_b 1 p_i m"
  let ?r = "Receive p_i 1 p_b m"
  let ?es = "Internal p_i 2"
  have step1: "hb_step ?H ?s ?r"
    using message_order_fn_E_send_receive[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?r ?es"
    using program_order_fn_E_at_pi[OF assms] by (simp add: hb_step_def)
  have base: "(hb_step ?H)^++ ?s ?r"
    using step1 by blast
  have extend: "(hb_step ?H)^++ ?r ?es"
    using step2 by blast
  have "(hb_step ?H)^++ ?s ?es"
    using base extend by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)
qed

lemma hb_eval_send_to_estar_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb_eval (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
proof -
  have e1: "Send p_b 1 p_i m ∈ events_of (fn_E p_i p_b m)"
    using assms by (simp add: fn_E_events)
  have e2: "Internal p_i 2 ∈ events_of (fn_E p_i p_b m)"
    using assms by (simp add: fn_E_events)
  have h: "hb (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
    using assms by (rule hb_send_to_estar_in_fn_E)
  show ?thesis using e1 e2 h by (simp add: hb_eval_def)
qed

lemma adv_admissible_fn_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i"
  shows "adversary_admissible correct (fn_adv p_i p_b m)"
proof -
  have a: "wf_history (fn_E p_i p_b m)"
    using assms(2) by (rule wf_history_fn_E)
  have b: "p_i ∈ correct" by (rule assms(1))
  have c: "proc_of (Internal p_i 2) = p_i" by simp
  have d: "Internal p_i 2 ∈ events_of (fn_E p_i p_b m)"
    using assms(2) by (simp add: fn_E_events)
  show ?thesis using a b c d
    by (simp add: adversary_admissible_def fn_adv_def)
qed

theorem CD_FN_unavoidable:
  assumes byz_cor_distinct:
                "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
      and fin_F:  "∀p_i_in. finite (events_of
                                    (fst (alg p_i_in (Internal p_i_in 2))))"
  shows "∃adv. adversary_admissible correct adv ∧
              false_negative (adv_E adv)
```

```
                              (fst (alg (adv_i adv) (adv_e_star adv)))
                              (adv_e_star adv)"
proof -
  obtain p_i p_b where
      pi_cor: "p_i ∈ correct" and pb_byz: "p_b ∈ byzantine"
      and dist: "p_b ≠ p_i"
    using byz_cor_distinct by blast

  let ?e_star = "Internal p_i 2"
  let ?F = "fst (alg p_i ?e_star)"
  let ?m = "fresh_nat ?F"
  let ?adv = "fn_adv p_i p_b ?m"
  let ?s = "Send p_b 1 p_i ?m"

  have finF: "finite (events_of ?F)" using fin_F by blast

  have adm: "adversary_admissible correct ?adv"
    using pi_cor dist by (rule adv_admissible_fn_adv)

  have witness_in_E: "?s ∈ events_of (adv_E ?adv)"
    using dist by (simp add: fn_adv_def fn_E_events)
  have hbE: "hb_eval (adv_E ?adv) ?s (adv_e_star ?adv)"
  proof -
    have "hb_eval (fn_E p_i p_b ?m) ?s (Internal p_i 2)"
      by (rule hb_eval_send_to_estar_in_fn_E[OF dist])
    thus ?thesis by (simp add: fn_adv_def)
  qed
  have witness_not_in_F: "?s ∉ events_of ?F"
    using finF Send_at_fresh_nat_not_in_F[of ?F] by blast
  have not_hbF: "¬ hb_eval ?F ?s (adv_e_star ?adv)"
    using witness_not_in_F by (simp add: hb_eval_def fn_adv_def)

  from witness_in_E hbE not_hbF
  have "∃e ∈ events_of (adv_E ?adv) ∪ events_of ?F.
            hb_eval (adv_E ?adv) e (adv_e_star ?adv) ∧
            ¬ hb_eval ?F e (adv_e_star ?adv)"
    by blast
  hence FN: "false_negative (adv_E ?adv) ?F (adv_e_star ?adv)"
    by (simp add: false_negative_def)

  have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
    by (simp add: fn_adv_def)

  from adm FN F_eq show ?thesis by metis
qed
```

21 Theorem 2: false negatives or false positives are unavoidable for internal events

Paper, Theorem 2 (Section 4.1):

> "For an internal event `e_h^x`, it is impossible to prevent false negatives or

false positives in determining e_h^x → e_i^* at a correct process p_i in an asynchronous message passing system with one or more Byzantine processes."

Paper's proof sketch:

"There may be no other event in the rest of the system to corroborate the occurrence of an internal event at a process. A Byzantine process p_h can choose to not reveal about an internal event e_h^x to the rest of the system, leading to a false negative that cannot be prevented. It may also choose to add a fake internal event e_h^x in what it reveals to the rest of the system, leading to a false positive that cannot be prevented."

Our construction. We strengthen the Theorem 1 construction so that the witness event is an Internal event at a Byzantine process, rather than a Send event. The Byzantine process p_b performs a chain of k internal events followed by a single Send to the correct target p_i, where k = fresh_nat F is chosen so that the k-th internal event has a sequence number absent from events_of F. The internal event still has an hb-chain to e_star in E (through subsequent program order to the Send, then message order, then program order at p_i); but the internal event itself is not in events_of F, so a false negative arises.

Deviation: the paper's Theorem 2 is an FN-or-FP disjunction ("the Byzantine p_h can choose to not reveal ... OR ... add a fake internal event"). We discharge the FN side constructively. Since the statement is a disjunction (FN ∨ FP), exhibiting an adversary that produces FN is sufficient.

```
definition fn_internal_E :: "'p ⇒ 'p ⇒ nat ⇒ nat ⇒ 'p history" where
  "fn_internal_E p_i p_b k m ≡
     (λp. if p = p_b
          then (map (Internal p_b) [1..<Suc k])
                @ [Send p_b (Suc k) p_i m]
          else if p = p_i
          then [Receive p_i 1 p_b m, Internal p_i 2]
          else [])"

definition fn_internal_adv :: "'p ⇒ 'p ⇒ nat ⇒ nat ⇒ 'p adversary" where
  "fn_internal_adv p_i p_b k m ≡
     (| adv_E = fn_internal_E p_i p_b k m,
       adv_e_star = Internal p_i 2,
       adv_i = p_i |)"

lemma fn_internal_E_at_pb [simp]:
  assumes "p_b ≠ p_i"
  shows "fn_internal_E p_i p_b k m p_b
           = (map (Internal p_b) [1..<Suc k]) @ [Send p_b (Suc k) p_i m]"
  using assms by (simp add: fn_internal_E_def)

lemma fn_internal_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "fn_internal_E p_i p_b k m p_i
           = [Receive p_i 1 p_b m, Internal p_i 2]"
  using assms by (auto simp: fn_internal_E_def)
```

```
lemma fn_internal_E_elsewhere:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "fn_internal_E p_i p_b k m p = []"
  using assms by (simp add: fn_internal_E_def)

lemma length_fn_internal_E_pb:
  assumes "p_b ≠ p_i"
  shows "length (fn_internal_E p_i p_b k m p_b) = Suc k"
  using assms by simp

lemma nth_fn_internal_E_pb_lt:
  assumes "p_b ≠ p_i" "j < k"
  shows "fn_internal_E p_i p_b k m p_b ! j = Internal p_b (Suc j)"
proof -
  let ?xs = "map (Internal p_b) [1..<Suc k]"
  let ?L = "?xs @ [Send p_b (Suc k) p_i m]"
  have list_eq: "fn_internal_E p_i p_b k m p_b = ?L"
    using assms(1) by simp
  have len_xs: "length ?xs = k"
    using assms(2) by simp
  with assms(2) have j_lt_len: "j < length ?xs" by simp
  have j_lt_upt: "j < length [1..<Suc k]"
    using assms(2) by simp
  have step1: "?L ! j = ?xs ! j"
    using j_lt_len by (simp add: nth_append)
  have step2: "?xs ! j = Internal p_b ([1..<Suc k] ! j)"
    using j_lt_upt by (rule nth_map)
  have step3: "[1..<Suc k] ! j = Suc j"
  proof -
    have "[1..<Suc k] ! j = 1 + j"
      using assms(2) by (intro nth_upt) simp
    thus ?thesis by simp
  qed
  from step1 step2 step3 list_eq show ?thesis by simp
qed

lemma nth_fn_internal_E_pb_eq:
  assumes "p_b ≠ p_i"
  shows "fn_internal_E p_i p_b k m p_b ! k = Send p_b (Suc k) p_i m"
  using assms by (simp add: nth_append)

lemma set_fn_internal_E_pb:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "set (fn_internal_E p_i p_b k m p_b)
           = (Internal p_b ‘ {1..k}) ∪ {Send p_b (Suc k) p_i m}"
proof -
  have set_map: "set (map (Internal p_b) [1..<Suc k]) = Internal p_b ‘ {1..<Suc k}"
    by auto
  have rng_eq: "{1..<Suc k} = {1..k}" by auto
  have "set (fn_internal_E p_i p_b k m p_b)
          = set (map (Internal p_b) [1..<Suc k]) ∪ {Send p_b (Suc k) p_i m}"
```

```
    using assms(1) by simp
  also have "... = (Internal p_b ` {1..k}) ∪ {Send p_b (Suc k) p_i m}"
    using set_map rng_eq by simp
  finally show ?thesis .
qed

lemma fn_internal_E_events:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "events_of (fn_internal_E p_i p_b k m) =
           (Internal p_b ` {1..k})
           ∪ {Send p_b (Suc k) p_i m,
              Receive p_i 1 p_b m,
              Internal p_i 2}"
proof -
  have "events_of (fn_internal_E p_i p_b k m)
          = (⋃p. set (fn_internal_E p_i p_b k m p))"
    by (simp add: events_of_def)
  also have "... = set (fn_internal_E p_i p_b k m p_b)
                 ∪ set (fn_internal_E p_i p_b k m p_i)
                 ∪ (⋃p ∈ -{p_b, p_i}. set (fn_internal_E p_i p_b k m p))"
    by auto
  also have "(⋃p ∈ -{p_b, p_i}. set (fn_internal_E p_i p_b k m p)) = {}"
    by (auto simp: fn_internal_E_elsewhere)
  moreover have "set (fn_internal_E p_i p_b k m p_b)
                  = (Internal p_b ` {1..k}) ∪ {Send p_b (Suc k) p_i m}"
    using assms by (rule set_fn_internal_E_pb)
  moreover have "set (fn_internal_E p_i p_b k m p_i)
                  = {Receive p_i 1 p_b m, Internal p_i 2}"
    using assms(1) by (simp add: fn_internal_E_at_pi)
  ultimately show ?thesis by auto
qed

lemma wf_history_local_pb_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "wf_history_local p_b (fn_internal_E p_i p_b k m p_b)"
proof -
  let ?L = "fn_internal_E p_i p_b k m p_b"
  have len_L: "length ?L = Suc k" using assms(1) by simp

  have all_proc: "∀e ∈ set ?L. proc_of e = p_b"
  proof
    fix e assume "e ∈ set ?L"
    hence "e ∈ (Internal p_b ` {1..k}) ∪ {Send p_b (Suc k) p_i m}"
      using set_fn_internal_E_pb[OF assms] by simp
    thus "proc_of e = p_b" by (auto simp: image_iff)
  qed

  have all_seq: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)
    fix j assume "j < length ?L"
    hence j_lt: "j < Suc k" using len_L by simp
    show "seq_of (?L ! j) = Suc j"
```

```
    proof (cases "j < k")
      case True
      thus ?thesis using nth_fn_internal_E_pb_lt[OF assms(1)] by simp
    next
      case False
      with j_lt have "j = k" by simp
      thus ?thesis using nth_fn_internal_E_pb_eq[OF assms(1)] by simp
    qed
  qed

  show ?thesis using all_proc all_seq
    unfolding wf_history_local_def by blast
qed

lemma wf_history_local_pi_in_fn_internal_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (fn_internal_E p_i p_b k m p_i)"
proof -
  let ?L = "[Receive p_i 1 p_b m, Internal p_i 2]"
  have list_eq: "fn_internal_E p_i p_b k m p_i = ?L"
    using assms by (rule fn_internal_E_at_pi)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_i" by simp
  have seq_ok: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)
    fix j assume "j < length ?L"
    hence "j = 0 ∨ j = 1" by auto
    thus "seq_of (?L ! j) = Suc j" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_fn_internal_E:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "wf_history_local p (fn_internal_E p_i p_b k m p)"
  using assms by (simp add: fn_internal_E_elsewhere wf_history_local_def)

lemma wf_history_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "wf_history (fn_internal_E p_i p_b k m)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (fn_internal_E p_i p_b k m p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_fn_internal_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms(1) wf_history_local_pi_in_fn_internal_E by simp
```

```
    next
      case False
      with ‹p ≠ p_b› show ?thesis
        by (rule wf_history_local_elsewhere_in_fn_internal_E)
    qed
  qed
qed

lemma program_order_internal_to_send_at_pb:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "program_order (fn_internal_E p_i p_b k m)
           (Internal p_b k) (Send p_b (Suc k) p_i m)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  have len: "length (?H p_b) = Suc k" using assms(1) by simp
  from assms(2) have km1_lt_k: "k - 1 < k" by simp
  have at_km1: "(?H p_b) ! (k - 1) = Internal p_b (Suc (k - 1))"
    using assms(1) km1_lt_k nth_fn_internal_E_pb_lt by simp
  hence at_km1': "(?H p_b) ! (k - 1) = Internal p_b k"
    using assms(2) by simp
  have at_k: "(?H p_b) ! k = Send p_b (Suc k) p_i m"
    using assms(1) by (rule nth_fn_internal_E_pb_eq)
  have idx1: "k - 1 < k" using assms(2) by simp
  have idx2: "k < length (?H p_b)" using len by simp
  show ?thesis using at_km1' at_k idx1 idx2
    unfolding program_order_def by blast
qed

lemma program_order_receive_to_estar_in_fn_internal_E:
  assumes "p_b ≠ p_i"
  shows "program_order (fn_internal_E p_i p_b k m)
           (Receive p_i 1 p_b m) (Internal p_i 2)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
    using assms by (rule fn_internal_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Internal p_i 2" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis using e0 e1
    unfolding program_order_def by blast
qed

lemma message_order_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "message_order (fn_internal_E p_i p_b k m)
           (Send p_b (Suc k) p_i m) (Receive p_i 1 p_b m)"
  using assms unfolding message_order_def
  by (simp add: fn_internal_E_events)
```

```
lemma hb_internal_to_estar_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "hb (fn_internal_E p_i p_b k m) (Internal p_b k) (Internal p_i 2)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  let ?ih = "Internal p_b k"
  let ?s = "Send p_b (Suc k) p_i m"
  let ?r = "Receive p_i 1 p_b m"
  let ?es = "Internal p_i 2"
  have step1: "hb_step ?H ?ih ?s"
    using program_order_internal_to_send_at_pb[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?s ?r"
    using message_order_in_fn_internal_E[OF assms]
    by (simp add: hb_step_def)
  have step3: "hb_step ?H ?r ?es"
    using program_order_receive_to_estar_in_fn_internal_E[OF assms(1)]
    by (simp add: hb_step_def)
  have t1: "(hb_step ?H)^++ ?ih ?s" using step1 by blast
  have t2: "(hb_step ?H)^++ ?s ?r" using step2 by blast
  have t3: "(hb_step ?H)^++ ?r ?es" using step3 by blast
  have "(hb_step ?H)^++ ?ih ?r" using t1 t2 by (rule tranclp_trans)
  hence "(hb_step ?H)^++ ?ih ?es" using t3 by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)
qed

lemma hb_eval_internal_to_estar_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "hb_eval (fn_internal_E p_i p_b k m)
           (Internal p_b k) (Internal p_i 2)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  have e_ih: "Internal p_b k ∈ events_of ?H"
    using assms by (simp add: fn_internal_E_events)
  have e_es: "Internal p_i 2 ∈ events_of ?H"
    using assms by (simp add: fn_internal_E_events)
  have h: "hb ?H (Internal p_b k) (Internal p_i 2)"
    using assms by (rule hb_internal_to_estar_in_fn_internal_E)
  show ?thesis using e_ih e_es h by (simp add: hb_eval_def)
qed

lemma adv_admissible_fn_internal_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i" "k ≥ 1"
  shows "adversary_admissible correct (fn_internal_adv p_i p_b k m)"
proof -
  have a: "wf_history (fn_internal_E p_i p_b k m)"
    using assms(2) assms(3) by (rule wf_history_fn_internal_E)
  have b: "p_i ∈ correct" by (rule assms(1))
  have c: "proc_of (Internal p_i 2) = p_i" by simp
  have d: "Internal p_i 2 ∈ events_of (fn_internal_E p_i p_b k m)"
    using assms(2) assms(3) by (simp add: fn_internal_E_events)
  show ?thesis using a b c d
```

```
    by (simp add: adversary_admissible_def fn_internal_adv_def)
qed

theorem CD_FN_or_FP_unavoidable_internal:
  assumes byz_cor_distinct:
                "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
      and fin_F:  "∀p_i_in. finite (events_of
                                    (fst (alg p_i_in (Internal p_i_in 2))))"
  shows "∃adv e_h. adversary_admissible correct adv ∧
                   (∃p n. e_h = Internal p n) ∧
                   ((hb_eval (adv_E adv) e_h (adv_e_star adv) ∧
                     ¬ hb_eval (fst (alg (adv_i adv) (adv_e_star adv)))
                               e_h (adv_e_star adv))
                    ∨
                    (¬ hb_eval (adv_E adv) e_h (adv_e_star adv) ∧
                     hb_eval (fst (alg (adv_i adv) (adv_e_star adv)))
                             e_h (adv_e_star adv)))"
proof -
  obtain p_i p_b where
      pi_cor: "p_i ∈ correct" and pb_byz: "p_b ∈ byzantine"
      and dist: "p_b ≠ p_i"
    using byz_cor_distinct by blast

  let ?e_star = "Internal p_i 2"
  let ?F = "fst (alg p_i ?e_star)"
  let ?k = "fresh_nat ?F"
  let ?m = "fresh_nat ?F"
  let ?adv = "fn_internal_adv p_i p_b ?k ?m"
  let ?ih = "Internal p_b ?k"

  have finF: "finite (events_of ?F)" using fin_F by blast

  have k_pos: "?k ≥ 1"
    by (simp add: fresh_nat_def Suc_leI)

  have adm: "adversary_admissible correct ?adv"
    using pi_cor dist k_pos by (rule adv_admissible_fn_internal_adv)

  have witness_in_E: "?ih ∈ events_of (adv_E ?adv)"
    using dist k_pos
    by (auto simp: fn_internal_adv_def fn_internal_E_events)
  have hbE: "hb_eval (adv_E ?adv) ?ih (adv_e_star ?adv)"
  proof -
    have "hb_eval (fn_internal_E p_i p_b ?k ?m) ?ih (Internal p_i 2)"
      by (rule hb_eval_internal_to_estar_in_fn_internal_E[OF dist k_pos])
    thus ?thesis by (simp add: fn_internal_adv_def)
  qed
  have witness_not_in_F: "?ih ∉ events_of ?F"
    using finF Internal_at_fresh_nat_not_in_F[of ?F] by blast
  have not_hbF: "¬ hb_eval ?F ?ih (adv_e_star ?adv)"
    using witness_not_in_F by (simp add: hb_eval_def fn_internal_adv_def)
```

```
  have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
    by (simp add: fn_internal_adv_def)

  have e_h_internal: "∃p n. ?ih = Internal p n" by blast

  from adm e_h_internal hbE not_hbF F_eq witness_in_E
  show ?thesis by metis
qed

end — context process_partition

end


theory BlackBox_Unsolvable
  imports BlackBox Theorems_1_2
begin

context byzantineSystem
begin

theorem BlackBox_unsolvable:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_bb_F:
        "∀alg p_i_in.
           solves_BlackBox procs correct alg ⟶
             finite (events_of
                       (bb_F (alg procs (λ_. False)
                                   (Internal p_i_in 2) p_i_in)))"
  shows "¬ BlackBox_solvable procs correct"
proof
  assume BB: "BlackBox_solvable procs correct"
  then obtain bb_alg :: "'p bb_solver"
    where solver: "solves_BlackBox procs correct bb_alg"
    by (auto simp: BlackBox_solvable_def)

  — Project the BB-solver onto a CD-solver: take F from bb_F at the fixed input vector (λ_. False), and report decision True.
  define cd :: "'p cd_solver" where
    "cd ≡ (λi e. (bb_F (bb_alg procs (λ_. False) e i), True))"

  have cd_fst:
    "⋀i e. fst (cd i e) = bb_F (bb_alg procs (λ_. False) e i)"
    by (simp add: cd_def)

  — Step 1: the projected CD-solver produces valid F.
  have valid_cd: "produces_valid_F correct cd"
  proof (unfold produces_valid_F_def, intro allI impI)
    fix adv :: "'p adversary"
    assume adm: "adversary_admissible correct adv"
    from solver adm have
```

```
    "bb_correct_output procs correct (λ_. False)
       (adv_E adv) (adv_e_star adv) (adv_i adv)
       (bb_alg procs (λ_. False) (adv_e_star adv) (adv_i adv))"
    by (auto simp: solves_BlackBox_def)
  hence valid_at:
    "valid (adv_E adv)
           (bb_F (bb_alg procs (λ_. False)
                         (adv_e_star adv) (adv_i adv)))
           (adv_e_star adv)"
    by (simp add: bb_correct_output_def)
  show "let (F', _) = cd (adv_i adv) (adv_e_star adv) in
          valid (adv_E adv) F' (adv_e_star adv)"
    using valid_at by (simp add: cd_def Let_def)
qed

— Step 2: package the byz_cor_distinct hypothesis Theorem 1 wants.
have byz_cor_distinct:
  "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
proof -
  obtain p_i where pi: "p_i ∈ correct" using cor_ne by blast
  obtain p_b where pb: "p_b ∈ byzantine" using byz_ne by blast
  have "p_i ≠ p_b"
    using pi pb partition_disj by blast
  thus ?thesis using pi pb by blast
qed

— Step 3: the projected CD-solver satisfies the finiteness side condition Theorem 1 wants.
have fin_cd:
  "∀p_i_in. finite (events_of (fst (cd p_i_in (Internal p_i_in 2))))"
proof
  fix p_i_in :: 'p
  have "fst (cd p_i_in (Internal p_i_in 2))
          = bb_F (bb_alg procs (λ_. False) (Internal p_i_in 2) p_i_in)"
    by (rule cd_fst)
  moreover have
    "finite (events_of
               (bb_F (bb_alg procs (λ_. False)
                             (Internal p_i_in 2) p_i_in)))"
    using fin_bb_F solver by blast
  ultimately show
    "finite (events_of (fst (cd p_i_in (Internal p_i_in 2))))"
    by simp
qed

— Step 4: Theorem 1 says no CD-solver avoids false negatives.
obtain adv :: "'p adversary" where
  adm: "adversary_admissible correct adv"
  and FN: "false_negative (adv_E adv)
                          (fst (cd (adv_i adv) (adv_e_star adv)))
                          (adv_e_star adv)"
  using CD_FN_unavoidable[where alg = cd, OF byz_cor_distinct fin_cd]
  by blast
```

```
  — Step 5: false negative contradicts produces_valid_F.
  have valid_at_adv:
    "valid (adv_E adv)
           (fst (cd (adv_i adv) (adv_e_star adv)))
           (adv_e_star adv)"
    using valid_cd adm
    by (auto simp: produces_valid_F_def Let_def split: prod.split)
  hence no_FN:
    "¬ false_negative (adv_E adv)
                      (fst (cd (adv_i adv) (adv_e_star adv)))
                      (adv_e_star adv)"
    using valid_iff_no_FP_FN by blast
  from FN no_FN show False by contradiction
qed

end — context byzantineSystem

end


theory FLP_Consensus
  imports
    BlackBox
    FLP.AsynchronousSystem
    FLP.Execution
    FLP.FLPSystem
    FLP.FLPTheorem
begin
```

22 FLP-style consensus solvability

Paper, Section 4.2 (the Consensus problem):

> "In the Consensus problem, each process has an initial value and all correct processes must agree on a single value. The solution needs to satisfy the following three conditions.
>
> - **Agreement:** All non-faulty processes must agree on the same single value.
> - **Validity:** If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value.
> - **Termination:** Each non-faulty process must eventually decide on a value.
>
> According to the FLP impossibility result, it is impossible to solve Consensus in an asynchronous message-passing system with even a single crash failure prone process."

An algorithm in the FLP model (as formalised by the AFP entry) is a triple `(transFn, sendsFn, startFn)` of a transition function, a send function, and an initial-state function. We say the triple `flp-solves consensus` when it satisfies the AFP entry's `flpSystem` and `flpPseudoConsensus` locale axioms together with the three correctness conditions that the AFP entry's `flpPseudoConsensus.ConsensusFails` takes as preconditions:

- `flpSystem.terminationFLP`: every fair infinite execution terminates (paper's Termination, formalised in AFP);
- `flpSystem.validity`: validity (paper's Validity);
- `flpSystem.agreementInit`: agreement on initial configurations (paper's Agreement).

```
definition flp_consensus_solvable ::
  "('p ⇒ 's ⇒ 'v messageValue ⇒ 's)
   ⇒ ('p ⇒ 's ⇒ 'v messageValue ⇒ ('p, 'v) message multiset)
   ⇒ ('p ⇒ 's)
   ⇒ bool" where
  "flp_consensus_solvable transFn sendsFn startFn ⟷
     flpSystem sendsFn ∧
     flpPseudoConsensus transFn sendsFn startFn ∧
     (∀ fe ft. asynchronousSystem.fairInfiniteExecution transFn sendsFn startFn fe
ft
                 ⟶ flpSystem.terminationFLP transFn sendsFn startFn fe ft) ∧
     (∀ i c. flpSystem.validity transFn sendsFn startFn i c) ∧
     (∀ i c. flpSystem.agreementInit transFn sendsFn startFn i c)"
```

The FLP impossibility, on this predicate, is a one-step invocation of the AFP entry's `flpPseudoConsensus.ConsensusFails` theorem (proven, not assumed):

> "theorem ConsensusFails: assumes Termination: ... and Validity: ... and Agreement: ... shows False"

This is the genuine FLP result. We unpack the predicate into its five conjuncts, interpret `flpPseudoConsensus`, and apply `ConsensusFails`.

```
theorem flp_consensus_unsolvable:
  shows "¬ flp_consensus_solvable transFn sendsFn startFn"
proof
  assume A: "flp_consensus_solvable transFn sendsFn startFn"
  hence sys: "flpSystem sendsFn"
    and pc:  "flpPseudoConsensus transFn sendsFn startFn"
    and termFLP:
           "⋀fe ft. asynchronousSystem.fairInfiniteExecution
                       transFn sendsFn startFn fe ft ⟹
                  flpSystem.terminationFLP transFn sendsFn startFn fe ft"
    and val: "∀i c. flpSystem.validity transFn sendsFn startFn i c"
    and agr: "∀i c. flpSystem.agreementInit transFn sendsFn startFn i c"
    by (auto simp: flp_consensus_solvable_def)

  interpret flpPseudoConsensus transFn sendsFn startFn using pc .

  have "False"
```

```
    using flpPseudoConsensus.ConsensusFails[OF pc termFLP val agr] .
  thus False .
qed

end


theory Impossibility
  imports Reductions BlackBox_Unsolvable FLP_Consensus
begin

context byzantineSystem
begin
```

23 The direct Theorem-1 chain

Helper: `byzantine` $\neq$ `{}` and `correct` $\neq$ `{}` together yield the distinct-pair witness Theorem 1 wants (the locale's `partition_disj` separates the two sets).

```
lemma byz_cor_distinct_of_ne:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
  shows "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
proof -
  obtain p_i where pi: "p_i ∈ correct" using cor_ne by blast
  obtain p_b where pb: "p_b ∈ byzantine" using byz_ne by blast
  have "p_i ≠ p_b" using pi pb partition_disj by blast
  thus ?thesis using pi pb by blast
qed
```

The direct CD-impossibility lemma: given a finiteness side condition on every candidate solver's output history, Theorem 1 directly contradicts the existence of a `produces_valid_F` algorithm.

```
lemma no_produces_valid_F:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ (∃cd_alg. produces_valid_F correct cd_alg)"
proof
  assume "∃cd_alg. produces_valid_F correct cd_alg"
  then obtain cd_alg where val_F: "produces_valid_F correct cd_alg" by blast

  have byz_cor_distinct:
    "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
    by (rule byz_cor_distinct_of_ne[OF byz_ne cor_ne])

  have fin_F:
    "∀p_i_in. finite (events_of (fst (cd_alg p_i_in (Internal p_i_in 2))))"
    using fin_cd val_F by blast
```

```
  obtain adv where
    adm: "adversary_admissible correct adv"
    and FN: "false_negative (adv_E adv)
                            (fst (cd_alg (adv_i adv) (adv_e_star adv)))
                            (adv_e_star adv)"
    using CD_FN_unavoidable[where alg = cd_alg,
                            OF byz_cor_distinct fin_F]
    by blast

  have valid_at:
    "valid (adv_E adv)
           (fst (cd_alg (adv_i adv) (adv_e_star adv)))
           (adv_e_star adv)"
    using val_F adm
    by (auto simp: produces_valid_F_def Let_def split: prod.split)
  hence "¬ false_negative (adv_E adv)
                         (fst (cd_alg (adv_i adv) (adv_e_star adv)))
                         (adv_e_star adv)"
    using valid_iff_no_FP_FN by blast
  with FN show False by contradiction
qed
```

24 Theorem 3: CD impossible under unicast

Paper, Theorem 3 (Section 4.2):

> "It is impossible to solve causality determination (Definition 5) as specified by CD(`E, F, e_i^*`) in an asynchronous unicast-based message passing system with one or more Byzantine processes."

The paper's proof composes the two reductions of Section 4.2 with FLP's impossibility:

> "Transitivity of reductions implies that if the CD problem is solvable, then Consensus is also solvable. However, that contradicts the FLP impossibility result [35] when applied to a Byzantine system, hence CD cannot be solvable."

Deviation – the paper's chain is bypassed. Theorem 1 (`CD_FN_unavoidable`) already gives the impossibility directly: it exhibits, for any candidate CD-solver with finite output histories, an admissible adversary that defeats it. Composing that with the definition of `CD_solvable` (which existentially quantifies the solver) immediately yields the impossibility, without needing to route through the BlackBox-and-FLP detour the paper uses.

The chain in this theory is therefore:

> `CD_solvable` ⟶ ∃ `cd_alg. produces_valid_F correct cd_alg` ⟶ `False` (by Theorem 1, under `fin_cd`).

The paper's chain is preserved as paper-faithful documentation in `Reductions.thy` (R1, R2) and `BlackBox_Unsolvable.thy` (the discharge of ¬ `BlackBox_solvable` via Theorem 1), but is not on the critical path of the headline impossibility theorems below.

```
theorem CD_impossible_unicast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Unicast correct"
  using no_produces_valid_F[OF byz_ne cor_ne fin_cd]
  by (auto simp: CD_solvable_def)
```

25 Theorem 4: CD impossible under broadcast

Paper, Theorem 4 (Section 4.2):

> "It is impossible to solve causality determination (Definition 5) as specified by CD(`E, F, e_i^*`) in an asynchronous broadcast-based message passing system with one or more Byzantine processes."

The paper's proof of Theorem 4 "has the overall structure along the lines of that for Theorem 3". Its two differences:

1. "By doing broadcasts using the Byzantine Reliable Broadcast (BRB) [...] layer, false positives can be prevented by ensuring no fake events/causal dependencies are added to `F`."

2. "False negatives still cannot be prevented (Theorem 1 carries over)."

Deviation: we do not formalise BRB. Theorem 4 collapses to Theorem 3 at this abstraction level because `CD_solvable` is mode-agnostic. A richer development that adds BRB would refine our `Broadcast` predicate and re-state Theorem 4 with BRB as a hypothesis, but the conclusion – "CD is unsolvable" – is identical.

```
theorem CD_impossible_broadcast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Broadcast correct"
  using no_produces_valid_F[OF byz_ne cor_ne fin_cd]
  by (auto simp: CD_solvable_def)
```

26 Theorem 5: CD impossible under multicast

Paper, Theorem 5 (Section 4.2):

> "It is impossible to solve causality determination (Definition 5) as specified by CD(`E, F, e_i^*`) in an asynchronous multicast-based message passing system with one or more Byzantine processes.

Proof. Unicast mode of communication is a special case of multicast where each group is of size 1 (or 2 if the sender is included in the multicast group). Theorem 3 proved that causality determination in the presence of even a single Byzantine process under unicast communication is impossible to solve. As the special case of group size 1 (or 2) is not solvable, the general case of multicast is also not solvable."

```
theorem CD_impossible_multicast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Multicast correct"
  using no_produces_valid_F[OF byz_ne cor_ne fin_cd]
  by (auto simp: CD_solvable_def)
```

27 Summary corollary

One statement, all three modes.

```
theorem CD_impossible_all_modes:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Unicast   correct"
    and "¬ CD_solvable Broadcast correct"
    and "¬ CD_solvable Multicast correct"
proof -
  show "¬ CD_solvable Unicast correct"
    by (rule CD_impossible_unicast[OF byz_ne cor_ne fin_cd])
  show "¬ CD_solvable Broadcast correct"
    by (rule CD_impossible_broadcast[OF byz_ne cor_ne fin_cd])
  show "¬ CD_solvable Multicast correct"
    by (rule CD_impossible_multicast[OF byz_ne cor_ne fin_cd])
qed

end — context byzantineSystem

end


theory Foundation_Vacuity
  imports ByzantineSystem
begin
```

28 A pure-HOL "consensus solver" satisfying `solves_Consensus`

The abstract predicate `solves_Consensus` demands only Agreement and Validity on a HOL function; it places *no* constraint making the function implementable by an asynchronous distributed protocol. At this abstraction level the function below already satisfies the predicate, so any axiom of the shape "no abstract `alg` satisfies `solves_Consensus`" is unsatisfiable.

```
definition simple_alg :: "'p set ⇒ 'p consensus_alg" where
  "simple_alg C V p ≡ (∃q ∈ C. V q)"

lemma simple_alg_solves_Consensus:
  "solves_Consensus C (simple_alg C)"
proof -
  have AGR: "∀V. consensus_agreement C (simple_alg C) V"
    by (auto simp: consensus_agreement_def simple_alg_def)
  have VAL: "∀V. consensus_validity C (simple_alg C) V"
    by (auto simp: consensus_validity_def simple_alg_def)
  show ?thesis
    using AGR VAL by (simp add: solves_Consensus_def)
qed

lemma exists_consensus_alg:
  "∃alg. solves_Consensus (C::'p set) alg"
  using simple_alg_solves_Consensus by blast
```

29 The natural-looking "abstract-consensus is unsolvable" claim is false

This is what the retired axiom denied. The negation is a HOL theorem, hence the original axiom was unsatisfiable.

```
lemma exists_abstract_consensus_solver:
  shows "¬ (¬ (∃alg. solves_Consensus (C::'p set) alg))"
  using exists_consensus_alg by blast

end


theory BHB
  imports Events CD
begin
```

30 Byzantine happened-before relation (paper Definition 3)

Paper, Section 3:

> "Definition 3. The Byzantine happened before relation →_B on events at correct processes consists of the following rules: [program order at correct processes, message order between correct processes, transitive closure]."

We mechanise →_B as the transitive closure of a one-step relation that requires both endpoints of each step to be at correct processes (in the parameter C) and that the underlying step is either a program-order step or a message-order step. Restricting endpoints to C at each step suffices: by transitivity, any bhb chain has all its events at correct processes.

```
definition bhb_step ::
  "'p set ⇒ 'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "bhb_step C H e e' ⟷
     proc_of e ∈ C ∧ proc_of e' ∈ C ∧
     (program_order H e e' ∨ message_order H e e')"

definition bhb ::
  "'p set ⇒ 'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "bhb C H e e' ⟷ (bhb_step C H)^++ e e'"
```

Paper, Section 3:

> "e →_B e' |_E is defined as (e → e' |_E ∧ there is a causal path from e to e' going through correct processes in the execution)."

The paper's evaluation operator →_B|_H behaves analogously to hb_eval: an event pair satisfies it only if both endpoints are recorded in H.

```
definition bhb_eval ::
  "'p set ⇒ 'p history ⇒ 'p event ⇒ 'p event ⇒ bool" where
  "bhb_eval C H e e' ⟷
     e ∈ events_of H ∧ e' ∈ events_of H ∧ bhb C H e e'"
```

30.1 Structural lemmas: bhb is a sub-relation of hb

A single bhb_step is, by definition, also an hb_step; the converse fails because hb_step accepts steps through Byzantine processes.

```
lemma bhb_step_imp_hb_step:
  assumes "bhb_step C H e e'"
  shows "hb_step H e e'"
  using assms unfolding bhb_step_def hb_step_def by blast

lemma bhb_imp_hb:
  assumes "bhb C H e e'"
  shows "hb H e e'"
proof -
  from assms have "(bhb_step C H)^++ e e'" unfolding bhb_def .
  hence "(hb_step H)^++ e e'"
  proof (induction rule: tranclp_induct)
    case (base y)
    from bhb_step_imp_hb_step[OF base]
    show ?case by (rule tranclp.r_into_trancl)
  next
    case (step y z)
    have step_hb: "hb_step H y z"
      by (rule bhb_step_imp_hb_step[OF step.hyps(2)])
    from step.IH step_hb
```

```
    show ?case by (rule tranclp.trancl_into_trancl)
  qed
  thus "hb H e e'" by (simp add: hb_def)
qed

lemma bhb_eval_imp_hb_eval:
  assumes "bhb_eval C H e e'"
  shows "hb_eval H e e'"
  using assms bhb_imp_hb
  by (auto simp: bhb_eval_def hb_eval_def)
```

Domain restriction: a `bhb` chain begins and ends at correct processes (any intermediate node is at a correct process too, by the same step-wise restriction).

```
lemma bhb_proc_of_endpoints:
  assumes "bhb C H e e'"
  shows "proc_of e ∈ C ∧ proc_of e' ∈ C"
proof -
  from assms have "(bhb_step C H)^++ e e'" unfolding bhb_def .
  thus ?thesis
  proof (induction rule: tranclp_induct)
    case (base y)
    thus ?case unfolding bhb_step_def by blast
  next
    case (step y z)
    from step.hyps(2) have "proc_of z ∈ C" unfolding bhb_step_def by blast
    with step.IH show ?case by blast
  qed
qed
```

31 The `CD_B` problem (paper Definition 6)

Paper, Section 4.3:

> "Definition 6. The causality determination problem `CD_B(E, F, e_i^*)` for any event `e_i^* ∈ T(E)` at a correct process `p_i` is to devise an algorithm to collect the execution history `E` as `F` at `p_i` such that `valid_B(F) = 1`, where
>
> $$valid_B(F) = \begin{cases} 1 & \text{if } \forall e_h^x,\ e_h^x \rightarrow_B e_i^*|_E = e_h^x \rightarrow_B e_i^*|_F \\ 0 & \text{otherwise} \end{cases}$$
>
> "

The paper also identifies the analogues of FN and FP under `→_B`: `FN_B` is a witness of `e →_B e_i^* |_E = 1 ∧ e →_B e_i^* |_F = 0` on a causal path going through correct processes (Section 4.3, "denoting a false negative under `→_B`"), and analogously `FP_B` on the other side. Paper Section 4.3 notes that the second case "cannot occur as the first and third terms cannot both be true" – i.e., `FP_B` is vacuous in the paper's reading. We formalise both `FN_B` and `FP_B` uniformly here; the vacuity of `FP_B` is a lemma we leave to whoever proves Theorems 6/7 to bring out explicitly if needed.

```
definition valid_B ::
```

```
  "'p set ⇒ 'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "valid_B C E F e_star ⟷
     (∀e ∈ events_of E ∪ events_of F.
        bhb_eval C E e e_star = bhb_eval C F e e_star)"

definition false_negative_B ::
  "'p set ⇒ 'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "false_negative_B C E F e_star ⟷
     (∃e ∈ events_of E ∪ events_of F.
        bhb_eval C E e e_star ∧ ¬ bhb_eval C F e e_star)"

definition false_positive_B ::
  "'p set ⇒ 'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
  "false_positive_B C E F e_star ⟷
     (∃e ∈ events_of E ∪ events_of F.
        ¬ bhb_eval C E e e_star ∧ bhb_eval C F e e_star)"

lemma valid_B_iff_no_FP_FN:
  shows "valid_B C E F e_star
           ⟷ ¬ false_negative_B C E F e_star
              ∧ ¬ false_positive_B C E F e_star"
proof -
  have "valid_B C E F e_star
          ⟷ (∀e ∈ events_of E ∪ events_of F.
                bhb_eval C E e e_star = bhb_eval C F e e_star)"
    by (simp add: valid_B_def)
  also have "... ⟷ ¬ false_negative_B C E F e_star
                   ∧ ¬ false_positive_B C E F e_star"
    by (auto simp: false_negative_B_def false_positive_B_def)
  finally show ?thesis .
qed
```

`CD_B` solver signature and the `produces_valid_F_B` / `CD_B_solvable` predicates, parallel to those of `CD.thy`.

```
definition produces_valid_F_B ::
  "'p set ⇒ 'p cd_solver ⇒ bool" where
  "produces_valid_F_B C alg ⟷
     (∀adv. adversary_admissible C adv ⟶
        (let (F', _) = alg (adv_i adv) (adv_e_star adv) in
           valid_B C (adv_E adv) F' (adv_e_star adv)))"

definition CD_B_solvable :: "comm_mode ⇒ 'p set ⇒ bool" where
  "CD_B_solvable m C ⟷ (∃alg. produces_valid_F_B C alg)"
```

32 Out-of-scope: paper Section 4.3's Theorems 6, 7, 8

The three results of paper Section 4.3 – T6 (unicast, solvable), T7 (broadcast, solvable), T8 (multicast, unsolvable) – are deliberately not proved in this development. All three depend on a communication-level model the development does not provide:

- T6 and T7 are positive results. Their proofs construct explicit algorithms that use,

respectively, Byzantine Reliable Unicast and Byzantine Reliable / Causal Broadcast to collect the execution history E at p_i. Our abstract `cd_solver` signature, `'p ⇒ 'p event ⇒ 'p history × bool`, sees only the query indices $(i, e_\star)$ and has no input channel by which it could receive messages; modelling messages requires an extension along the lines of the AFP entry's asynchronousSystem locale.

- T8 is an impossibility that depends on the operational fact that Byzantine Reliable Multicast cannot be achieved without identifying Byzantine processes within the multicast group. Capturing that argument requires modelling multicast groups, the BRM primitive, and its operational requirements – again a communication-level extension.

These three theorems are in the same out-of-scope band as the "CD is solvable under crash failures" half of Theorem 16 (see `CD_vs_Consensus.thy`). The definitions above are the foundation a future development would build on; the structural lemmas (`bhb_imp_hb` et cetera) factor the obvious connections between `bhb` and `hb` so they don't have to be re-derived.

end

theory `CD_B_Algorithm`
  imports `BHB Theorems_1_2`
begin

33 Algorithm signature with a received-view input

An algorithm of this richer type takes, in addition to its local query `(i, e_star)`, a per-peer reported history `recv` – "what `p_i` has been told about each other process's execution". For correct peers, the paper assumes `recv p = E p` via the communication primitive (BRU for unicast in T6, BCB/BRB for broadcast in T7); for Byzantine peers, `recv p` can be anything the Byzantine sent.

type_synonym `'p cd_alg_with_recv =`
  `"('p ⇒ 'p history_local) ⇒ 'p ⇒ 'p event ⇒ 'p history × bool"`

context `byzantineSystem`
begin

The operational fidelity property the paper relies on: correct processes report their actual local history truthfully.

definition `correct_reporting ::`
  `"'p set ⇒ ('p ⇒ 'p history_local) ⇒ 'p history ⇒ bool"` **where**
  `"correct_reporting C recv E ⟷ (∀p ∈ C. recv p = E p)"`

The `CD_B` correctness predicate for algorithms of the richer type: the algorithm must produce a valid F (in the bhb sense) for every admissible adversary AND every received-view that respects correct reporting from correct processes.

definition `produces_valid_F_B_recv ::`
  `"'p set ⇒ 'p cd_alg_with_recv ⇒ bool"` **where**

```
  "produces_valid_F_B_recv C alg ⟷
     (∀adv recv.
        adversary_admissible C adv ⟶
        wf_history recv ⟶
        correct_reporting C recv (adv_E adv) ⟶
          (let (F', _) = alg recv (adv_i adv) (adv_e_star adv) in
             valid_B C (adv_E adv) F' (adv_e_star adv)))"
```

The naive algorithm: at `p_i`, just output the received view as `F`. Decision is always `True`.

```
definition naive_cd_B_alg :: "'p cd_alg_with_recv" where
  "naive_cd_B_alg recv _ _ = (recv, True)"
```

34 Structural lemmas: under correct_reporting, bhb is invariant

The key technical observation: bhb's definition consults events only at correct processes (`bhb_step` requires both endpoints to be in `C`), and the underlying program-order / message-order relations consult `H p` only for the process `p` that owns the event. Under `correct_reporting`, those owners are correct and `recv p = E p`, so the entire bhb relation is invariant across the two histories.

Engineering note: the `by blast` tactic does not apply equality rewrites; in proofs below, where an equality `F p = E p` needs to be substituted into a goal, we use `by (simp add: ...)` instead. An earlier version of this theory used `blast` for those substitutions and timed out (>10 min wall time).

34.1 events_of at correct processes

Helper: in a well-formed history, an event's owner is the process whose list it appears in.

```
lemma proc_of_in_wf_history:
  assumes "wf_history H" and "e ∈ set (H p)"
  shows "proc_of e = p"
proof -
  from assms(1) have "wf_history_local p (H p)"
    unfolding wf_history_def by blast
  with assms(2) show ?thesis
    unfolding wf_history_local_def by blast
qed

lemma events_at_correct_eq:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
      and pC:  "proc_of e ∈ C"
  shows "e ∈ events_of E ⟷ e ∈ events_of F"
proof
  assume "e ∈ events_of E"
  then obtain p where p_E: "e ∈ set (E p)"
    by (auto simp: events_of_def)
```

```
  from wfE p_E have "proc_of e = p"
    by (rule proc_of_in_wf_history)
  with pC have pC': "p ∈ C" by simp
  from rep pC' have "F p = E p"
    unfolding correct_reporting_def by blast
  with p_E have "e ∈ set (F p)" by simp
  thus "e ∈ events_of F" by (auto simp: events_of_def)
next
  assume "e ∈ events_of F"
  then obtain p where p_F: "e ∈ set (F p)"
    by (auto simp: events_of_def)
  from wfF p_F have "proc_of e = p"
    by (rule proc_of_in_wf_history)
  with pC have pC': "p ∈ C" by simp
  from rep pC' have "F p = E p"
    unfolding correct_reporting_def by blast
  with p_F have "e ∈ set (E p)" by simp
  thus "e ∈ events_of E" by (auto simp: events_of_def)
qed

lemma program_order_at_correct:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
      and pC:  "proc_of e ∈ C"
  shows "program_order E e e' = program_order F e e'"
proof
  assume "program_order E e e'"
  then obtain p i j where
    ij: "i < j" "j < length (E p)"
    and ei: "(E p) ! i = e"
    and ej: "(E p) ! j = e'"
    unfolding program_order_def by blast
  have proc_e: "proc_of e = p"
  proof -
    from ij have "i < length (E p)" by simp
    hence "(E p) ! i ∈ set (E p)" by (rule nth_mem)
    with ei have "e ∈ set (E p)" by simp
    with wfE show ?thesis by (rule proc_of_in_wf_history)
  qed
  with pC have pC': "p ∈ C" by simp
  from rep pC' have eq: "F p = E p"
    unfolding correct_reporting_def by blast
  have "i < j ∧ j < length (F p) ∧ F p ! i = e ∧ F p ! j = e'"
    using ij ei ej by (simp add: eq)
  thus "program_order F e e'"
    unfolding program_order_def by blast
next
  assume "program_order F e e'"
  then obtain p i j where
    ij: "i < j" "j < length (F p)"
    and ei: "(F p) ! i = e"
```

```
    and ej: "(F p) ! j = e'"
    unfolding program_order_def by blast
  have proc_e: "proc_of e = p"
  proof -
    from ij have "i < length (F p)" by simp
    hence "(F p) ! i ∈ set (F p)" by (rule nth_mem)
    with ei have "e ∈ set (F p)" by simp
    with wfF show ?thesis by (rule proc_of_in_wf_history)
  qed
  with pC have pC': "p ∈ C" by simp
  from rep pC' have eq: "F p = E p"
    unfolding correct_reporting_def by blast
  have "i < j ∧ j < length (E p) ∧ E p ! i = e ∧ E p ! j = e'"
    using ij ei ej by (simp add: eq[symmetric])
  thus "program_order E e e'"
    unfolding program_order_def by blast
qed

lemma message_order_at_correct:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
      and pC:  "proc_of e ∈ C"
      and pC': "proc_of e' ∈ C"
  shows "message_order E e e' = message_order F e e'"
proof -
  have eqE: "e ∈ events_of E ⟷ e ∈ events_of F"
    by (rule events_at_correct_eq[OF wfE wfF rep pC])
  have eqE': "e' ∈ events_of E ⟷ e' ∈ events_of F"
    by (rule events_at_correct_eq[OF wfE wfF rep pC'])
  show ?thesis
    unfolding message_order_def by (simp add: eqE eqE')
qed
```

34.2 One bhb step is invariant under correct_reporting

```
lemma bhb_step_eq:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
  shows "bhb_step C E e e' = bhb_step C F e e'"
proof
  assume H: "bhb_step C E e e'"
  hence pC: "proc_of e ∈ C" and pC': "proc_of e' ∈ C"
    unfolding bhb_step_def by blast+
  from H have step_in_E:
    "program_order E e e' ∨ message_order E e e'"
    unfolding bhb_step_def by blast
  have po_eq: "program_order E e e' = program_order F e e'"
    by (rule program_order_at_correct[OF wfE wfF rep pC])
  have mo_eq: "message_order E e e' = message_order F e e'"
    by (rule message_order_at_correct[OF wfE wfF rep pC pC'])
```

```
  from step_in_E po_eq mo_eq
  have step_in_F: "program_order F e e' ∨ message_order F e e'" by simp
  show "bhb_step C F e e'"
    unfolding bhb_step_def using pC pC' step_in_F by blast
next
  assume H: "bhb_step C F e e'"
  hence pC: "proc_of e ∈ C" and pC': "proc_of e' ∈ C"
    unfolding bhb_step_def by blast+
  from H have step_in_F:
    "program_order F e e' ∨ message_order F e e'"
    unfolding bhb_step_def by blast
  have po_eq: "program_order E e e' = program_order F e e'"
    by (rule program_order_at_correct[OF wfE wfF rep pC])
  have mo_eq: "message_order E e e' = message_order F e e'"
    by (rule message_order_at_correct[OF wfE wfF rep pC pC'])
  from step_in_F po_eq mo_eq
  have step_in_E: "program_order E e e' ∨ message_order E e e'" by simp
  show "bhb_step C E e e'"
    unfolding bhb_step_def using pC pC' step_in_E by blast
qed
```

34.3 bhb itself is invariant under correct_reporting

```
lemma bhb_eq_under_correct_reporting:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
  shows "bhb C E e e' = bhb C F e e'"
proof
  assume "bhb C E e e'"
  hence "(bhb_step C E)^++ e e'" unfolding bhb_def .
  hence "(bhb_step C F)^++ e e'"
  proof (induction rule: tranclp_induct)
    case (base y)
    have "bhb_step C F e y"
      using base bhb_step_eq[OF wfE wfF rep] by simp
    thus ?case by (rule tranclp.r_into_trancl)
  next
    case (step y z)
    have step_F: "bhb_step C F y z"
      using step.hyps(2) bhb_step_eq[OF wfE wfF rep] by simp
    from step.IH step_F show ?case by (rule tranclp.trancl_into_trancl)
  qed
  thus "bhb C F e e'" unfolding bhb_def .
next
  assume "bhb C F e e'"
  hence "(bhb_step C F)^++ e e'" unfolding bhb_def .
  hence "(bhb_step C E)^++ e e'"
  proof (induction rule: tranclp_induct)
    case (base y)
    have "bhb_step C E e y"
      using base bhb_step_eq[OF wfE wfF rep] by simp
```

```
    thus ?case by (rule tranclp.r_into_trancl)
  next
    case (step y z)
    have step_E: "bhb_step C E y z"
      using step.hyps(2) bhb_step_eq[OF wfE wfF rep] by simp
    from step.IH step_E show ?case by (rule tranclp.trancl_into_trancl)
  qed
  thus "bhb C E e e'" unfolding bhb_def .
qed
```

34.4 bhb$_{eval}$ agrees on relevant events

Both bhb endpoints must be at correct processes for the relation to hold (by `bhb ?C ?H ?e ?e' ⟹ proc_of ?e ∈ ?C ∧ proc_of ?e' ∈ ?C`), so it suffices to check `events_of` at correct processes; under correct reporting those agree.

```
lemma bhb_eval_eq_under_correct_reporting:
  assumes wfE: "wf_history E"
      and wfF: "wf_history F"
      and rep: "correct_reporting C F E"
      and pC_star: "proc_of e_star ∈ C"
  shows "bhb_eval C E e e_star = bhb_eval C F e e_star"
proof (cases "proc_of e ∈ C")
  case True
  have e_eq: "e ∈ events_of E ⟷ e ∈ events_of F"
    by (rule events_at_correct_eq[OF wfE wfF rep True])
  have es_eq: "e_star ∈ events_of E ⟷ e_star ∈ events_of F"
    by (rule events_at_correct_eq[OF wfE wfF rep pC_star])
  have bhb_eq: "bhb C E e e_star = bhb C F e e_star"
    by (rule bhb_eq_under_correct_reporting[OF wfE wfF rep])
  show ?thesis
    unfolding bhb_eval_def by (simp add: e_eq es_eq bhb_eq)
next
  case False
  hence noE: "¬ bhb C E e e_star"
    using bhb_proc_of_endpoints by blast
  from False have noF: "¬ bhb C F e e_star"
    using bhb_proc_of_endpoints by blast
  show ?thesis
    unfolding bhb_eval_def using noE noF by blast
qed
```

35 The naive CD_B algorithm and its correctness

The abstract correctness theorem: under correct reporting from correct processes, the naive algorithm `F = recv` satisfies `CD_B` (in the bhb sense). This is the algorithmic core of paper Theorems 6 and 7.

```
theorem naive_cd_B_alg_correct:
  shows "produces_valid_F_B_recv correct naive_cd_B_alg"
proof (unfold produces_valid_F_B_recv_def, intro allI impI)
  fix adv recv
```

```
  assume adm: "adversary_admissible correct adv"
     and wfR: "wf_history recv"
     and rep: "correct_reporting correct recv (adv_E adv)"

  have wfE: "wf_history (adv_E adv)"
    using adm by (auto simp: adversary_admissible_def)

  have pC_star: "proc_of (adv_e_star adv) ∈ correct"
  proof -
    have "proc_of (adv_e_star adv) = adv_i adv"
      using adm by (auto simp: adversary_admissible_def)
    moreover have "adv_i adv ∈ correct"
      using adm by (auto simp: adversary_admissible_def)
    ultimately show ?thesis by simp
  qed

  have "valid_B correct (adv_E adv) recv (adv_e_star adv)"
  proof (unfold valid_B_def, intro ballI)
    fix e
    assume e_in: "e ∈ events_of (adv_E adv) ∪ events_of recv"
    show "bhb_eval correct (adv_E adv) e (adv_e_star adv)
            = bhb_eval correct recv e (adv_e_star adv)"
      by (rule bhb_eval_eq_under_correct_reporting[OF wfE wfR rep pC_star])
  qed
  thus "let (F', _) = naive_cd_B_alg recv (adv_i adv) (adv_e_star adv) in
          valid_B correct (adv_E adv) F' (adv_e_star adv)"
    by (simp add: naive_cd_B_alg_def Let_def)
qed

corollary CD_B_solvable_under_correct_reporting:
  shows "∃alg. produces_valid_F_B_recv correct alg"
  using naive_cd_B_alg_correct by blast
```

36 Theorems 6 and 7: CD_B solvable under unicast and broadcast

Paper, Theorem 6 (Section 4.3.2, "possible"):

> "It is possible to solve causality determination (Definition 6) as specified by CD_B(E, F, e_i^*), now defined in terms of the →_B relation, in an asynchronous unicast-based message passing system with one or more Byzantine processes."

Paper, Theorem 7 (Section 4.3.1, "possible"):

> "It is possible to solve causality determination (Definition 6) as specified by CD_B(E, F, e_i^*), now defined in terms of the →_B relation, in an asynchronous broadcast-based message passing system with one or more Byzantine processes."

At the abstraction level of this theory, Theorems 6 and 7 are the same statement: "in mode m, some algorithm of type cd_alg_with_recv satisfies produces_valid_F_B_recv".

Both are direct corollaries of `produces_valid_F_B_recv correct naive_cd_B_alg` since the abstract correctness of the naive algorithm does not depend on the mode.

Where the paper's two theorems differ. The paper's distinction between T6 and T7 lies in which communication primitive discharges `correct_reporting`:

- T6 (unicast): correct processes report their local histories via point-to-point messages (Byzantine Reliable Unicast, trivially achievable since point-to-point is already reliable in the absence of Byzantine senders along the path), supplemented by simulated broadcasts of control information after application unicast send events.
- T7 (broadcast): correct processes report via Byzantine Causal Broadcast over Byzantine Reliable Broadcast (BCB-over-BRB), which guarantees that any message a correct process delivers was previously delivered by all other correct processes in the causal order required to reconstruct the history.

Both primitives are operational and require a communication-level model – explicit messages, channels, fairness – not present in this development. We therefore state Theorems 6 and 7 as mode-tagged existential statements about `produces_valid_F_B_recv`, leaving the operational discharge of `correct_reporting` to whoever extends the development with a communication model.

```
definition CD_B_solvable_with_recv ::
  "comm_mode ⇒ 'p set ⇒ bool" where
  "CD_B_solvable_with_recv m C ⟷
     (∃alg. produces_valid_F_B_recv C alg)"

theorem CD_B_solvable_unicast:
  shows "CD_B_solvable_with_recv Unicast correct"
  unfolding CD_B_solvable_with_recv_def
  by (rule CD_B_solvable_under_correct_reporting)

theorem CD_B_solvable_broadcast:
  shows "CD_B_solvable_with_recv Broadcast correct"
  unfolding CD_B_solvable_with_recv_def
  by (rule CD_B_solvable_under_correct_reporting)
```

37 Theorem 8: `CD_B` impossible under multicast

Paper, Theorem 8 (Section 4.3, "impossible"):

> "It is impossible to solve causality determination (Definition 6) as specified by `CD_B(E, F, e_i^*)`, now defined in terms of the `→_B` relation, in an asynchronous multicast-based message passing system with one or more Byzantine processes."

The paper's proof is operational: BRM (Byzantine Reliable Multicast) is the primitive that would discharge `correct_reporting` under multicast, but BRM is unachievable without first identifying the Byzantine processes within each multicast group. Hence under multicast, `correct_reporting` cannot be assumed, and an algorithm must be correct without it – which we capture as the "strong" predicate below.

Our formalisation. We give T8 a concrete formal content by showing that no algorithm satisfies `produces_valid_F_B_recv_strong` – the strengthened correctness predicate that drops the `correct_reporting` hypothesis. The proof reuses the fresh-id adversary construction of `Theorems_1_2.thy`, specialised so the witness chain runs through two distinct *correct* processes `p_c` and `p_i` (rather than a Byzantine sender). The chain becomes a genuine $\rightarrow$`_B` chain in `E`; if the algorithm's output `F` has finite `events_of`, a fresh message id exists outside `F`, and the bhb chain in `E` is missing in `F` – a bhb-false-negative.

The operational "BRM is unachievable" argument is not formalised (same out-of-scope band as T6/T7's BRU and BCB/BRB primitives); we state it as the meta-level fact that translates "under multicast" into "without `correct_reporting`" into the strong predicate.

```
definition produces_valid_F_B_recv_strong ::
  "'p set ⇒ 'p cd_alg_with_recv ⇒ bool" where
  "produces_valid_F_B_recv_strong C alg ⟷
     (∀adv recv.
        adversary_admissible C adv ⟶
        wf_history recv ⟶
          (let (F', _) = alg recv (adv_i adv) (adv_e_star adv) in
             valid_B C (adv_E adv) F' (adv_e_star adv)))"
```

37.1 The bhb chain in `fn_E` when both endpoints are correct

The fresh-id history `fn_E p_i p_c m` from `Theorems_1_2.thy`, when `p_c` and `p_i` are both correct and distinct, exhibits a genuine $\rightarrow$`_B` chain from the `Send p_c 1 p_i m` event to the `Internal p_i 2` event. Both endpoints and the intermediate `Receive p_i 1 p_c m` are at correct processes, so every step is a `bhb_step`.

```
lemma bhb_send_to_estar_in_fn_E:
  assumes pi_cor: "p_i ∈ correct"
      and pc_cor: "p_c ∈ correct"
      and dist:   "p_c ≠ p_i"
  shows "bhb correct (fn_E p_i p_c m) (Send p_c 1 p_i m) (Internal p_i 2)"
proof -
  let ?H = "fn_E p_i p_c m"
  let ?s = "Send p_c 1 p_i m"
  let ?r = "Receive p_i 1 p_c m"
  let ?es = "Internal p_i 2"

  have step_sr_under: "program_order ?H ?s ?r ∨ message_order ?H ?s ?r"
    using message_order_fn_E_send_receive[OF dist] by blast
  have proc_s: "proc_of ?s = p_c" by simp
  have proc_r: "proc_of ?r = p_i" by simp
  have proc_es: "proc_of ?es = p_i" by simp

  have step_sr: "bhb_step correct ?H ?s ?r"
    using step_sr_under pc_cor pi_cor proc_s proc_r
    unfolding bhb_step_def by simp

  have step_re_under: "program_order ?H ?r ?es ∨ message_order ?H ?r ?es"
    using program_order_fn_E_at_pi[OF dist] by blast
  have step_re: "bhb_step correct ?H ?r ?es"
```

```
    using step_re_under pi_cor proc_r proc_es
    unfolding bhb_step_def by simp

  have base: "(bhb_step correct ?H)^++ ?s ?r"
    using step_sr by blast
  have extend: "(bhb_step correct ?H)^++ ?r ?es"
    using step_re by blast
  have "(bhb_step correct ?H)^++ ?s ?es"
    using base extend by (rule tranclp_trans)
  thus ?thesis unfolding bhb_def .
qed

lemma bhb_eval_send_to_estar_in_fn_E:
  assumes pi_cor: "p_i ∈ correct"
      and pc_cor: "p_c ∈ correct"
      and dist:   "p_c ≠ p_i"
  shows "bhb_eval correct (fn_E p_i p_c m)
           (Send p_c 1 p_i m) (Internal p_i 2)"
proof -
  have e1: "Send p_c 1 p_i m ∈ events_of (fn_E p_i p_c m)"
    using dist by (simp add: fn_E_events)
  have e2: "Internal p_i 2 ∈ events_of (fn_E p_i p_c m)"
    using dist by (simp add: fn_E_events)
  have h: "bhb correct (fn_E p_i p_c m)
             (Send p_c 1 p_i m) (Internal p_i 2)"
    by (rule bhb_send_to_estar_in_fn_E[OF pi_cor pc_cor dist])
  show ?thesis using e1 e2 h by (simp add: bhb_eval_def)
qed
```

37.2 The impossibility

Following the same shape as ⟦∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i; ∀p_i_in. finite (events_of (fst (?alg p_i_in (Internal p_i_in 2))))⟧ ⟹ ∃adv. adversary_admissible correct adv ∧ false_negative (adv_E adv) (fst (?alg (adv_i adv) (adv_e_star adv))) (adv_e_star adv): any purported strong-correct algorithm has finite output, fresh ids exist, the adversary places a bhb chain through two correct processes using a fresh id, the algorithm's F misses the chain start, and a bhb-false-negative results.

```
theorem produces_valid_F_B_recv_strong_unsolvable:
  assumes cor_two:
        "∃p_i p_c. p_i ∈ correct ∧ p_c ∈ correct ∧ p_c ≠ p_i"
      and fin_F:
        "∀alg. produces_valid_F_B_recv_strong correct alg ⟶
           (∀recv p_i_in.
              finite (events_of
                        (fst (alg recv p_i_in (Internal p_i_in 2)))))"
  shows "¬ (∃alg. produces_valid_F_B_recv_strong correct alg)"
proof
  assume "∃alg. produces_valid_F_B_recv_strong correct alg"
  then obtain alg where solver:
    "produces_valid_F_B_recv_strong correct alg" by blast
```

```
obtain p_i p_c where
    pi_cor: "p_i ∈ correct"
    and pc_cor: "p_c ∈ correct"
    and dist:   "p_c ≠ p_i"
  using cor_two by blast

define recv :: "'p ⇒ 'p history_local" where
  "recv ≡ (λ_. [])"

have wfR: "wf_history recv"
  unfolding recv_def wf_history_def wf_history_local_def by simp

let ?e_star = "Internal p_i 2"
let ?F = "fst (alg recv p_i ?e_star)"
let ?m = "fresh_nat ?F"
let ?adv = "fn_adv p_i p_c ?m"
let ?s = "Send p_c 1 p_i ?m"

have finF: "finite (events_of ?F)"
  using fin_F solver by blast

have adm: "adversary_admissible correct ?adv"
  using pi_cor dist by (rule adv_admissible_fn_adv)

have adv_eq:
  "adv_E ?adv = fn_E p_i p_c ?m ∧
   adv_i ?adv = p_i ∧
   adv_e_star ?adv = ?e_star"
  by (simp add: fn_adv_def)

have witness_in_E: "?s ∈ events_of (adv_E ?adv)"
  using dist adv_eq by (simp add: fn_E_events)
have bhb_E_chain:
  "bhb_eval correct (adv_E ?adv) ?s (adv_e_star ?adv)"
  using bhb_eval_send_to_estar_in_fn_E[OF pi_cor pc_cor dist]
        adv_eq by simp
have witness_not_in_F: "?s ∉ events_of ?F"
  using finF Send_at_fresh_nat_not_in_F[of ?F] by blast
have not_bhb_F:
  "¬ bhb_eval correct ?F ?s (adv_e_star ?adv)"
  using witness_not_in_F by (simp add: bhb_eval_def)

— The adversary establishes a bhb-FN: chain in E, not in F.
have FN_witness:
  "?s ∈ events_of (adv_E ?adv) ∪ events_of ?F ∧
   bhb_eval correct (adv_E ?adv) ?s (adv_e_star ?adv) ∧
   ¬ bhb_eval correct ?F ?s (adv_e_star ?adv)"
  using witness_in_E bhb_E_chain not_bhb_F by blast

hence FN: "false_negative_B correct (adv_E ?adv) ?F (adv_e_star ?adv)"
  unfolding false_negative_B_def by blast
```

```
  — But the alleged solver claims valid_B on this very adversary.
  have F_eq:
    "?F = fst (alg recv (adv_i ?adv) (adv_e_star ?adv))"
    using adv_eq by simp
  have "valid_B correct (adv_E ?adv) ?F (adv_e_star ?adv)"
  proof -
    from solver adm wfR have
      "let (F', _) = alg recv (adv_i ?adv) (adv_e_star ?adv) in
         valid_B correct (adv_E ?adv) F' (adv_e_star ?adv)"
      by (simp add: produces_valid_F_B_recv_strong_def)
    thus ?thesis using F_eq by (simp add: Let_def split: prod.split_asm)
  qed
  hence "¬ false_negative_B correct (adv_E ?adv) ?F (adv_e_star ?adv)"
    using valid_B_iff_no_FP_FN by blast
  with FN show False by contradiction
qed
```

Mapping to paper Theorem 8. The paper says: under multicast, `correct_reporting` cannot be guaranteed (BRM is unachievable), so an algorithm would have to work without it – but the previous theorem says no algorithm does. Hence `CD_B` is unsolvable under multicast.

The operational "BRM is unachievable" claim itself is out of scope (it requires modelling multicast groups, the BRM primitive, and the unachievability of BRM without Byzantine identification). We state the impossibility conditionally on the operational claim: "if BRM-style `correct_reporting` is not guaranteed under multicast, then `CD_B` is unsolvable under multicast in the `recv`-augmented signature".

In our predicate vocabulary:

- "`correct_reporting` guaranteed" corresponds to using the *conditional* predicate `produces_valid_F_B_recv` – which is satisfiable (T6/T7).
- "`correct_reporting` not guaranteed" corresponds to using the *strong* predicate `produces_valid_F_B_recv_strong` – which is *not* satisfiable (⟦∃p_i p_c. p_i ∈ correct ∧ p_c ∈ correct ∧ p_c ≠ p_i; ∀alg. produces_valid_F_B_recv_strong correct alg ⟶ (∀recv p_i_in. finite (events_of (fst (alg recv p_i_in (Internal p_i_in 2)))))⟧ ⟹ ∄alg. produces_valid_F_B_recv_strong correct alg).

```
theorem CD_B_unsolvable_multicast_abstract:
  assumes cor_two:
        "∃p_i p_c. p_i ∈ correct ∧ p_c ∈ correct ∧ p_c ≠ p_i"
      and fin_F:
        "∀alg. produces_valid_F_B_recv_strong correct alg ⟶
           (∀recv p_i_in.
              finite (events_of
                        (fst (alg recv p_i_in (Internal p_i_in 2)))))"
  shows "¬ (∃alg. produces_valid_F_B_recv_strong correct alg)"
  by (rule produces_valid_F_B_recv_strong_unsolvable[OF cor_two fin_F])

end — context byzantineSystem
```

end

theory Delivery
  imports CD_B_Algorithm
begin

context byzantineSystem
begin

38 The delivery property

The abstract operational property: a global history H in which every send by a correct process to a correct process has a matching receive at the peer. We do not constrain delivery between Byzantine senders/receivers – their events are irrelevant to bhb because bhb chains run through correct processes only.

definition messages_delivered_among ::
  "'p set ⇒ 'p history ⇒ bool" **where**
  "messages_delivered_among C H ⟷
     (∀p n q m. Send p n q m ∈ events_of H
                 ⟶ p ∈ C ⟶ q ∈ C
                 ⟶ (∃n'. Receive q n' p m ∈ events_of H))"

39 From a delivering history to a faithful received view

The trivial received view: at process p, report what the global history records at each peer q.

This is the abstract content of "at p_i, the algorithm reads each F_q from what it has been told". Operationally, p_i would derive this view by collecting messages it has received from or about q; at this abstraction level we identify the view with the history's q component directly.

definition recv_from_history ::
  "'p ⇒ 'p history ⇒ ('p ⇒ 'p history_local)" **where**
  "recv_from_history p H q = H q"

lemma recv_from_history_simp [simp]:
  "recv_from_history p H q = H q"
  by (simp add: recv_from_history_def)

lemma recv_from_history_eq:
  "recv_from_history p H = H"
  by (rule ext) (simp add: recv_from_history_def)

Sanity: a well-formed history yields a well-formed received view. This is needed so that the produces_valid_F_B_recv correct naive_cd_B_alg hypothesis wf_history recv is satisfied at every correct receiver.

lemma wf_recv_from_history:
  assumes "wf_history H"
  shows   "wf_history (recv_from_history p H)"

```
  using assms by (simp add: recv_from_history_eq)
```

The headline reduction: under operational delivery, the trivial received view satisfies `correct_reporting`.

At this abstraction level the proof is by definition (both sides collapse to `H q` at correct `q`); the `messages_delivered_among` hypothesis is not directly used in the proof. We retain it as the named hypothesis because it is the property that downstream operational developments will need to discharge: an algorithm that derives `recv` from received messages will satisfy this lemma only when correct-to-correct delivery is operationally enforced, otherwise the algorithm will see something other than `H q`.

```
lemma correct_reporting_of_recv_from_history:
  assumes "messages_delivered_among correct H"
  shows "correct_reporting correct (recv_from_history p H) H"
  unfolding correct_reporting_def
  by (simp add: recv_from_history_eq)
```

40 The naive algorithm as a CD_B-solver under operational delivery

Composing `messages_delivered_among correct ?H ⟹ correct_reporting correct (recv_from_history ?p ?H) ?H` with `produces_valid_F_B_recv correct naive_cd_B_alg` gives: under operational `messages_delivered_among`, the naive algorithm's output is valid in the bhb sense at any admissible adversary's view.

```
theorem naive_cd_B_alg_correct_under_delivery:
  assumes adm: "adversary_admissible correct adv"
      and del: "messages_delivered_among correct (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                 (fst (naive_cd_B_alg
                        (recv_from_history (adv_i adv) (adv_E adv))
                        (adv_i adv) (adv_e_star adv)))
                 (adv_e_star adv)"
proof -
  let ?recv = "recv_from_history (adv_i adv) (adv_E adv)"
  have wfE: "wf_history (adv_E adv)"
    using adm by (auto simp: adversary_admissible_def)
  have wfR: "wf_history ?recv"
    using wfE by (rule wf_recv_from_history)
  have rep: "correct_reporting correct ?recv (adv_E adv)"
    using del by (rule correct_reporting_of_recv_from_history)
  have body:
    "let (F', _) = naive_cd_B_alg ?recv (adv_i adv) (adv_e_star adv) in
       valid_B correct (adv_E adv) F' (adv_e_star adv)"
    using naive_cd_B_alg_correct[unfolded produces_valid_F_B_recv_def]
          adm wfR rep
    by blast
  thus ?thesis by (simp add: Let_def split: prod.split_asm)
qed
```

41 Operational versions of Theorems 6 and 7

Paper Theorem 6 (unicast) and Theorem 7 (broadcast) operationally: under the corresponding mode, the communication primitive achieves `messages_delivered_among`, so the naive algorithm solves `CD_B`.

What is mechanised here. Given the operational hypothesis ∀`H. mode_admissible m H` ⟶ `messages_delivered_among correct H` – "every history reachable under mode `m` delivers correct-to- correct messages" – the naive algorithm solves `CD_B` in mode `m`.

What remains operational. The hypothesis itself is the operational fact about the communication primitive. For unicast it follows from BRU (correct-to-correct point-to-point is direct, no Byzantine sender in the path can corrupt); for broadcast it follows from BCB-over-BRB (the BRB layer ensures any correct sender's broadcast is delivered to every correct receiver). Mechanising either operational fact requires modelling the communication primitive itself – a separate development.

Mode-specific admissibility on global histories. For unicast and broadcast the admissibility predicate *includes* the correct-to-correct delivery property – that is the structural content of "the unicast / broadcast communication primitive is in use". For multicast, no delivery guarantee is built in – matching the paper's T8 "BRM unachievable" claim that correct-to-correct delivery cannot be enforced.

At our event-based abstraction level the structural content of T6's unicast property and T7's broadcast property collapse to the same condition: every correct-to-correct `Send` event has a matching `Receive` event at the peer. (The paper's distinction lies in the operational primitives – BRU for T6, BCB-over-BRB for T7 – not in what the histories look like at the level of `event`.)

```
definition mode_admissible :: "comm_mode ⇒ 'p history ⇒ bool" where
  "mode_admissible m H ⟷
     wf_history H ∧
     (case m of
        Unicast ⇒ messages_delivered_among correct H
      | Broadcast ⇒ messages_delivered_among correct H
      | Multicast ⇒ True)"
```

Sanity: under unicast / broadcast, delivery is automatic; under multicast it is not.

```
lemma mode_admissible_unicast_delivers:
  assumes "mode_admissible Unicast H"
  shows "messages_delivered_among correct H"
  using assms by (simp add: mode_admissible_def)

lemma mode_admissible_broadcast_delivers:
  assumes "mode_admissible Broadcast H"
  shows "messages_delivered_among correct H"
  using assms by (simp add: mode_admissible_def)

lemma mode_admissible_wf:
  assumes "mode_admissible m H"
  shows "wf_history H"
  using assms by (simp add: mode_admissible_def)

theorem CD_B_solvable_under_unicast_delivery:
```

```
  assumes mode_delivers:
        "∀H. mode_admissible Unicast H
              ⟶ messages_delivered_among correct H"
  shows "∀adv. adversary_admissible correct adv
              ⟶ mode_admissible Unicast (adv_E adv)
              ⟶ valid_B correct (adv_E adv)
                          (fst (naive_cd_B_alg
                                 (recv_from_history (adv_i adv) (adv_E adv))
                                 (adv_i adv) (adv_e_star adv)))
                          (adv_e_star adv)"
proof (intro allI impI)
  fix adv
  assume adm: "adversary_admissible correct adv"
     and mode_ok: "mode_admissible Unicast (adv_E adv)"
  from mode_delivers mode_ok have
    "messages_delivered_among correct (adv_E adv)" by blast
  with adm show
    "valid_B correct (adv_E adv)
              (fst (naive_cd_B_alg
                     (recv_from_history (adv_i adv) (adv_E adv))
                     (adv_i adv) (adv_e_star adv)))
              (adv_e_star adv)"
    by (rule naive_cd_B_alg_correct_under_delivery)
qed

theorem CD_B_solvable_under_broadcast_delivery:
  assumes mode_delivers:
        "∀H. mode_admissible Broadcast H
              ⟶ messages_delivered_among correct H"
  shows "∀adv. adversary_admissible correct adv
              ⟶ mode_admissible Broadcast (adv_E adv)
              ⟶ valid_B correct (adv_E adv)
                          (fst (naive_cd_B_alg
                                 (recv_from_history (adv_i adv) (adv_E adv))
                                 (adv_i adv) (adv_e_star adv)))
                          (adv_e_star adv)"
proof (intro allI impI)
  fix adv
  assume adm: "adversary_admissible correct adv"
     and mode_ok: "mode_admissible Broadcast (adv_E adv)"
  from mode_delivers mode_ok have
    "messages_delivered_among correct (adv_E adv)" by blast
  with adm show
    "valid_B correct (adv_E adv)
              (fst (naive_cd_B_alg
                     (recv_from_history (adv_i adv) (adv_E adv))
                     (adv_i adv) (adv_e_star adv)))
              (adv_e_star adv)"
    by (rule naive_cd_B_alg_correct_under_delivery)
qed
```

42 Unconditional operational T6 and T7 (Phase 5)

Under the refined `mode_admissible` – where the unicast and broadcast cases structurally include `messages_delivered_among` – the operational hypothesis of the Phase 4 theorems is automatically discharged. The Phase 5 theorems below are therefore unconditional in mode-admissibility, with no separate operational hypothesis required.

What this means precisely. The Phase 4 statement was parameterised by the meta-claim "every history reachable under mode `m` delivers correct-to-correct messages". In Phase 5 that meta-claim is internalised: a history is *mode-admissible* under unicast / broadcast *iff* it satisfies that delivery property. The conditional Phase 4 theorems collapse to the unconditional Phase 5 theorems below.

What still needs a real model. The remaining gap moves one level down: we have not constructed an actual communication primitive whose reachable histories are unicast/broadcast- admissible in the new sense. Doing so requires modelling sends-in-flight, schedulers, fairness, and (for T7) BRB internals – a separate development. When that development arrives, the mode-admissibility predicate above is exactly the interface it has to validate: "every history reachable under your operational model is mode-admissible".

```
theorem CD_B_solvable_unicast_operational:
  assumes adm: "adversary_admissible correct adv"
      and mode_ok: "mode_admissible Unicast (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i adv) (adv_E adv))
                       (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
proof -
  have "messages_delivered_among correct (adv_E adv)"
    by (rule mode_admissible_unicast_delivers[OF mode_ok])
  with adm show ?thesis
    by (rule naive_cd_B_alg_correct_under_delivery)
qed

theorem CD_B_solvable_broadcast_operational:
  assumes adm: "adversary_admissible correct adv"
      and mode_ok: "mode_admissible Broadcast (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i adv) (adv_E adv))
                       (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
proof -
  have "messages_delivered_among correct (adv_E adv)"
    by (rule mode_admissible_broadcast_delivers[OF mode_ok])
  with adm show ?thesis
    by (rule naive_cd_B_alg_correct_under_delivery)
qed
```

Discharge of the Phase 4 operational hypothesis: under the refined `mode_admissible`, the meta-claim $\forall$`H. mode_admissible Unicast H` $\longrightarrow$ `messages_delivered_among correct H` becomes a theorem. Likewise for broadcast.

```
theorem unicast_delivers_unconditionally:
  shows "∀H. mode_admissible Unicast H
             ⟶ messages_delivered_among correct H"
  by (auto simp: mode_admissible_def)

theorem broadcast_delivers_unconditionally:
  shows "∀H. mode_admissible Broadcast H
             ⟶ messages_delivered_among correct H"
  by (auto simp: mode_admissible_def)

end — context byzantineSystem

end
```

```
theory Execution_Model
  imports Delivery
begin
```

43 Configurations: history + in-flight buffer

An in-flight message is a triple `(sender, receiver, msg_id)`; we track only correct-to-correct sends in the buffer because only those are constrained by `messages_delivered_among` restricted to `correct`.

These declarations live at the theory level (outside the `byzantineSystem` locale context) because Isabelle does not allow `record` or `type_synonym` in a context that locally fixes type variables.

```
type_synonym 'q in_flight = "('q × 'q × nat) multiset"

record 'q config =
  cfg_hist     :: "'q history"
  cfg_inflight :: "'q in_flight"

context byzantineSystem
begin
```

Naming the empty buffer separately avoids a parser interaction between record syntax (`(| ... |)`) and the empty multiset literal (`{#}`).

```
definition empty_inflight :: "'p in_flight" where
  "empty_inflight = (λ_. 0)"

definition init_config :: "'p config" where
  "init_config = (| cfg_hist = (λ_. []), cfg_inflight = empty_inflight |)"
```

44 One operational step

`run_step` covers four kinds of step.

- `step_internal`: a correct `p` appends an `Internal` event with the next sequence number.

- `step_send`: a correct `p` sends a fresh-id message to a correct `q`, appending a `Send` event and adding the triple to the buffer.
- `step_recv`: a correct `q` consumes an in-flight message from a correct `p`, appending a matching `Receive` and removing the triple.
- `step_byzantine`: a Byzantine `p` appends one event whose `proc_of` is `p`; the buffer is unchanged. Byzantine sends to correct receivers are not added to the buffer – conservatively, this models a network in which Byzantine senders cannot enforce delivery at correct receivers, which matches the paper's unicast / broadcast setting where correct receivers only believe what they actually receive.

```
inductive run_step :: "'p config ⇒ 'p config ⇒ bool" where
  step_internal:
    "p ∈ correct
       ⟹ n = Suc (length (cfg_hist cfg p))
       ⟹ cfg' = cfg
             (| cfg_hist := (cfg_hist cfg)
                              (p := cfg_hist cfg p @ [Internal p n]) |)
       ⟹ run_step cfg cfg'"
| step_send:
    "p ∈ correct
       ⟹ q ∈ correct
       ⟹ n = Suc (length (cfg_hist cfg p))
       ⟹ cfg' = cfg
             (| cfg_hist := (cfg_hist cfg)
                              (p := cfg_hist cfg p @ [Send p n q m]),
                cfg_inflight := cfg_inflight cfg ∪# {# (p, q, m) } |)
       ⟹ run_step cfg cfg'"
| step_recv:
    "q ∈ correct
       ⟹ p ∈ correct
       ⟹ (p, q, m) ∈# cfg_inflight cfg
       ⟹ n = Suc (length (cfg_hist cfg q))
       ⟹ cfg' = cfg
             (| cfg_hist := (cfg_hist cfg)
                              (q := cfg_hist cfg q @ [Receive q n p m]),
                cfg_inflight := cfg_inflight cfg -# (p, q, m) |)
       ⟹ run_step cfg cfg'"
| step_byzantine:
    "p ∈ byzantine
       ⟹ proc_of new_event = p
       ⟹ seq_of new_event = Suc (length (cfg_hist cfg p))
       ⟹ cfg' = cfg
             (| cfg_hist := (cfg_hist cfg)
                              (p := cfg_hist cfg p @ [new_event]) |)
       ⟹ run_step cfg cfg'"
```

45 Runs: zero-or-more steps from the initial configuration

```
definition run :: "'p config ⇒ bool" where
```

```
  "run cfg ⟷ run_step** init_config cfg"

lemma run_init [simp]: "run init_config"
  by (simp add: run_def)

lemma run_extend:
  "run cfg ⟹ run_step cfg cfg' ⟹ run cfg'"
  unfolding run_def by simp
```

46 events_of on per-process updates

```
lemma events_of_extend:
  "events_of (H(p := H p @ [e])) = events_of H ∪ {e}"
proof -
  have "(⋃q. set ((H(p := H p @ [e])) q))
          = (⋃q. if q = p then set (H p @ [e]) else set (H q))"
    by (rule SUP_cong) auto
  also have
    "... = set (H p @ [e]) ∪ (⋃q∈{q. q ≠ p}. set (H q))"
    by (auto split: if_split_asm)
  also have
    "... = (set (H p) ∪ {e}) ∪ (⋃q∈{q. q ≠ p}. set (H q))"
    by simp
  also have
    "(⋃q. set (H q)) = set (H p) ∪ (⋃q∈{q. q ≠ p}. set (H q))"
    by auto
  ultimately have "events_of (H(p := H p @ [e])) = events_of H ∪ {e}"
    by (simp add: events_of_def)
  thus ?thesis .
qed
```

47 The key invariant: correct-to-correct Sends are buffered or received

For every correct-to-correct `(p, q, m)` triple, if there is a `Send p n q m` event in the history then there is either a matching `Receive q n' p m` event or a buffer entry `(p, q, m)`.

This is the standard "conservation of in-flight messages" invariant: every correct-to-correct Send introduces one buffer entry; every correct-from-correct Receive removes one buffer entry that pairs with an existing Send. Byzantine steps neither add correct-to-correct sends nor remove buffer entries, so the invariant is preserved trivially.

```
definition sends_match_inv :: "'p config ⇒ bool" where
  "sends_match_inv cfg ⟷
     (∀p n q m. p ∈ correct ⟶ q ∈ correct
        ⟶ Send p n q m ∈ events_of (cfg_hist cfg)
        ⟶ (∃n'. Receive q n' p m ∈ events_of (cfg_hist cfg))
            ∨ (p, q, m) ∈# cfg_inflight cfg)"

lemma sends_match_inv_init [simp]:
```

```
  "sends_match_inv init_config"
  by (simp add: sends_match_inv_def init_config_def empty_inflight_def
                events_of_def)
```

Every step preserves `sends_match_inv`.

```
lemma sends_match_inv_step:
  assumes "sends_match_inv cfg"
      and "run_step cfg cfg'"
  shows "sends_match_inv cfg'"
  using assms(2,1)
proof induction
  case (step_internal p n cfg cfg')
  have ev: "events_of (cfg_hist cfg') = events_of (cfg_hist cfg) ∪ {Internal p n}"
    using step_internal.hyps(3) events_of_extend by simp
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_internal.hyps(3) by simp
  show ?case
  proof (unfold sends_match_inv_def, intro allI impI)
    fix p' n' q' m'
    assume pc: "p' ∈ correct" and qc: "q' ∈ correct"
       and sevent: "Send p' n' q' m' ∈ events_of (cfg_hist cfg')"
    from sevent ev have "Send p' n' q' m' ∈ events_of (cfg_hist cfg)"
      by auto
    with step_internal.prems pc qc
    have IH:
      "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg))
        ∨ (p', q', m') ∈# cfg_inflight cfg"
      by (auto simp: sends_match_inv_def)
    thus "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg'))
          ∨ (p', q', m') ∈# cfg_inflight cfg'"
    proof
      assume "∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)"
      then obtain n'' where "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)" by
blast
      hence "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg')" using ev by auto
      thus ?thesis by blast
    next
      assume "(p', q', m') ∈# cfg_inflight cfg"
      thus ?thesis using buf by simp
    qed
  qed
next
  case (step_send p q n cfg cfg' m)
  let ?new = "Send p n q m"
  have ev: "events_of (cfg_hist cfg') = events_of (cfg_hist cfg) ∪ {?new}"
    using step_send.hyps(4) events_of_extend by simp
  have buf: "cfg_inflight cfg' = cfg_inflight cfg ∪# {# (p, q, m) }"
    using step_send.hyps(4) by simp
  show ?case
  proof (unfold sends_match_inv_def, intro allI impI)
    fix p' n' q' m'
    assume pc: "p' ∈ correct" and qc: "q' ∈ correct"
```

```
      and sevent: "Send p' n' q' m' ∈ events_of (cfg_hist cfg')"
    from sevent ev
    have "Send p' n' q' m' ∈ events_of (cfg_hist cfg) ∨
          Send p' n' q' m' = ?new"
      by auto
    thus "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg'))
           ∨ (p', q', m') ∈# cfg_inflight cfg'"
    proof
      assume "Send p' n' q' m' ∈ events_of (cfg_hist cfg)"
      with step_send.prems pc qc
      have IH:
        "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg))
         ∨ (p', q', m') ∈# cfg_inflight cfg"
        by (auto simp: sends_match_inv_def)
      thus ?thesis
      proof
        assume "∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)"
        then obtain n''
          where "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)" by blast
        hence "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg')"
          using ev by auto
        thus ?thesis by blast
      next
        assume "(p', q', m') ∈# cfg_inflight cfg"
        thus ?thesis using buf by simp
      qed
    next
      assume eq: "Send p' n' q' m' = ?new"
      hence eq': "(p', q', m') = (p, q, m)" by simp
      have "(p, q, m) ∈# cfg_inflight cfg'"
        using buf by simp
      with eq' have "(p', q', m') ∈# cfg_inflight cfg'" by simp
      thus ?thesis by blast
    qed
  qed
next
  case (step_recv q p m cfg n cfg')
  let ?new = "Receive q n p m"
  have ev: "events_of (cfg_hist cfg') = events_of (cfg_hist cfg) ∪ {?new}"
    using step_recv.hyps(5) events_of_extend by simp
  have buf: "cfg_inflight cfg' = cfg_inflight cfg -# (p, q, m)"
    using step_recv.hyps(5) by simp
  have buf_in: "(p, q, m) ∈# cfg_inflight cfg" by (rule step_recv.hyps(3))
  show ?case
  proof (unfold sends_match_inv_def, intro allI impI)
    fix p' n' q' m'
    assume pc: "p' ∈ correct" and qc: "q' ∈ correct"
       and sevent: "Send p' n' q' m' ∈ events_of (cfg_hist cfg')"
    — A Send event cannot equal the Receive ?new.
    from sevent ev
    have "Send p' n' q' m' ∈ events_of (cfg_hist cfg)" by auto
    with step_recv.prems pc qc
```

```
    have IH:
      "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg))
       ∨ (p', q', m') ∈# cfg_inflight cfg"
      by (auto simp: sends_match_inv_def)
    thus "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg'))
          ∨ (p', q', m') ∈# cfg_inflight cfg'"
    proof
      assume "∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)"
      then obtain n''
        where "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)" by blast
      hence "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg')"
        using ev by auto
      thus ?thesis by blast
    next
      assume buf_in': "(p', q', m') ∈# cfg_inflight cfg"
      show ?thesis
      proof (cases "(p', q', m') = (p, q, m)")
        case True
        have "Receive q n p m ∈ events_of (cfg_hist cfg')"
          using ev by simp
        with True show ?thesis by auto
      next
        case neq: False
        have neq': "(p', q', m') ≠ (p, q, m)"
          using neq by simp
        have eq_count:
          "(cfg_inflight cfg -# (p, q, m)) (p', q', m')
             = cfg_inflight cfg (p', q', m')"
          using neq' by auto
        have "(p', q', m') ∈# cfg_inflight cfg -# (p, q, m)"
          using buf_in' eq_count by simp
        hence "(p', q', m') ∈# cfg_inflight cfg'"
          using buf by simp
        thus ?thesis by blast
      qed
    qed
  qed
next
  case (step_byzantine p new_event cfg cfg')
  have ev: "events_of (cfg_hist cfg') = events_of (cfg_hist cfg) ∪ {new_event}"
    using step_byzantine.hyps(4) events_of_extend by simp
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_byzantine.hyps(4) by simp
  show ?case
  proof (unfold sends_match_inv_def, intro allI impI)
    fix p' n' q' m'
    assume pc: "p' ∈ correct" and qc: "q' ∈ correct"
       and sevent: "Send p' n' q' m' ∈ events_of (cfg_hist cfg')"
    — A correct-Send cannot be the newly added Byzantine event: the new event's proc_of is
p ∈ byzantine, whereas the Send's proc_of would be p' ∈ correct.
    have pby: "p ∈ byzantine" by (rule step_byzantine.hyps(1))
    have neq: "p' ≠ p"
```

```
    using pc pby partition_disj by blast
  have proc_new: "proc_of new_event = p"
    by (rule step_byzantine.hyps(2))
  have proc_send: "proc_of (Send p' n' q' m') = p'" by simp
  have not_new: "Send p' n' q' m' ≠ new_event"
    using neq proc_new proc_send by metis
  from sevent ev
  have "Send p' n' q' m' ∈ events_of (cfg_hist cfg) ∨
        Send p' n' q' m' = new_event"
    by auto
  with not_new
  have sevent_cfg: "Send p' n' q' m' ∈ events_of (cfg_hist cfg)"
    by blast
  with step_byzantine.prems pc qc
  have IH:
    "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg))
     ∨ (p', q', m') ∈# cfg_inflight cfg"
    by (auto simp: sends_match_inv_def)
  thus "(∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg'))
        ∨ (p', q', m') ∈# cfg_inflight cfg'"
  proof
    assume "∃n''. Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)"
    then obtain n''
      where "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg)" by blast
    hence "Receive q' n'' p' m' ∈ events_of (cfg_hist cfg')"
      using ev by auto
    thus ?thesis by blast
  next
    assume "(p', q', m') ∈# cfg_inflight cfg"
    thus ?thesis using buf by simp
  qed
 qed
qed
```

Invariant is preserved by any run (zero-or-more steps).

```
lemma sends_match_inv_run:
  assumes "run cfg"
  shows "sends_match_inv cfg"
  using assms
  unfolding run_def
proof (induction rule: rtranclp_induct)
  case base
  show ?case by (rule sends_match_inv_init)
next
  case (step y z)
  from sends_match_inv_step[OF step.IH step.hyps(2)]
  show ?case .
qed
```

48 Fairness implies delivery

The headline Phase 6 theorem: if a run reaches a configuration with empty in-flight buffer, the configuration's history satisfies `messages_delivered_among correct`. Operationally: "the schedule was completed fairly" $\Longrightarrow$ "every correct-to-correct send was delivered".

```
theorem fairness_implies_delivery:
  assumes "run cfg"
      and "cfg_inflight cfg = empty_inflight"
  shows "messages_delivered_among correct (cfg_hist cfg)"
proof (unfold messages_delivered_among_def, intro allI impI)
  fix p n q m
  assume pc: "p ∈ correct" and qc: "q ∈ correct"
     and sevent: "Send p n q m ∈ events_of (cfg_hist cfg)"
  from sends_match_inv_run[OF assms(1)] pc qc sevent
  have alt: "(∃n'. Receive q n' p m ∈ events_of (cfg_hist cfg))
             ∨ (p, q, m) ∈# cfg_inflight cfg"
    by (auto simp: sends_match_inv_def)
  have "¬ (p, q, m) ∈# cfg_inflight cfg"
    using assms(2) by (simp add: empty_inflight_def)
  with alt show
    "∃n'. Receive q n' p m ∈ events_of (cfg_hist cfg)"
    by blast
qed
```

49 wf_history is a run invariant

Each step appends one event whose `proc_of` matches the target process and whose `seq_of` is exactly `Suc (length ...)` of the existing per-process list. These are exactly the `wf_history_local` conditions for an extension by one element, so each step preserves `wf_history`.

For `step_byzantine` the `proc_of` and `seq_of` constraints are explicit premises; for `step_internal` / `step_send` / `step_recv` they follow from the event constructors used and the `n = Suc (length ...)` side condition.

```
lemma wf_history_local_extend:
  assumes wf:     "wf_history_local p es"
      and proc_e: "proc_of e = p"
      and seq_e:  "seq_of e = Suc (length es)"
  shows   "wf_history_local p (es @ [e])"
proof (unfold wf_history_local_def, intro conjI ballI allI impI)
  fix ev assume mem: "ev ∈ set (es @ [e])"
  show "proc_of ev = p"
  proof (cases "ev ∈ set es")
    case True
    thus ?thesis using wf unfolding wf_history_local_def by blast
  next
    case False
    with mem have "ev = e" by simp
    thus ?thesis using proc_e by simp
  qed
```

```
next
  fix k assume k: "k < length (es @ [e])"
  show "seq_of ((es @ [e]) ! k) = Suc k"
  proof (cases "k < length es")
    case True
    have "(es @ [e]) ! k = es ! k"
      using True by (simp add: nth_append)
    moreover have "seq_of (es ! k) = Suc k"
      using wf True unfolding wf_history_local_def by blast
    ultimately show ?thesis by simp
  next
    case False
    with k have k_eq: "k = length es" by simp
    have "(es @ [e]) ! k = e" using k_eq by simp
    moreover have "seq_of e = Suc (length es)" by (rule seq_e)
    ultimately show ?thesis using k_eq by simp
  qed
qed

lemma wf_history_extend:
  assumes wf:     "wf_history H"
      and proc_e: "proc_of e = p"
      and seq_e:  "seq_of e = Suc (length (H p))"
  shows "wf_history (H(p := H p @ [e]))"
proof (unfold wf_history_def, intro allI)
  fix p'
  show "wf_history_local p' ((H(p := H p @ [e])) p')"
  proof (cases "p' = p")
    case True
    have "wf_history_local p (H p)"
      using wf unfolding wf_history_def by blast
    hence "wf_history_local p (H p @ [e])"
      by (rule wf_history_local_extend[OF _ proc_e seq_e])
    thus ?thesis using True by simp
  next
    case False
    hence eq: "(H(p := H p @ [e])) p' = H p'" by simp
    have "wf_history_local p' (H p')"
      using wf unfolding wf_history_def by blast
    thus ?thesis using eq by simp
  qed
qed

lemma wf_history_init [simp]:
  "wf_history (cfg_hist init_config)"
  by (simp add: init_config_def wf_history_def wf_history_local_def)

lemma wf_history_step:
  assumes wf:   "wf_history (cfg_hist cfg)"
      and step: "run_step cfg cfg'"
  shows "wf_history (cfg_hist cfg')"
  using step wf
```

```
proof induction
  case (step_internal p n cfg cfg')
  have proc_new: "proc_of (Internal p n) = p" by simp
  have seq_new: "seq_of (Internal p n) = Suc (length (cfg_hist cfg p))"
    using step_internal.hyps(2) by simp
  have wf_prev: "wf_history (cfg_hist cfg)" by (rule step_internal.prems)
  have wf_ext:
    "wf_history ((cfg_hist cfg)(p := cfg_hist cfg p @ [Internal p n]))"
    by (rule wf_history_extend[OF wf_prev proc_new seq_new])
  have cfg'_eq:
    "cfg_hist cfg' = (cfg_hist cfg)(p := cfg_hist cfg p @ [Internal p n])"
    using step_internal.hyps(3) by simp
  show ?case using wf_ext cfg'_eq by metis
next
  case (step_send p q n cfg cfg' m)
  have proc_new: "proc_of (Send p n q m) = p" by simp
  have seq_new: "seq_of (Send p n q m) = Suc (length (cfg_hist cfg p))"
    using step_send.hyps(3) by simp
  have wf_prev: "wf_history (cfg_hist cfg)" by (rule step_send.prems)
  have wf_ext:
    "wf_history ((cfg_hist cfg)(p := cfg_hist cfg p @ [Send p n q m]))"
    by (rule wf_history_extend[OF wf_prev proc_new seq_new])
  have cfg'_eq:
    "cfg_hist cfg' = (cfg_hist cfg)(p := cfg_hist cfg p @ [Send p n q m])"
    using step_send.hyps(4) by simp
  show ?case using wf_ext cfg'_eq by metis
next
  case (step_recv q p m cfg n cfg')
  have proc_new: "proc_of (Receive q n p m) = q" by simp
  have seq_new: "seq_of (Receive q n p m) = Suc (length (cfg_hist cfg q))"
    using step_recv.hyps(4) by simp
  have wf_prev: "wf_history (cfg_hist cfg)" by (rule step_recv.prems)
  have wf_ext:
    "wf_history ((cfg_hist cfg)(q := cfg_hist cfg q @ [Receive q n p m]))"
    by (rule wf_history_extend[OF wf_prev proc_new seq_new])
  have cfg'_eq:
    "cfg_hist cfg' = (cfg_hist cfg)(q := cfg_hist cfg q @ [Receive q n p m])"
    using step_recv.hyps(5) by simp
  show ?case using wf_ext cfg'_eq by metis
next
  case (step_byzantine p new_event cfg cfg')
  have proc_new: "proc_of new_event = p" by (rule step_byzantine.hyps(2))
  have seq_new: "seq_of new_event = Suc (length (cfg_hist cfg p))"
    by (rule step_byzantine.hyps(3))
  have wf_prev: "wf_history (cfg_hist cfg)" by (rule step_byzantine.prems)
  have wf_ext:
    "wf_history ((cfg_hist cfg)(p := cfg_hist cfg p @ [new_event]))"
    by (rule wf_history_extend[OF wf_prev proc_new seq_new])
  have cfg'_eq:
    "cfg_hist cfg' = (cfg_hist cfg)(p := cfg_hist cfg p @ [new_event])"
    using step_byzantine.hyps(4) by simp
  show ?case using wf_ext cfg'_eq by metis
```

```
qed

lemma wf_history_run:
  assumes "run cfg"
  shows "wf_history (cfg_hist cfg)"
  using assms unfolding run_def
proof (induction rule: rtranclp_induct)
  case base
  show ?case by simp
next
  case (step y z)
  from wf_history_step[OF step.IH step.hyps(2)]
  show ?case .
qed
```

50 Closing the Phase 5 gap: run produces mode-admissible histories

The combined headline: any run that fairly completes (empty in-flight buffer at the end) produces a history that is `mode_admissible` under both `Unicast` and `Broadcast` – because the run preserves `wf_history` (`wf_history_run`) and empty buffer gives `messages_delivered_among` (`fairness_implies_delivery`).

This closes the Phase 5 gap: the unicast / broadcast cases of `mode_admissible` were the structural content of T6 / T7's operational claim, and we have now exhibited an operational construction that produces such histories. Composing with Phase 5's `CD_B_solvable_unicast_operational` and `CD_B_solvable_broadcast_operational` gives a fully derived chain

`operational run` $\longrightarrow$ `mode-admissible H` $\longrightarrow$ `naive algorithm solves CD_B`

with no remaining operational hypothesis.

```
theorem run_completes_to_mode_admissible_unicast:
  assumes run_cfg: "run cfg"
      and empty:   "cfg_inflight cfg = empty_inflight"
  shows "mode_admissible Unicast (cfg_hist cfg)"
proof -
  have wf: "wf_history (cfg_hist cfg)" by (rule wf_history_run[OF run_cfg])
  have del: "messages_delivered_among correct (cfg_hist cfg)"
    by (rule fairness_implies_delivery[OF run_cfg empty])
  show ?thesis
    unfolding mode_admissible_def using wf del by simp
qed

theorem run_completes_to_mode_admissible_broadcast:
  assumes run_cfg: "run cfg"
      and empty:   "cfg_inflight cfg = empty_inflight"
  shows "mode_admissible Broadcast (cfg_hist cfg)"
proof -
  have wf: "wf_history (cfg_hist cfg)" by (rule wf_history_run[OF run_cfg])
  have del: "messages_delivered_among correct (cfg_hist cfg)"
    by (rule fairness_implies_delivery[OF run_cfg empty])
```

```
  show ?thesis
    unfolding mode_admissible_def using wf del by simp
qed
```

51 Deadlock freedom

The remaining gap below Phase 7 is the temporal-fairness gap: in a real network the buffer is drained by a fair scheduler over an infinite (or unboundedly long) execution. Phase 8 takes one step toward closing it by establishing *deadlock freedom*: from any configuration reachable via `run` with a non-empty in-flight buffer, a `run_step` can always be taken (specifically a `step_recv`). Together with ⟦`run ?cfg; cfg_inflight ?cfg = empty_inflight`⟧ ⟹ `messages_delivered_among correct (cfg_hist ?cfg)` this means: as long as the scheduler keeps making moves, drainage continues; the model never gets stuck.

What is left for a true closure is the liveness theorem "every fair execution eventually has empty buffer". That requires a temporal predicate over infinite executions – stream / coinductive machinery – and is documented as the remaining out-of-scope step.

51.1 The buffer only contains correct-to-correct triples

Invariant `buffer_correct_inv`: every triple `(p, q, m)` in the buffer has both `p ∈ correct` and `q ∈ correct`. This follows directly from the structure of `run_step`: only the `step_send` rule adds to the buffer, and its premises require both endpoints to be in `correct`.

```
definition buffer_correct_inv :: "'p config ⇒ bool" where
  "buffer_correct_inv cfg ⟷
     (∀p q m. (p, q, m) ∈# cfg_inflight cfg ⟶ p ∈ correct ∧ q ∈ correct)"

lemma buffer_correct_inv_init [simp]:
  "buffer_correct_inv init_config"
  by (simp add: buffer_correct_inv_def init_config_def empty_inflight_def)

lemma buffer_correct_inv_step:
  assumes "buffer_correct_inv cfg"
      and "run_step cfg cfg'"
  shows "buffer_correct_inv cfg'"
  using assms(2,1)
proof induction
  case (step_internal p n cfg cfg')
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_internal.hyps(3) by simp
  thus ?case
    using step_internal.prems by (simp add: buffer_correct_inv_def)
next
  case (step_send p q n cfg cfg' m)
  have buf: "cfg_inflight cfg' = cfg_inflight cfg ∪# {# (p, q, m) }"
    using step_send.hyps(4) by simp
  have pc: "p ∈ correct" by (rule step_send.hyps(1))
  have qc: "q ∈ correct" by (rule step_send.hyps(2))
  show ?case
```

```
  proof (unfold buffer_correct_inv_def, intro allI impI)
    fix p' q' m'
    assume "(p', q', m') ∈# cfg_inflight cfg'"
    hence in_buf: "0 < (cfg_inflight cfg ∪# {# (p, q, m) }) (p', q', m')"
      using buf by simp
    show "p' ∈ correct ∧ q' ∈ correct"
    proof (cases "(p', q', m') = (p, q, m)")
      case True
      thus ?thesis using pc qc by simp
    next
      case nq: False
      have neq: "(p, q, m) ≠ (p', q', m')" using nq by auto
      have "(cfg_inflight cfg ∪# {# (p, q, m) }) (p', q', m')
              = cfg_inflight cfg (p', q', m')"
        using neq by simp
      with in_buf have "0 < cfg_inflight cfg (p', q', m')" by simp
      hence in_cfg: "(p', q', m') ∈# cfg_inflight cfg" by simp
      show ?thesis
        using step_send.prems in_cfg
        unfolding buffer_correct_inv_def by blast
    qed
  qed
next
  case (step_recv q p m cfg n cfg')
  have buf: "cfg_inflight cfg' = cfg_inflight cfg -# (p, q, m)"
    using step_recv.hyps(5) by simp
  show ?case
  proof (unfold buffer_correct_inv_def, intro allI impI)
    fix p' q' m'
    assume "(p', q', m') ∈# cfg_inflight cfg'"
    hence "0 < (cfg_inflight cfg -# (p, q, m)) (p', q', m')"
      using buf by simp
```

— Removing one occurrence only decreases the count, so `(p',q',m')` was already in the original buffer.

```
    hence "0 < cfg_inflight cfg (p', q', m')"
      by (auto split: if_split_asm)
    hence in_cfg: "(p', q', m') ∈# cfg_inflight cfg" by simp
    show "p' ∈ correct ∧ q' ∈ correct"
      using step_recv.prems in_cfg
      unfolding buffer_correct_inv_def by blast
  qed
next
  case (step_byzantine p new_event cfg cfg')
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_byzantine.hyps(4) by simp
  thus ?case
    using step_byzantine.prems by (simp add: buffer_correct_inv_def)
qed

lemma buffer_correct_inv_run:
  assumes "run cfg"
  shows "buffer_correct_inv cfg"
```

```
  using assms unfolding run_def
proof (induction rule: rtranclp_induct)
  case base
  show ?case by simp
next
  case (step y z)
  from buffer_correct_inv_step[OF step.IH step.hyps(2)]
  show ?case .
qed
```

51.2 Progress: a non-empty buffer always permits a step

From a run-reachable configuration with non-empty buffer, a `step_recv` rule application is always available: pick any buffered triple (it exists, since the buffer is non-empty), and the `buffer_correct_inv` invariant guarantees both endpoints are correct, which is exactly what `step_recv`'s premises require.

```
lemma not_drained_can_step:
  assumes run_cfg:  "run cfg"
      and not_done: "cfg_inflight cfg ≠ empty_inflight"
  shows "∃cfg'. run_step cfg cfg'"
proof -
  have inv: "buffer_correct_inv cfg" by (rule buffer_correct_inv_run[OF run_cfg])
```

— A non-empty buffer means `cfg_inflight cfg` is not the constantly-zero function, so some triple has positive count.

```
  from not_done obtain p q m
    where pos: "cfg_inflight cfg (p, q, m) > 0"
  proof -
    have "∃x. cfg_inflight cfg x ≠ empty_inflight x"
      using not_done by (auto simp: empty_inflight_def fun_eq_iff)
    then obtain x where x_ne: "cfg_inflight cfg x ≠ 0"
      by (auto simp: empty_inflight_def)
    obtain p' q' m' where x_eq: "x = (p', q', m')"
      by (cases x) auto
    from x_ne x_eq have "cfg_inflight cfg (p', q', m') > 0" by simp
    thus thesis by (rule that)
  qed
  hence in_buf: "(p, q, m) ∈# cfg_inflight cfg" by simp
  from inv in_buf have pc: "p ∈ correct" and qc: "q ∈ correct"
    by (auto simp: buffer_correct_inv_def)
  let ?n = "Suc (length (cfg_hist cfg q))"
  let ?cfg' =
    "cfg (| cfg_hist := (cfg_hist cfg)
                          (q := cfg_hist cfg q @ [Receive q ?n p m]),
           cfg_inflight := cfg_inflight cfg -# (p, q, m) |)"
  have "run_step cfg ?cfg'"
    by (rule run_step.step_recv[OF qc pc in_buf refl refl])
  thus ?thesis by blast
qed
```

What this proves. ⟦`run ?cfg; cfg_inflight ?cfg ≠ empty_inflight`⟧ ⟹ ∃`cfg'. run_step ?cfg cfg'` together with ⟦`run ?cfg; cfg_inflight ?cfg = empty_inflight`⟧ ⟹ `messages_delivered_among correct (cfg_hist ?cfg)` gives the two halves of behavioural well-formedness for our

model:

- `step_recv` is applicable whenever the buffer is non-empty (deadlock freedom).
- Once the buffer is empty, the history is `messages_delivered_among`-complete (delivery).

What remains. A real-world fairness theorem of the form "every fair infinite execution eventually has empty buffer" would require modelling infinite executions as streams and stating fairness as a temporal predicate – separate machinery the development can be parametric over.

end — context `byzantineSystem`

end

theory `Liveness`
  imports `Execution_Model`
begin

context `byzantineSystem`
begin

52 Infinite executions

An infinite execution starts at `init_config` and takes a single `run_step` at every adjacent index. Treating an infinite execution as a function `nat ⇒ 'p config` sidesteps the need for a coinductive stream codatatype while still letting us state and prove temporal properties ("eventually", "always").

```
definition infinite_run :: "(nat ⇒ 'p config) ⇒ bool" where
  "infinite_run E ⟷
     E 0 = init_config ∧ (∀i. run_step (E i) (E (Suc i)))"
```

Every prefix of an infinite run is itself a finite run. Composed with the existing invariant lemmas (`wf_history_run`, `sends_match_inv_run`, `buffer_correct_inv_run`) this gives the pointwise invariants below.

```
lemma infinite_run_reach_each:
  assumes "infinite_run E"
  shows   "run (E i)"
proof (induction i)
  case 0
  have "E 0 = init_config" using assms by (simp add: infinite_run_def)
  thus ?case by (simp add: run_def)
next
  case (Suc i)
  have step: "run_step (E i) (E (Suc i))"
    using assms by (simp add: infinite_run_def)
  show ?case by (rule run_extend[OF Suc.IH step])
qed
```

```
corollary infinite_run_wf_history:
  assumes "infinite_run E"
  shows   "wf_history (cfg_hist (E i))"
  using wf_history_run[OF infinite_run_reach_each[OF assms, of i]] .

corollary infinite_run_sends_match_inv:
  assumes "infinite_run E"
  shows   "sends_match_inv (E i)"
  using sends_match_inv_run[OF infinite_run_reach_each[OF assms, of i]] .

corollary infinite_run_buffer_correct_inv:
  assumes "infinite_run E"
  shows   "buffer_correct_inv (E i)"
  using buffer_correct_inv_run[OF infinite_run_reach_each[OF assms, of i]] .
```

53 Event monotonicity in the run index

Every kind of step appends exactly one event to a single process's history. In particular, `events_of (cfg_hist _)` only grows.

```
lemma run_step_hist_extends:
  assumes "run_step cfg cfg'"
  shows "∃p e. cfg_hist cfg' = (cfg_hist cfg)(p := cfg_hist cfg p @ [e])"
  using assms
proof induction
  case (step_internal p n cfg cfg')
  show ?case using step_internal.hyps(3) by auto
next
  case (step_send p q n cfg cfg' m)
  show ?case using step_send.hyps(4) by auto
next
  case (step_recv q p m cfg n cfg')
  show ?case using step_recv.hyps(5) by auto
next
  case (step_byzantine p new_event cfg cfg')
  show ?case using step_byzantine.hyps(4) by auto
qed

lemma events_of_subset_step:
  assumes "run_step cfg cfg'"
  shows "events_of (cfg_hist cfg) ⊆ events_of (cfg_hist cfg')"
proof -
  obtain p e where eq:
    "cfg_hist cfg' = (cfg_hist cfg)(p := cfg_hist cfg p @ [e])"
    using run_step_hist_extends[OF assms] by blast
  hence "events_of (cfg_hist cfg') = events_of (cfg_hist cfg) ∪ {e}"
    by (simp add: events_of_extend)
  thus ?thesis by blast
qed

lemma events_of_subset_infinite:
  assumes inf: "infinite_run E"
```

```
      and ij:  "i ≤ j"
  shows "events_of (cfg_hist (E i)) ⊆ events_of (cfg_hist (E j))"
  using ij
proof (induction j)
  case 0
  hence "i = 0" by simp
  thus ?case by simp
next
  case (Suc j)
  show ?case
  proof (cases "i = Suc j")
    case True
    thus ?thesis by simp
  next
    case False
    with Suc.prems have "i ≤ j" by simp
    hence sub1: "events_of (cfg_hist (E i)) ⊆ events_of (cfg_hist (E j))"
      by (rule Suc.IH)
    have step: "run_step (E j) (E (Suc j))"
      using inf by (simp add: infinite_run_def)
    have sub2: "events_of (cfg_hist (E j)) ⊆ events_of (cfg_hist (E (Suc j)))"
      by (rule events_of_subset_step[OF step])
    show ?thesis using sub1 sub2 by blast
  qed
qed
```

54 A buffer triple can only disappear via `step_recv`

The pivotal technical lemma: if a triple is in the buffer before a step and not after, the step must be `step_recv` removing that very triple – which also adds a matching `Receive` event to the history.

The variables `pp, qq, mm` name the triple from outside the lemma, to avoid name clashes with the case-bound variables of the `run_step` rule (which use `p, q, m`).

```
lemma step_removes_triple_is_recv:
  fixes pp qq :: 'p and mm :: nat
  assumes "run_step cfg cfg'"
      and "(pp, qq, mm) ∈# cfg_inflight cfg"
      and "¬ (pp, qq, mm) ∈# cfg_inflight cfg'"
  shows "∃n. Receive qq n pp mm ∈ events_of (cfg_hist cfg')"
  using assms
proof induction
  case (step_internal p n cfg cfg')
  — Internal steps leave the buffer unchanged; the assumption pair contradicts.
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_internal.hyps(3) by simp
  with step_internal.prems show ?case by simp
next
  case (step_send p q n cfg cfg' m)
  — Sends only add to the buffer; (pp, qq, mm) stays in.
  have buf: "cfg_inflight cfg' = cfg_inflight cfg ∪# {# (p, q, m) }"
    using step_send.hyps(4) by simp
```

```
  have count_le:
    "cfg_inflight cfg (pp, qq, mm) ≤ cfg_inflight cfg' (pp, qq, mm)"
    using buf by simp
  hence "(pp, qq, mm) ∈# cfg_inflight cfg'"
    using step_send.prems(1) by simp
  with step_send.prems(2) show ?case by simp
next
  case (step_recv q p m cfg n cfg')
  have buf: "cfg_inflight cfg' = cfg_inflight cfg -# (p, q, m)"
    using step_recv.hyps(5) by simp
  show ?case
  proof (cases "(pp, qq, mm) = (p, q, m)")
    case True
    — This is the live case: the step adds Receive q n p m, which equals Receive qq n pp
mm.
    have hist_eq:
      "cfg_hist cfg'
         = (cfg_hist cfg)(q := cfg_hist cfg q @ [Receive q n p m])"
      using step_recv.hyps(5) by simp
    have "events_of (cfg_hist cfg')
          = events_of (cfg_hist cfg) ∪ {Receive q n p m}"
      using hist_eq by (simp add: events_of_extend)
    hence "Receive q n p m ∈ events_of (cfg_hist cfg')" by blast
    with True show ?thesis by blast
  next
    case False
    — A different triple was removed; (pp, qq, mm) count is unchanged, contradicting the
post-assumption.
    have eq_count:
      "cfg_inflight cfg' (pp, qq, mm) = cfg_inflight cfg (pp, qq, mm)"
      using False buf by auto
    hence "(pp, qq, mm) ∈# cfg_inflight cfg'"
      using step_recv.prems(1) by simp
    with step_recv.prems(2) show ?thesis by simp
  qed
next
  case (step_byzantine p new_event cfg cfg')
  — Byzantine steps leave the buffer unchanged.
  have buf: "cfg_inflight cfg' = cfg_inflight cfg"
    using step_byzantine.hyps(4) by simp
  with step_byzantine.prems show ?case by simp
qed
```

55 Finding the step at which a triple is removed

If a triple is in the buffer at index `i` and not in the buffer at some later index `j`, there is a specific step `k ∈ [i, j)` where the triple goes from "in" to "out". By ⟦`run_step ?cfg ?cfg'; (?pp, ?qq, ?mm) ∈# cfg_inflight ?cfg; ¬ (?pp, ?qq, ?mm) ∈# cfg_inflight ?cfg'`⟧ ⟹ ∃`n. Receive ?qq n ?pp ?mm ∈ events_of (cfg_hist ?cfg')` the step at `k` adds a matching `Receive`.

lemma `triple_disappears_implies_recv:`

```
  assumes inf:    "infinite_run E"
      and ij:     "i ≤ j"
      and in_buf: "(pp, qq, mm) ∈# cfg_inflight (E i)"
      and out:    "¬ (pp, qq, mm) ∈# cfg_inflight (E j)"
  shows "∃k n. i ≤ k ∧ k < j
              ∧ Receive qq n pp mm ∈ events_of (cfg_hist (E (Suc k)))"
proof -
```

— Find the smallest index `k_min ∈ [i, j]` where the triple is absent. It is at least `i + 1`, because the triple is present at `i`.

```
  define S where
    "S = {k. i ≤ k ∧ k ≤ j ∧ ¬ (pp, qq, mm) ∈# cfg_inflight (E k)}"
  have S_subset: "S ⊆ {i..j}" by (auto simp: S_def)
  have S_fin: "finite S"
    using S_subset by (rule finite_subset) simp
  have j_in_S: "j ∈ S"
    using ij out by (simp add: S_def)
  hence S_ne: "S ≠ {}" by blast
  define k_min where "k_min = Min S"
  have k_min_in_S: "k_min ∈ S" using S_fin S_ne k_min_def Min_in by blast
  have k_min_le_j: "k_min ≤ j" using k_min_in_S by (auto simp: S_def)
  have k_min_ge_i: "i ≤ k_min" using k_min_in_S by (auto simp: S_def)
  have not_in_k_min: "¬ (pp, qq, mm) ∈# cfg_inflight (E k_min)"
    using k_min_in_S by (auto simp: S_def)
  have i_neq_k_min: "i ≠ k_min" using in_buf not_in_k_min by metis
  hence k_min_gt_i: "i < k_min" using k_min_ge_i by simp

  define k where "k = k_min - 1"
  have Suc_k_eq: "Suc k = k_min" using k_min_gt_i k_def by simp
  have i_le_k: "i ≤ k" using k_min_gt_i k_def by simp
  have k_lt_j: "k < j" using k_min_le_j k_min_gt_i k_def by simp
```

— `k` is below `k_min`, so `k ∉ S`; combined with the range constraints, this forces `(pp, qq, mm) ∈# E k`.

```
  have k_lt_k_min: "k < k_min" using k_min_gt_i k_def by simp
  have k_not_in_S: "k ∉ S"
  proof
    assume "k ∈ S"
    hence "k_min ≤ k" using S_fin k_min_def by (simp add: Min_le)
    thus False using k_lt_k_min by simp
  qed
  have k_le_j: "k ≤ j" using k_lt_j by simp
  have k_in_buf: "(pp, qq, mm) ∈# cfg_inflight (E k)"
  proof (rule ccontr)
    assume not_in_k: "¬ (pp, qq, mm) ∈# cfg_inflight (E k)"
    have "k ∈ S"
      using i_le_k k_le_j not_in_k by (auto simp: S_def)
    with k_not_in_S show False by contradiction
  qed
  have suc_k_not_in_buf: "¬ (pp, qq, mm) ∈# cfg_inflight (E (Suc k))"
    using Suc_k_eq not_in_k_min by simp

  have step: "run_step (E k) (E (Suc k))"
```

```
    using inf by (simp add: infinite_run_def)
  from step_removes_triple_is_recv[OF step k_in_buf suc_k_not_in_buf]
  obtain n where rec:
    "Receive qq n pp mm ∈ events_of (cfg_hist (E (Suc k)))" by blast

  from i_le_k k_lt_j rec show ?thesis by blast
qed
```

56 Fair runs

A run is fair if every buffer triple is eventually removed from the buffer. This is the standard weak-fairness condition for `step_recv`: each enabled receive is eventually taken (or, more formally, each in-flight triple eventually leaves the buffer).

```
definition fair_run :: "(nat ⇒ 'p config) ⇒ bool" where
  "fair_run E ⟷
     (∀i p q m. (p, q, m) ∈# cfg_inflight (E i)
                ⟶ (∃j. i ≤ j ∧ ¬ (p, q, m) ∈# cfg_inflight (E j)))"
```

57 Liveness: fair runs deliver every correct-to-correct send

The headline liveness theorem. For every fair infinite run and every correct-to-correct `Send p n q m` event in some `cfg_hist (E i)`, there is a step `j` and seq number `n'` such that `Receive q n' p m` is in `cfg_hist (E j)`.

The proof composes:

1. `infinite_run ?E ⟹ sends_match_inv (?E ?i)`: the `sends_match_inv` invariant holds at every step. So either the matching receive is already in `E i`, or the triple is in the buffer at `E i`.
2. In the latter case, fairness gives a `j' ≥ i` at which the triple is out of the buffer.
3. ⟦`infinite_run ?E; ?i ≤ ?j; (?pp, ?qq, ?mm) ∈# cfg_inflight (?E ?i); ¬ (?pp, ?qq, ?mm) ∈# cfg_inflight (?E ?j)`⟧ ⟹ `∃k n. ?i ≤ k ∧ k < ?j ∧ Receive ?qq n ?pp ?mm ∈ events_of (cfg_hist (?E (Suc k)))` then yields an intermediate step at which a matching receive was added.

```
theorem fair_run_delivers:
  assumes inf:  "infinite_run E"
      and fair: "fair_run E"
      and pc:   "p ∈ correct"
      and qc:   "q ∈ correct"
      and send: "Send p n q m ∈ events_of (cfg_hist (E i))"
  shows "∃j n'. Receive q n' p m ∈ events_of (cfg_hist (E j))"
proof -
  have inv: "sends_match_inv (E i)"
    by (rule infinite_run_sends_match_inv[OF inf])
  hence alt:
    "(∃n'. Receive q n' p m ∈ events_of (cfg_hist (E i)))
       ∨ (p, q, m) ∈# cfg_inflight (E i)"
```

```
    using pc qc send by (auto simp: sends_match_inv_def)
  show ?thesis
  proof (cases "∃n'. Receive q n' p m ∈ events_of (cfg_hist (E i))")
    case True
    thus ?thesis by blast
  next
    case no_rec_at_i: False
    with alt have in_buf: "(p, q, m) ∈# cfg_inflight (E i)" by blast
    from fair[unfolded fair_run_def] in_buf
    obtain j' where j'_ge: "i ≤ j'"
                and j'_out: "¬ (p, q, m) ∈# cfg_inflight (E j')"
      by blast
    from triple_disappears_implies_recv[OF inf j'_ge in_buf j'_out]
    obtain k n' where
      i_k: "i ≤ k" and k_j': "k < j'"
      and rec: "Receive q n' p m ∈ events_of (cfg_hist (E (Suc k)))"
      by blast
    thus ?thesis by blast
  qed
qed
```

A useful packaging of the liveness theorem at the `messages_delivered_among` level, ranging over a fixed "sufficiently late" index. Saying "every send is eventually delivered" is the same as "the union of histories $\bigcup$`_i events_of (cfg_hist (E i))` satisfies `messages_delivered_among correct`".

```
definition history_union :: "(nat ⇒ 'p config) ⇒ 'p history" where
  "history_union E = (λp. SOME es.
                           (∃i. es = cfg_hist (E i) p
                                 ∧ (∀j ≥ i. cfg_hist (E j) p = es)))"
```

The `history_union` definition above is a stub for a fuller treatment. The temporally-informed statement that is actually useful in downstream developments is the form below: it says that the messages-delivered property is achieved cumulatively over the infinite run, not at any single index.

```
theorem fair_run_delivers_all:
  assumes inf:  "infinite_run E"
      and fair: "fair_run E"
  shows "∀p n q m i.
           p ∈ correct ∧ q ∈ correct
           ∧ Send p n q m ∈ events_of (cfg_hist (E i))
           ⟶ (∃j n'. Receive q n' p m ∈ events_of (cfg_hist (E j)))"
  using fair_run_delivers[OF inf fair] by blast

end — context byzantineSystem

end
```

```
theory Primitives
  imports Liveness
begin
```

context byzantineSystem
begin

58 BRU: Byzantine Reliable Unicast

The *operational content* of Byzantine Reliable Unicast collapses, at our event-based abstraction, to the structural condition that every correct-to-correct send has a matching receive. BRU's interesting operational properties – a Byzantine intermediary in the network path cannot drop, reorder, or fabricate messages between two correct endpoints – are below the abstraction layer of this development; their downstream effect on the global history is exactly the `messages_delivered_among` predicate.

We expose a named alias `bru_satisfied` below so downstream theorems can read "BRU-satisfied" rather than "messages-delivered" and so the operational role of the primitive is documented at the predicate name.

```
definition bru_satisfied :: "'p history ⇒ bool" where
  "bru_satisfied H ⟷ messages_delivered_among correct H"

lemma bru_satisfied_iff:
  "bru_satisfied H ⟷ messages_delivered_among correct H"
  by (simp add: bru_satisfied_def)
```

58.1 Operational discharge of BRU from the run model

A finite run of the inductive `run_step` relation that drains its in-flight buffer realises BRU. This is exactly `⟦run ?cfg; cfg_inflight ?cfg = empty_inflight⟧ ⟹ messages_delivered_among correct (cfg_hist ?cfg)`, repackaged under the BRU name. Operationally: a unicast network where every correct-to-correct in-flight message is eventually delivered produces histories that satisfy BRU.

```
theorem drained_run_satisfies_bru:
  assumes run_cfg: "run cfg"
      and drained: "cfg_inflight cfg = empty_inflight"
  shows "bru_satisfied (cfg_hist cfg)"
  unfolding bru_satisfied_def
  by (rule fairness_implies_delivery[OF run_cfg drained])
```

Fair infinite runs realise BRU *per event*: every correct-to-correct `Send` that appears in some `E i` eventually has a matching `Receive` in some `E j`. This is the infinite- execution analogue of `⟦run ?cfg; cfg_inflight ?cfg = empty_inflight⟧ ⟹ bru_satisfied (cfg_hist ?cfg)`, proved in `⟦infinite_run ?E; fair_run ?E; ?p ∈ correct; ?q ∈ correct; Send ?p ?n ?q ?m ∈ events_of (cfg_hist (?E ?i))⟧ ⟹ ∃j n'. Receive ?q n' ?p ?m ∈ events_of (cfg_hist (?E j))`.

```
theorem fair_run_satisfies_bru_pointwise:
  assumes inf:  "infinite_run E"
      and fair: "fair_run E"
      and pc:   "p ∈ correct"
      and qc:   "q ∈ correct"
      and send: "Send p n q m ∈ events_of (cfg_hist (E i))"
  shows "∃j nr. Receive q nr p m ∈ events_of (cfg_hist (E j))"
  by (rule fair_run_delivers[OF inf fair pc qc send])
```

59 BCB-over-BRB: BRU plus causal-order delivery

Byzantine Causal Broadcast over Byzantine Reliable Broadcast is a strictly stronger primitive than BRU. On top of BRU's correct-to-correct reliability (the BRB part), BCB guarantees that the delivery order at every correct receiver respects the causal order of the sends. Concretely, if two send events at correct processes (possibly different senders) are causally ordered in the global history, then every correct receiver sees the matching receives in the same causal order.

We capture the CB part as `bcb_causal_order` below. Together with `bru_satisfied` it gives BCB-over-BRB: `bcb_over_brb_satisfied`.

```
definition bcb_causal_order :: "'p set ⇒ 'p history ⇒ bool" where
  "bcb_causal_order C H ⟷
     (∀p1 n1 q m1 p2 n2 m2 nr1 nr2.
        p1 ∈ C ∧ p2 ∈ C ∧ q ∈ C ∧
        Send p1 n1 q m1 ∈ events_of H ∧
        Send p2 n2 q m2 ∈ events_of H ∧
        Receive q nr1 p1 m1 ∈ events_of H ∧
        Receive q nr2 p2 m2 ∈ events_of H ∧
        hb H (Send p1 n1 q m1) (Send p2 n2 q m2)
        ⟶ hb H (Receive q nr1 p1 m1) (Receive q nr2 p2 m2))"

definition bcb_over_brb_satisfied :: "'p set ⇒ 'p history ⇒ bool" where
  "bcb_over_brb_satisfied C H ⟷
     bru_satisfied H ∧ bcb_causal_order C H"

lemma bcb_over_brb_implies_bru:
  assumes "bcb_over_brb_satisfied C H"
  shows   "bru_satisfied H"
  using assms by (simp add: bcb_over_brb_satisfied_def)

lemma bcb_over_brb_implies_causal_order:
  assumes "bcb_over_brb_satisfied C H"
  shows   "bcb_causal_order C H"
  using assms by (simp add: bcb_over_brb_satisfied_def)
```

60 Operational T6 and T7 named explicitly via the primitives

Operational Theorem 6 (unicast via BRU): under `mode_admissible` unicast on the adversary's execution, the naive algorithm with the `recv_from_history` view solves `CD_B`.

This is ⟦`adversary_admissible correct ?adv; mode_admissible Unicast (adv_E ?adv)`⟧ ⟹ `valid_B correct (adv_E ?adv) (fst (naive_cd_B_alg (recv_from_history (adv_i ?adv) (adv_E ?adv)) (adv_i ?adv) (adv_e_star ?adv))) (adv_e_star ?adv)`, repackaged under the BRU name to make the operational role of the primitive explicit.

```
theorem T6_unicast_via_bru:
  assumes adm:     "adversary_admissible correct adv"
      and mode_ok: "mode_admissible Unicast (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                 (fst (naive_cd_B_alg
```

```
                    (recv_from_history (adv_i adv) (adv_E adv))
                    (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
  by (rule CD_B_solvable_unicast_operational[OF adm mode_ok])
```

Connection from BRU to T6. Any history that satisfies BRU (plus the well-formedness baked into `mode_admissible`) is unicast-mode-admissible by construction, so any algorithm whose view is built from the history's correct components solves `CD_B`.

```
lemma bru_realises_mode_admissible_unicast:
  assumes wf:  "wf_history H"
      and bru: "bru_satisfied H"
  shows "mode_admissible Unicast H"
  using wf bru by (simp add: mode_admissible_def bru_satisfied_def)

theorem bru_solves_CD_B_unicast:
  assumes adm:  "adversary_admissible correct adv"
      and wf:   "wf_history (adv_E adv)"
      and bru:  "bru_satisfied (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                    (recv_from_history (adv_i adv) (adv_E adv))
                    (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
proof -
  have mode_ok: "mode_admissible Unicast (adv_E adv)"
    by (rule bru_realises_mode_admissible_unicast[OF wf bru])
  show ?thesis by (rule T6_unicast_via_bru[OF adm mode_ok])
qed
```

Operational Theorem 7 (broadcast via BCB-over-BRB): under `mode_admissible` broadcast on the adversary's execution, the naive algorithm with the `recv_from_history` view solves `CD_B`.

This is ⟦`adversary_admissible correct ?adv; mode_admissible Broadcast (adv_E ?adv)`⟧ ⟹ `valid_B correct (adv_E ?adv) (fst (naive_cd_B_alg (recv_from_history (adv_i ?adv) (adv_E ?adv)) (adv_i ?adv) (adv_e_star ?adv))) (adv_e_star ?adv)`, repackaged under the BCB-over-BRB name.

```
theorem T7_broadcast_via_bcb_over_brb:
  assumes adm:     "adversary_admissible correct adv"
      and mode_ok: "mode_admissible Broadcast (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                    (recv_from_history (adv_i adv) (adv_E adv))
                    (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
  by (rule CD_B_solvable_broadcast_operational[OF adm mode_ok])
```

Connection from BCB-over-BRB to T7. A history satisfying BCB-over-BRB also satisfies BRU (by `bcb_over_brb_satisfied ?C ?H` ⟹ `bru_satisfied ?H`) and hence – together with well-formedness – is broadcast-mode- admissible. The causal-order half of BCB-over-BRB is not directly needed at this abstraction (it would matter if our recv view were derived from the receive sequence at each `q` rather than from the global history

H directly). It is nonetheless the operational property that, in a richer scheduler model, would let a correct receiver assemble the senders' histories in the order they actually happened.

```
lemma bcb_over_brb_realises_mode_admissible_broadcast:
  assumes wf:    "wf_history H"
      and bcb:   "bcb_over_brb_satisfied correct H"
  shows "mode_admissible Broadcast H"
proof -
  have bru: "bru_satisfied H" by (rule bcb_over_brb_implies_bru[OF bcb])
  show ?thesis
    using wf bru by (simp add: mode_admissible_def bru_satisfied_def)
qed

theorem bcb_over_brb_solves_CD_B_broadcast:
  assumes adm:  "adversary_admissible correct adv"
      and wf:   "wf_history (adv_E adv)"
      and bcb:  "bcb_over_brb_satisfied correct (adv_E adv)"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i adv) (adv_E adv))
                       (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
proof -
  have mode_ok: "mode_admissible Broadcast (adv_E adv)"
    by (rule bcb_over_brb_realises_mode_admissible_broadcast[OF wf bcb])
  show ?thesis by (rule T7_broadcast_via_bcb_over_brb[OF adm mode_ok])
qed
```

61 End-to-end: from a fair run of the model to a CD_B-solved configuration

The full operational chain: a fair finite run (one that drains the in-flight buffer) plus a unicast or broadcast mode yields a configuration whose history is mode-admissible, and the naive algorithm with the run-derived recv view solves CD_B at every admissible adversary whose execution matches the run.

```
theorem fair_drained_run_solves_CD_B_unicast:
  assumes run_cfg: "run cfg"
      and drained: "cfg_inflight cfg = empty_inflight"
      and adm:     "adversary_admissible correct adv"
      and adv_eq:  "adv_E adv = cfg_hist cfg"
  shows "valid_B correct (adv_E adv)
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i adv) (adv_E adv))
                       (adv_i adv) (adv_e_star adv)))
                (adv_e_star adv)"
proof -
  have wf:  "wf_history (cfg_hist cfg)"
    by (rule wf_history_run[OF run_cfg])
  have bru: "bru_satisfied (cfg_hist cfg)"
    by (rule drained_run_satisfies_bru[OF run_cfg drained])
```

```
  have wf':  "wf_history (adv_E adv)"  using wf  adv_eq by simp
  have bru': "bru_satisfied (adv_E adv)" using bru adv_eq by simp
  show ?thesis by (rule bru_solves_CD_B_unicast[OF adm wf' bru'])
qed
```

The broadcast counterpart of ⟦run ?cfg; cfg_inflight ?cfg = empty_inflight; adversary_admissible correct ?adv; adv_E ?adv = cfg_hist ?cfg⟧ ⟹ valid_B correct (adv_E ?adv) (fst (naive_cd_B_alg (recv_from_history (adv_i ?adv) (adv_E ?adv)) (adv_i ?adv) (adv_e_star ?adv))) (adv_e_star ?adv). We need the additional BCB causal-order property to assert bcb_over_brb_satisfied; under our event model the underlying scheduler does not enforce it operationally, so we leave it as a hypothesis. In a richer scheduler model satisfying BCB operationally, that hypothesis is discharged.

```
theorem fair_drained_run_solves_CD_B_broadcast:
  assumes run_cfg: "run cfg"
      and drained: "cfg_inflight cfg = empty_inflight"
      and bcb_ord: "bcb_causal_order correct (cfg_hist cfg)"
      and adm:     "adversary_admissible correct adv"
      and adv_eq:  "adv_E adv = cfg_hist cfg"
  shows "valid_B correct (adv_E adv)
                 (fst (naive_cd_B_alg
                        (recv_from_history (adv_i adv) (adv_E adv))
                        (adv_i adv) (adv_e_star adv)))
                 (adv_e_star adv)"
proof -
  have wf:  "wf_history (cfg_hist cfg)"
    by (rule wf_history_run[OF run_cfg])
  have bru: "bru_satisfied (cfg_hist cfg)"
    by (rule drained_run_satisfies_bru[OF run_cfg drained])
  have bcb: "bcb_over_brb_satisfied correct (cfg_hist cfg)"
    using bru bcb_ord by (simp add: bcb_over_brb_satisfied_def)
  have wf':  "wf_history (adv_E adv)"   using wf  adv_eq by simp
  have bcb': "bcb_over_brb_satisfied correct (adv_E adv)"
    using bcb adv_eq by simp
  show ?thesis by (rule bcb_over_brb_solves_CD_B_broadcast[OF adm wf' bcb'])
qed

end — context byzantineSystem

end


theory T6_Concrete
  imports Primitives
begin

context byzantineSystem
begin
```

62 Demo scenario: two correct processes, one send and receive

The target history of the demonstration. Two distinct correct processes `p_a` and `p_b`: `p_b` sends one message `m = 0` to `p_a`; `p_a` receives it; `p_a` then performs an internal event. Other processes have empty histories.

```
definition demo_H :: "'p ⇒ 'p ⇒ 'p history" where
  "demo_H p_a p_b =
     (λp. if p = p_a then [Receive p_a 1 p_b 0, Internal p_a 2]
          else if p = p_b then [Send p_b 1 p_a 0]
          else [])"

definition demo_adv :: "'p ⇒ 'p ⇒ 'p adversary" where
  "demo_adv p_a p_b =
     (| adv_E = demo_H p_a p_b,
       adv_e_star = Internal p_a 2,
       adv_i = p_a |)"
```

The events that appear in the demo history.

```
lemma demo_H_at_p_a:
  assumes "p_a ≠ p_b"
  shows "demo_H p_a p_b p_a = [Receive p_a 1 p_b 0, Internal p_a 2]"
  using assms by (simp add: demo_H_def)

lemma demo_H_at_p_b:
  assumes "p_a ≠ p_b"
  shows "demo_H p_a p_b p_b = [Send p_b 1 p_a 0]"
  using assms by (simp add: demo_H_def)

lemma demo_H_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b"
  shows "demo_H p_a p_b p = []"
  using assms by (simp add: demo_H_def)

lemma demo_H_events:
  assumes "p_a ≠ p_b"
  shows "events_of (demo_H p_a p_b) =
           {Receive p_a 1 p_b 0, Internal p_a 2, Send p_b 1 p_a 0}"
proof -
  have "events_of (demo_H p_a p_b) = (⋃p. set (demo_H p_a p_b p))"
    by (simp add: events_of_def)
  also have "... = set (demo_H p_a p_b p_a) ∪ set (demo_H p_a p_b p_b)
                   ∪ (⋃p ∈ -{p_a, p_b}. set (demo_H p_a p_b p))"
    by auto
  also have "set (demo_H p_a p_b p_a) = {Receive p_a 1 p_b 0, Internal p_a 2}"
    using assms by (simp add: demo_H_at_p_a)
  moreover have "set (demo_H p_a p_b p_b) = {Send p_b 1 p_a 0}"
    using assms by (simp add: demo_H_at_p_b)
  moreover have "(⋃p ∈ -{p_a, p_b}. set (demo_H p_a p_b p)) = {}"
    by (auto simp: demo_H_elsewhere)
  ultimately show ?thesis by auto
```

qed

63 The three configurations along the demo run

Three intermediate configurations, named after the step that produced them. Starting from `init_config`:

- `cfg0 = init_config`
- `cfg1 = after step_send p_b 1 p_a 0`
- `cfg2 = after step_recv p_a 1 p_b 0`
- `cfg3 = after step_internal p_a 2` (this is the drained final configuration)

```
definition demo_cfg1 :: "'p ⇒ 'p ⇒ 'p config" where
  "demo_cfg1 p_a p_b =
     (| cfg_hist = (λp. if p = p_b then [Send p_b 1 p_a 0] else []),
       cfg_inflight = empty_inflight ∪# {# (p_b, p_a, 0) } |)"

definition demo_cfg2 :: "'p ⇒ 'p ⇒ 'p config" where
  "demo_cfg2 p_a p_b =
     (| cfg_hist = (λp. if p = p_a then [Receive p_a 1 p_b 0]
                        else if p = p_b then [Send p_b 1 p_a 0]
                        else []),
       cfg_inflight = empty_inflight |)"

definition demo_cfg3 :: "'p ⇒ 'p ⇒ 'p config" where
  "demo_cfg3 p_a p_b =
     (| cfg_hist = demo_H p_a p_b,
       cfg_inflight = empty_inflight |)"
```

64 The three run_step transitions

Step 1: `p_b` sends to `p_a`, getting the buffer entry `(p_b, p_a, 0)`.

```
lemma demo_step_send:
  assumes pa: "p_a ∈ correct"
      and pb: "p_b ∈ correct"
      and dist: "p_a ≠ p_b"
  shows "run_step init_config (demo_cfg1 p_a p_b)"
proof -
  let ?cfg0 = "init_config"
  have hist_pb_init: "cfg_hist ?cfg0 p_b = []"
    by (simp add: init_config_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg0 p_b))"
    using hist_pb_init by simp
  have hist_eq: "cfg_hist (demo_cfg1 p_a p_b)
                 = (cfg_hist ?cfg0)
                     (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 0])"
    by (rule ext) (simp add: demo_cfg1_def init_config_def)
```

```
  have inflight_eq: "cfg_inflight (demo_cfg1 p_a p_b)
                       = cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 0) }"
    by (simp add: demo_cfg1_def init_config_def)
  have cfg_eq: "demo_cfg1 p_a p_b
                  = ?cfg0 (| cfg_hist :=
                               (cfg_hist ?cfg0)
                                 (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 0]),
                             cfg_inflight :=
                               cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 0) } |)"
    using hist_eq inflight_eq by (simp add: demo_cfg1_def init_config_def)
  show ?thesis
    by (rule run_step.step_send[OF pb pa n_eq cfg_eq])
qed
```

Step 2: p_a receives the buffered message from p_b, producing the matching receive event.

```
lemma demo_step_recv:
  assumes pa: "p_a ∈ correct"
      and pb: "p_b ∈ correct"
      and dist: "p_a ≠ p_b"
  shows "run_step (demo_cfg1 p_a p_b) (demo_cfg2 p_a p_b)"
proof -
  let ?cfg1 = "demo_cfg1 p_a p_b"
  have buf_in: "(p_b, p_a, 0) ∈# cfg_inflight ?cfg1"
    by (simp add: demo_cfg1_def empty_inflight_def)
  have hist_pa_eq: "cfg_hist ?cfg1 p_a = []"
    using dist by (simp add: demo_cfg1_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg1 p_a))"
    using hist_pa_eq by simp
  have hist_eq: "cfg_hist (demo_cfg2 p_a p_b)
                  = (cfg_hist ?cfg1)
                      (p_a := cfg_hist ?cfg1 p_a @ [Receive p_a 1 p_b 0])"
    using dist hist_pa_eq
    by (rule_tac ext) (simp add: demo_cfg1_def demo_cfg2_def)
  have inflight_eq: "cfg_inflight (demo_cfg2 p_a p_b)
                       = cfg_inflight ?cfg1 -# (p_b, p_a, 0)"
    by (rule ext) (simp add: demo_cfg1_def demo_cfg2_def empty_inflight_def)
  have cfg_eq: "demo_cfg2 p_a p_b
                  = ?cfg1 (| cfg_hist :=
                               (cfg_hist ?cfg1)
                                 (p_a := cfg_hist ?cfg1 p_a @ [Receive p_a 1 p_b 0]),
                             cfg_inflight :=
                               cfg_inflight ?cfg1 -# (p_b, p_a, 0) |)"
    using hist_eq inflight_eq by (simp add: demo_cfg2_def)
  show ?thesis
    by (rule run_step.step_recv[OF pa pb buf_in n_eq cfg_eq])
qed
```

Step 3: p_a performs an internal event, completing the local history at the target event.

```
lemma demo_step_internal:
  assumes pa: "p_a ∈ correct"
```

```
    and pb: "p_b ∈ correct"
    and dist: "p_a ≠ p_b"
  shows "run_step (demo_cfg2 p_a p_b) (demo_cfg3 p_a p_b)"
proof -
  let ?cfg2 = "demo_cfg2 p_a p_b"
  have hist_pa_eq: "cfg_hist ?cfg2 p_a = [Receive p_a 1 p_b 0]"
    using dist by (simp add: demo_cfg2_def)
  have n_eq: "(2 :: nat) = Suc (length (cfg_hist ?cfg2 p_a))"
    using hist_pa_eq by simp
  have hist_eq: "cfg_hist (demo_cfg3 p_a p_b)
                 = (cfg_hist ?cfg2)
                     (p_a := cfg_hist ?cfg2 p_a @ [Internal p_a 2])"
    using dist hist_pa_eq
    by (rule_tac ext) (simp add: demo_cfg2_def demo_cfg3_def demo_H_def)
  have inflight_eq: "cfg_inflight (demo_cfg3 p_a p_b)
                      = cfg_inflight ?cfg2"
    by (simp add: demo_cfg2_def demo_cfg3_def)
  have cfg_eq: "demo_cfg3 p_a p_b
                = ?cfg2 (| cfg_hist :=
                             (cfg_hist ?cfg2)
                               (p_a := cfg_hist ?cfg2 p_a @ [Internal p_a 2]) |)"
    using hist_eq inflight_eq by (simp add: demo_cfg3_def)
  show ?thesis
    by (rule run_step.step_internal[OF pa n_eq cfg_eq])
qed
```

65 The demo run is fair, drained, and produces `demo_H`

Composing the three steps into a single `run`: from `init_config` via three `run_step` transitions to `demo_cfg3`.

```
lemma demo_run:
  assumes pa: "p_a ∈ correct"
      and pb: "p_b ∈ correct"
      and dist: "p_a ≠ p_b"
  shows "run (demo_cfg3 p_a p_b)"
proof -
  have base: "run init_config" by simp
  have s1: "run_step init_config (demo_cfg1 p_a p_b)"
    by (rule demo_step_send[OF pa pb dist])
  have r1: "run (demo_cfg1 p_a p_b)"
    by (rule run_extend[OF base s1])
  have s2: "run_step (demo_cfg1 p_a p_b) (demo_cfg2 p_a p_b)"
    by (rule demo_step_recv[OF pa pb dist])
  have r2: "run (demo_cfg2 p_a p_b)"
    by (rule run_extend[OF r1 s2])
  have s3: "run_step (demo_cfg2 p_a p_b) (demo_cfg3 p_a p_b)"
    by (rule demo_step_internal[OF pa pb dist])
  show ?thesis by (rule run_extend[OF r2 s3])
qed

lemma demo_drained:
```

```
  shows "cfg_inflight (demo_cfg3 p_a p_b) = empty_inflight"
  by (simp add: demo_cfg3_def)

lemma demo_cfg_hist_eq:
  shows "cfg_hist (demo_cfg3 p_a p_b) = demo_H p_a p_b"
  by (simp add: demo_cfg3_def)
```

66 The naive algorithm solves CD_B at the demo adversary

Adversary admissibility on the demo adversary: the target event `Internal p_a 2` is at `p_a` (which is correct by assumption) and lies in `events_of (demo_H p_a p_b)`.

```
lemma demo_wf_history_local_p_a:
  assumes "p_a ≠ p_b"
  shows "wf_history_local p_a (demo_H p_a p_b p_a)"
proof -
  let ?L = "[Receive p_a 1 p_b 0, Internal p_a 2]"
  have list_eq: "demo_H p_a p_b p_a = ?L"
    using assms by (rule demo_H_at_p_a)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_a" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis
    using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma demo_wf_history_local_p_b:
  assumes "p_a ≠ p_b"
  shows "wf_history_local p_b (demo_H p_a p_b p_b)"
proof -
  let ?L = "[Send p_b 1 p_a 0]"
  have list_eq: "demo_H p_a p_b p_b = ?L"
    using assms by (rule demo_H_at_p_b)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_b" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0" by auto
    thus "seq_of (?L ! k) = Suc k" by simp
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma demo_wf_history_local_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b"
  shows "wf_history_local p (demo_H p_a p_b p)"
```

```
  using assms by (simp add: demo_H_elsewhere wf_history_local_def)

lemma demo_wf_history:
  assumes "p_a ≠ p_b"
  shows "wf_history (demo_H p_a p_b)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (demo_H p_a p_b p)"
  proof (cases "p = p_a")
    case True
    thus ?thesis using assms demo_wf_history_local_p_a by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_b")
      case True
      thus ?thesis using assms demo_wf_history_local_p_b by simp
    next
      case False
      with ‹p ≠ p_a› show ?thesis
        by (rule demo_wf_history_local_elsewhere)
    qed
  qed
qed

lemma demo_adv_admissible:
  assumes pa: "p_a ∈ correct"
      and pb: "p_b ∈ correct"
      and dist: "p_a ≠ p_b"
  shows "adversary_admissible correct (demo_adv p_a p_b)"
proof -
  have wf: "wf_history (demo_H p_a p_b)"
    using dist by (rule demo_wf_history)
  have i_corr: "adv_i (demo_adv p_a p_b) ∈ correct"
    using pa by (simp add: demo_adv_def)
  have proc_eq:
    "proc_of (adv_e_star (demo_adv p_a p_b)) = adv_i (demo_adv p_a p_b)"
    by (simp add: demo_adv_def)
  have target_in:
    "adv_e_star (demo_adv p_a p_b) ∈ events_of (adv_E (demo_adv p_a p_b))"
    using dist by (simp add: demo_adv_def demo_H_events)
  have hist_eq: "adv_E (demo_adv p_a p_b) = demo_H p_a p_b"
    by (simp add: demo_adv_def)
  show ?thesis
    using wf i_corr proc_eq target_in
    by (simp add: adversary_admissible_def demo_adv_def)
qed
```

The headline concrete-demo theorem. Composing the 3-step run with `⟦run ?cfg; cfg_inflight ?cfg = empty_inflight; adversary_admissible correct ?adv; adv_E ?adv = cfg_hist ?cfg⟧ ⟹ valid_B correct (adv_E ?adv) (fst (naive_cd_B_alg (recv_from_history (adv_i ?adv) (adv_E ?adv)) (adv_i ?adv) (adv_e_star ?adv))) (adv_e_star ?adv)` gives

that the naive algorithm at the demo configuration's `recv_from_history` view solves `CD_B` at the demo adversary's target event.

Spelled out: under the demo adversary, the naive algorithm

1. takes its received view to be exactly the global history (`recv = adv_E adv`),
2. outputs `F = adv_E adv`,
3. satisfies `valid_B(adv_E adv, F, Internal p_a 2)` trivially because `F = adv_E adv`.

The composition rests on:

4. ⟦`?p_a ∈ correct; ?p_b ∈ correct; ?p_a ≠ ?p_b`⟧ ⟹ `run (demo_cfg3 ?p_a ?p_b): run (demo_cfg3 p_a p_b)`.
5. `cfg_inflight (demo_cfg3 ?p_a ?p_b) = empty_inflight`: the in-flight buffer is empty at `demo_cfg3`.
6. ⟦`?p_a ∈ correct; ?p_b ∈ correct; ?p_a ≠ ?p_b`⟧ ⟹ `adversary_admissible correct (demo_adv ?p_a ?p_b)`: the demo adversary is admissible for the correct set.
7. `cfg_hist (demo_cfg3 ?p_a ?p_b) = demo_H ?p_a ?p_b`: `cfg_hist (demo_cfg3 p_a p_b)` equals `demo_H p_a p_b`, which is `adv_E (demo_adv p_a p_b)`.

```
theorem T6_concrete_demo:
  assumes pa: "p_a ∈ correct"
      and pb: "p_b ∈ correct"
      and dist: "p_a ≠ p_b"
  shows "valid_B correct (adv_E (demo_adv p_a p_b))
                 (fst (naive_cd_B_alg
                        (recv_from_history (adv_i (demo_adv p_a p_b))
                                           (adv_E (demo_adv p_a p_b)))
                        (adv_i (demo_adv p_a p_b))
                        (adv_e_star (demo_adv p_a p_b))))
                 (adv_e_star (demo_adv p_a p_b))"
proof -
  let ?cfg = "demo_cfg3 p_a p_b"
  let ?adv = "demo_adv p_a p_b"
  have run_cfg:    "run ?cfg"
    by (rule demo_run[OF pa pb dist])
  have drained:    "cfg_inflight ?cfg = empty_inflight"
    by (rule demo_drained)
  have adm:        "adversary_admissible correct ?adv"
    by (rule demo_adv_admissible[OF pa pb dist])
  have adv_E_eq:   "adv_E ?adv = cfg_hist ?cfg"
    using demo_cfg_hist_eq[of p_a p_b] by (simp add: demo_adv_def)
  show ?thesis
    by (rule fair_drained_run_solves_CD_B_unicast[OF run_cfg drained adm adv_E_eq])
qed
```

67 Existence of a non-trivial T6 witness

From the concrete-demo theorem we extract the headline existence statement: under the standard locale assumption that two distinct correct processes exist, the operational CD_B unicast theorem is non-vacuously satisfiable – there is an admissible adversary, a fair drained run, and an algorithm output witnessing T6's content.

```
theorem T6_witnessed:
  assumes two_correct: "∃p_a p_b. p_a ∈ correct ∧ p_b ∈ correct ∧ p_a ≠ p_b"
  shows "∃adv cfg.
           run cfg
         ∧ cfg_inflight cfg = empty_inflight
         ∧ adversary_admissible correct adv
         ∧ adv_E adv = cfg_hist cfg
         ∧ valid_B correct (adv_E adv)
                  (fst (naive_cd_B_alg
                         (recv_from_history (adv_i adv) (adv_E adv))
                         (adv_i adv) (adv_e_star adv)))
                  (adv_e_star adv)"
proof -
  obtain p_a p_b where pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
                       and dist: "p_a ≠ p_b"
    using two_correct by blast
  let ?cfg = "demo_cfg3 p_a p_b"
  let ?adv = "demo_adv p_a p_b"
  have run_cfg: "run ?cfg" by (rule demo_run[OF pa pb dist])
  have drained: "cfg_inflight ?cfg = empty_inflight" by (rule demo_drained)
  have adm: "adversary_admissible correct ?adv"
    by (rule demo_adv_admissible[OF pa pb dist])
  have adv_E_eq: "adv_E ?adv = cfg_hist ?cfg"
    using demo_cfg_hist_eq[of p_a p_b] by (simp add: demo_adv_def)
  have solves: "valid_B correct (adv_E ?adv)
                       (fst (naive_cd_B_alg
                              (recv_from_history (adv_i ?adv) (adv_E ?adv))
                              (adv_i ?adv) (adv_e_star ?adv)))
                       (adv_e_star ?adv)"
    by (rule T6_concrete_demo[OF pa pb dist])
  from run_cfg drained adm adv_E_eq solves show ?thesis by blast
qed

end — context byzantineSystem

end


theory T6_Multihop
  imports T6_Concrete
begin

context byzantineSystem
begin
```

68 Three-process multihop scenario

The target history of the multihop demonstration. Three distinct correct processes `p_a`, `p_b`, `p_c`:

- `p_b` sends $\mathtt{m}_1$ `= 1` to `p_a`;
- `p_a` receives $\mathtt{m}_1$;
- `p_a` sends $\mathtt{m}_2$ `= 2` to `p_c`;
- `p_c` receives $\mathtt{m}_2$;
- `p_c` performs an internal event (the adversary's target).

```
definition multi_H :: "'p ⇒ 'p ⇒ 'p ⇒ 'p history" where
  "multi_H p_a p_b p_c =
     (λp. if p = p_a then [Receive p_a 1 p_b 1, Send p_a 2 p_c 2]
          else if p = p_b then [Send p_b 1 p_a 1]
          else if p = p_c then [Receive p_c 1 p_a 2, Internal p_c 2]
          else [])"

definition multi_adv :: "'p ⇒ 'p ⇒ 'p ⇒ 'p adversary" where
  "multi_adv p_a p_b p_c =
     (| adv_E = multi_H p_a p_b p_c,
       adv_e_star = Internal p_c 2,
       adv_i = p_c |)"

lemma multi_H_at_p_a:
  assumes "p_a ≠ p_b" "p_a ≠ p_c"
  shows "multi_H p_a p_b p_c p_a = [Receive p_a 1 p_b 1, Send p_a 2 p_c 2]"
  using assms by (simp add: multi_H_def)

lemma multi_H_at_p_b:
  assumes "p_a ≠ p_b" "p_b ≠ p_c"
  shows "multi_H p_a p_b p_c p_b = [Send p_b 1 p_a 1]"
  using assms by (simp add: multi_H_def)

lemma multi_H_at_p_c:
  assumes "p_a ≠ p_c" "p_b ≠ p_c"
  shows "multi_H p_a p_b p_c p_c = [Receive p_c 1 p_a 2, Internal p_c 2]"
  using assms by (simp add: multi_H_def)

lemma multi_H_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b" "p ≠ p_c"
  shows "multi_H p_a p_b p_c p = []"
  using assms by (simp add: multi_H_def)

lemma multi_H_events:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "events_of (multi_H p_a p_b p_c) =
           {Receive p_a 1 p_b 1, Send p_a 2 p_c 2, Send p_b 1 p_a 1,
```

```
          Receive p_c 1 p_a 2, Internal p_c 2}"
proof -
  have "events_of (multi_H p_a p_b p_c) = (⋃p. set (multi_H p_a p_b p_c p))"
    by (simp add: events_of_def)
  also have "... = set (multi_H p_a p_b p_c p_a)
                 ∪ set (multi_H p_a p_b p_c p_b)
                 ∪ set (multi_H p_a p_b p_c p_c)
                 ∪ (⋃p ∈ -{p_a, p_b, p_c}. set (multi_H p_a p_b p_c p))"
    by auto
  also have "set (multi_H p_a p_b p_c p_a)
              = {Receive p_a 1 p_b 1, Send p_a 2 p_c 2}"
    using ab ac by (simp add: multi_H_at_p_a)
  moreover have "set (multi_H p_a p_b p_c p_b) = {Send p_b 1 p_a 1}"
    using ab bc by (simp add: multi_H_at_p_b)
  moreover have "set (multi_H p_a p_b p_c p_c)
                 = {Receive p_c 1 p_a 2, Internal p_c 2}"
    using ac bc by (simp add: multi_H_at_p_c)
  moreover have "(⋃p ∈ -{p_a, p_b, p_c}. set (multi_H p_a p_b p_c p)) = {}"
    by (auto simp: multi_H_elsewhere)
  ultimately show ?thesis by auto
qed
```

69 Five intermediate configurations along the multihop run

From `init_config` we trace five `run_step` transitions:

- `multi_cfg1`: after `step_send p_b 1 p_a 1`
- `multi_cfg2`: after `step_recv p_a 1 p_b 1`
- `multi_cfg3`: after `step_send p_a 2 p_c 2`
- `multi_cfg4`: after `step_recv p_c 1 p_a 2`
- `multi_cfg5`: after `step_internal p_c 2` (drained final)

```
definition multi_cfg1 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "multi_cfg1 p_a p_b p_c =
     (| cfg_hist = (λp. if p = p_b then [Send p_b 1 p_a 1] else []),
       cfg_inflight = empty_inflight ∪# {# (p_b, p_a, 1) } |)"

definition multi_cfg2 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "multi_cfg2 p_a p_b p_c =
     (| cfg_hist = (λp. if p = p_a then [Receive p_a 1 p_b 1]
                        else if p = p_b then [Send p_b 1 p_a 1]
                        else []),
       cfg_inflight = empty_inflight |)"

definition multi_cfg3 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "multi_cfg3 p_a p_b p_c =
```

```
    (| cfg_hist = (λp. if p = p_a
                        then [Receive p_a 1 p_b 1, Send p_a 2 p_c 2]
                      else if p = p_b then [Send p_b 1 p_a 1]
                      else []),
       cfg_inflight = empty_inflight ∪# {# (p_a, p_c, 2) } |)"

definition multi_cfg4 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "multi_cfg4 p_a p_b p_c =
    (| cfg_hist = (λp. if p = p_a
                        then [Receive p_a 1 p_b 1, Send p_a 2 p_c 2]
                      else if p = p_b then [Send p_b 1 p_a 1]
                      else if p = p_c then [Receive p_c 1 p_a 2]
                      else []),
       cfg_inflight = empty_inflight |)"

definition multi_cfg5 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "multi_cfg5 p_a p_b p_c =
    (| cfg_hist = multi_H p_a p_b p_c,
       cfg_inflight = empty_inflight |)"
```

70 The five run_step transitions

```
lemma multi_step_1_send:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step init_config (multi_cfg1 p_a p_b p_c)"
proof -
  let ?cfg0 = "init_config"
  have hist_pb: "cfg_hist ?cfg0 p_b = []"
    by (simp add: init_config_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg0 p_b))"
    using hist_pb by simp
  have hist_eq: "cfg_hist (multi_cfg1 p_a p_b p_c)
                 = (cfg_hist ?cfg0)
                     (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 1])"
    by (rule ext) (simp add: multi_cfg1_def init_config_def)
  have inflight_eq: "cfg_inflight (multi_cfg1 p_a p_b p_c)
                       = cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 1) }"
    by (simp add: multi_cfg1_def init_config_def)
  have cfg_eq: "multi_cfg1 p_a p_b p_c
                 = ?cfg0 (| cfg_hist :=
                              (cfg_hist ?cfg0)
                                (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 1]),
                            cfg_inflight :=
                              cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 1) } |)"
    using hist_eq inflight_eq by (simp add: multi_cfg1_def init_config_def)
  show ?thesis
    by (rule run_step.step_send[OF pb pa n_eq cfg_eq])
qed

lemma multi_step_2_recv:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
```

```
    and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (multi_cfg1 p_a p_b p_c) (multi_cfg2 p_a p_b p_c)"
proof -
  let ?cfg1 = "multi_cfg1 p_a p_b p_c"
  have buf_in: "(p_b, p_a, 1) ∈# cfg_inflight ?cfg1"
    by (simp add: multi_cfg1_def empty_inflight_def)
  have hist_pa: "cfg_hist ?cfg1 p_a = []"
    using ab by (simp add: multi_cfg1_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg1 p_a))"
    using hist_pa by simp
  have hist_eq: "cfg_hist (multi_cfg2 p_a p_b p_c)
                 = (cfg_hist ?cfg1)
                     (p_a := cfg_hist ?cfg1 p_a @ [Receive p_a 1 p_b 1])"
    using ab hist_pa
    by (rule_tac ext) (simp add: multi_cfg1_def multi_cfg2_def)
  have inflight_eq: "cfg_inflight (multi_cfg2 p_a p_b p_c)
                      = cfg_inflight ?cfg1 -# (p_b, p_a, 1)"
    by (rule ext) (simp add: multi_cfg1_def multi_cfg2_def empty_inflight_def)
  have cfg_eq: "multi_cfg2 p_a p_b p_c
                = ?cfg1 (| cfg_hist :=
                             (cfg_hist ?cfg1)
                               (p_a := cfg_hist ?cfg1 p_a @ [Receive p_a 1 p_b 1]),
                           cfg_inflight :=
                             cfg_inflight ?cfg1 -# (p_b, p_a, 1) |)"
    using hist_eq inflight_eq by (simp add: multi_cfg2_def)
  show ?thesis
    by (rule run_step.step_recv[OF pa pb buf_in n_eq cfg_eq])
qed

lemma multi_step_3_send:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (multi_cfg2 p_a p_b p_c) (multi_cfg3 p_a p_b p_c)"
proof -
  let ?cfg2 = "multi_cfg2 p_a p_b p_c"
  have hist_pa: "cfg_hist ?cfg2 p_a = [Receive p_a 1 p_b 1]"
    using ab by (simp add: multi_cfg2_def)
  have n_eq: "(2 :: nat) = Suc (length (cfg_hist ?cfg2 p_a))"
    using hist_pa by simp
  have hist_eq: "cfg_hist (multi_cfg3 p_a p_b p_c)
                 = (cfg_hist ?cfg2)
                     (p_a := cfg_hist ?cfg2 p_a @ [Send p_a 2 p_c 2])"
    using ab ac hist_pa
    by (rule_tac ext) (simp add: multi_cfg2_def multi_cfg3_def)
  have inflight_eq: "cfg_inflight (multi_cfg3 p_a p_b p_c)
                      = cfg_inflight ?cfg2 ∪# {# (p_a, p_c, 2) }"
    by (rule ext) (simp add: multi_cfg2_def multi_cfg3_def empty_inflight_def)
  have cfg_eq: "multi_cfg3 p_a p_b p_c
                = ?cfg2 (| cfg_hist :=
                             (cfg_hist ?cfg2)
                               (p_a := cfg_hist ?cfg2 p_a @ [Send p_a 2 p_c 2]),
                           cfg_inflight :=
```

```
                              cfg_inflight ?cfg2 ∪# {# (p_a, p_c, 2) } ⦈"
    using hist_eq inflight_eq by (simp add: multi_cfg3_def)
  show ?thesis
    by (rule run_step.step_send[OF pa pc n_eq cfg_eq])
qed

lemma multi_step_4_recv:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (multi_cfg3 p_a p_b p_c) (multi_cfg4 p_a p_b p_c)"
proof -
  let ?cfg3 = "multi_cfg3 p_a p_b p_c"
  have buf_in: "(p_a, p_c, 2) ∈# cfg_inflight ?cfg3"
    by (simp add: multi_cfg3_def empty_inflight_def)
  have hist_pc: "cfg_hist ?cfg3 p_c = []"
    using ac bc by (simp add: multi_cfg3_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg3 p_c))"
    using hist_pc by simp
  have hist_eq: "cfg_hist (multi_cfg4 p_a p_b p_c)
                 = (cfg_hist ?cfg3)
                     (p_c := cfg_hist ?cfg3 p_c @ [Receive p_c 1 p_a 2])"
    using ac bc hist_pc
    by (rule_tac ext) (simp add: multi_cfg3_def multi_cfg4_def)
  have inflight_eq: "cfg_inflight (multi_cfg4 p_a p_b p_c)
                      = cfg_inflight ?cfg3 -# (p_a, p_c, 2)"
    by (rule ext) (simp add: multi_cfg3_def multi_cfg4_def empty_inflight_def)
  have cfg_eq: "multi_cfg4 p_a p_b p_c
                 = ?cfg3 ⦇ cfg_hist :=
                             (cfg_hist ?cfg3)
                               (p_c := cfg_hist ?cfg3 p_c @ [Receive p_c 1 p_a 2]),
                           cfg_inflight :=
                             cfg_inflight ?cfg3 -# (p_a, p_c, 2) ⦈"
    using hist_eq inflight_eq by (simp add: multi_cfg4_def)
  show ?thesis
    by (rule run_step.step_recv[OF pc pa buf_in n_eq cfg_eq])
qed

lemma multi_step_5_internal:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (multi_cfg4 p_a p_b p_c) (multi_cfg5 p_a p_b p_c)"
proof -
  let ?cfg4 = "multi_cfg4 p_a p_b p_c"
  have hist_pc: "cfg_hist ?cfg4 p_c = [Receive p_c 1 p_a 2]"
    using ac bc by (simp add: multi_cfg4_def)
  have n_eq: "(2 :: nat) = Suc (length (cfg_hist ?cfg4 p_c))"
    using hist_pc by simp
  have hist_eq: "cfg_hist (multi_cfg5 p_a p_b p_c)
                 = (cfg_hist ?cfg4)
                     (p_c := cfg_hist ?cfg4 p_c @ [Internal p_c 2])"
    using ab ac bc hist_pc
    by (rule_tac ext) (simp add: multi_cfg4_def multi_cfg5_def multi_H_def)
```

```
  have inflight_eq: "cfg_inflight (multi_cfg5 p_a p_b p_c)
                       = cfg_inflight ?cfg4"
    by (simp add: multi_cfg4_def multi_cfg5_def)
  have cfg_eq: "multi_cfg5 p_a p_b p_c
                  = ?cfg4 (| cfg_hist :=
                               (cfg_hist ?cfg4)
                                 (p_c := cfg_hist ?cfg4 p_c @ [Internal p_c 2]) |)"
    using hist_eq inflight_eq by (simp add: multi_cfg5_def)
  show ?thesis
    by (rule run_step.step_internal[OF pc n_eq cfg_eq])
qed
```

71 The multihop run is fair, drained, and produces `multi_H`

```
lemma multi_run:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run (multi_cfg5 p_a p_b p_c)"
proof -
  have r0: "run init_config" by simp
  have s1: "run_step init_config (multi_cfg1 p_a p_b p_c)"
    by (rule multi_step_1_send[OF pa pb pc ab ac bc])
  have r1: "run (multi_cfg1 p_a p_b p_c)" by (rule run_extend[OF r0 s1])
  have s2: "run_step (multi_cfg1 p_a p_b p_c) (multi_cfg2 p_a p_b p_c)"
    by (rule multi_step_2_recv[OF pa pb pc ab ac bc])
  have r2: "run (multi_cfg2 p_a p_b p_c)" by (rule run_extend[OF r1 s2])
  have s3: "run_step (multi_cfg2 p_a p_b p_c) (multi_cfg3 p_a p_b p_c)"
    by (rule multi_step_3_send[OF pa pb pc ab ac bc])
  have r3: "run (multi_cfg3 p_a p_b p_c)" by (rule run_extend[OF r2 s3])
  have s4: "run_step (multi_cfg3 p_a p_b p_c) (multi_cfg4 p_a p_b p_c)"
    by (rule multi_step_4_recv[OF pa pb pc ab ac bc])
  have r4: "run (multi_cfg4 p_a p_b p_c)" by (rule run_extend[OF r3 s4])
  have s5: "run_step (multi_cfg4 p_a p_b p_c) (multi_cfg5 p_a p_b p_c)"
    by (rule multi_step_5_internal[OF pa pb pc ab ac bc])
  show ?thesis by (rule run_extend[OF r4 s5])
qed

lemma multi_drained:
  shows "cfg_inflight (multi_cfg5 p_a p_b p_c) = empty_inflight"
  by (simp add: multi_cfg5_def)

lemma multi_cfg_hist_eq:
  shows "cfg_hist (multi_cfg5 p_a p_b p_c) = multi_H p_a p_b p_c"
  by (simp add: multi_cfg5_def)
```

72 Well-formedness and adversary admissibility

```
lemma multi_wf_history_local_p_a:
  assumes "p_a ≠ p_b" "p_a ≠ p_c"
  shows "wf_history_local p_a (multi_H p_a p_b p_c p_a)"
proof -
```

```
  let ?L = "[Receive p_a 1 p_b 1, Send p_a 2 p_c 2]"
  have list_eq: "multi_H p_a p_b p_c p_a = ?L"
    using assms by (rule multi_H_at_p_a)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_a" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma multi_wf_history_local_p_b:
  assumes "p_a ≠ p_b" "p_b ≠ p_c"
  shows "wf_history_local p_b (multi_H p_a p_b p_c p_b)"
proof -
  let ?L = "[Send p_b 1 p_a 1]"
  have list_eq: "multi_H p_a p_b p_c p_b = ?L"
    using assms by (rule multi_H_at_p_b)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_b" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0" by auto
    thus "seq_of (?L ! k) = Suc k" by simp
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma multi_wf_history_local_p_c:
  assumes "p_a ≠ p_c" "p_b ≠ p_c"
  shows "wf_history_local p_c (multi_H p_a p_b p_c p_c)"
proof -
  let ?L = "[Receive p_c 1 p_a 2, Internal p_c 2]"
  have list_eq: "multi_H p_a p_b p_c p_c = ?L"
    using assms by (rule multi_H_at_p_c)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_c" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma multi_wf_history_local_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b" "p ≠ p_c"
```

```
  shows "wf_history_local p (multi_H p_a p_b p_c p)"
  using assms by (simp add: multi_H_elsewhere wf_history_local_def)

lemma multi_wf_history:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "wf_history (multi_H p_a p_b p_c)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (multi_H p_a p_b p_c p)"
  proof (cases "p = p_a")
    case True
    thus ?thesis using ab ac multi_wf_history_local_p_a by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_b")
      case True
      thus ?thesis using ab bc multi_wf_history_local_p_b by simp
    next
      case False
      show ?thesis
      proof (cases "p = p_c")
        case True
        thus ?thesis using ac bc multi_wf_history_local_p_c by simp
      next
        case False
        with ‹p ≠ p_a› ‹p ≠ p_b› show ?thesis
          by (rule multi_wf_history_local_elsewhere)
      qed
    qed
  qed
qed

lemma multi_adv_admissible:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "adversary_admissible correct (multi_adv p_a p_b p_c)"
proof -
  have wf: "wf_history (multi_H p_a p_b p_c)"
    using ab ac bc by (rule multi_wf_history)
  have i_corr: "adv_i (multi_adv p_a p_b p_c) ∈ correct"
    using pc by (simp add: multi_adv_def)
  have proc_eq:
    "proc_of (adv_e_star (multi_adv p_a p_b p_c)) = adv_i (multi_adv p_a p_b p_c)"
    by (simp add: multi_adv_def)
  have target_in:
    "adv_e_star (multi_adv p_a p_b p_c)
        ∈ events_of (adv_E (multi_adv p_a p_b p_c))"
    using ab ac bc by (simp add: multi_adv_def multi_H_events)
  show ?thesis
    using wf i_corr proc_eq target_in
    by (simp add: adversary_admissible_def multi_adv_def)
```

qed

73 The naive algorithm solves CD_B at the multihop adversary

Headline multihop demo: the naive algorithm at recv_from_history solves CD_B at the multihop adversary (target event at p_c via a two-hop bhb chain).

```
theorem T6_multihop_demo:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "valid_B correct (adv_E (multi_adv p_a p_b p_c))
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i (multi_adv p_a p_b p_c))
                                          (adv_E (multi_adv p_a p_b p_c)))
                       (adv_i (multi_adv p_a p_b p_c))
                       (adv_e_star (multi_adv p_a p_b p_c))))
                (adv_e_star (multi_adv p_a p_b p_c))"
proof -
  let ?cfg = "multi_cfg5 p_a p_b p_c"
  let ?adv = "multi_adv p_a p_b p_c"
  have run_cfg: "run ?cfg"
    by (rule multi_run[OF pa pb pc ab ac bc])
  have drained: "cfg_inflight ?cfg = empty_inflight"
    by (rule multi_drained)
  have adm: "adversary_admissible correct ?adv"
    by (rule multi_adv_admissible[OF pa pb pc ab ac bc])
  have adv_E_eq: "adv_E ?adv = cfg_hist ?cfg"
    using multi_cfg_hist_eq[of p_a p_b p_c] by (simp add: multi_adv_def)
  show ?thesis
    by (rule fair_drained_run_solves_CD_B_unicast[OF run_cfg drained adm adv_E_eq])
qed
```

Existential witness: T6 is non-vacuously satisfiable at the multihop scenario whenever three distinct correct processes exist.

```
theorem T6_multihop_witnessed:
  assumes three_correct:
    "∃p_a p_b p_c. p_a ∈ correct ∧ p_b ∈ correct ∧ p_c ∈ correct
                   ∧ p_a ≠ p_b ∧ p_a ≠ p_c ∧ p_b ≠ p_c"
  shows "∃adv cfg.
           run cfg
         ∧ cfg_inflight cfg = empty_inflight
         ∧ adversary_admissible correct adv
         ∧ adv_E adv = cfg_hist cfg
         ∧ valid_B correct (adv_E adv)
                   (fst (naive_cd_B_alg
                          (recv_from_history (adv_i adv) (adv_E adv))
                          (adv_i adv) (adv_e_star adv)))
                   (adv_e_star adv)"
proof -
  obtain p_a p_b p_c
```

```
    where pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
    using three_correct by blast
  let ?cfg = "multi_cfg5 p_a p_b p_c"
  let ?adv = "multi_adv p_a p_b p_c"
  have run_cfg: "run ?cfg" by (rule multi_run[OF pa pb pc ab ac bc])
  have drained: "cfg_inflight ?cfg = empty_inflight" by (rule multi_drained)
  have adm: "adversary_admissible correct ?adv"
    by (rule multi_adv_admissible[OF pa pb pc ab ac bc])
  have adv_E_eq: "adv_E ?adv = cfg_hist ?cfg"
    using multi_cfg_hist_eq[of p_a p_b p_c] by (simp add: multi_adv_def)
  have solves: "valid_B correct (adv_E ?adv)
                      (fst (naive_cd_B_alg
                            (recv_from_history (adv_i ?adv) (adv_E ?adv))
                            (adv_i ?adv) (adv_e_star ?adv)))
                      (adv_e_star ?adv)"
    by (rule T6_multihop_demo[OF pa pb pc ab ac bc])
  from run_cfg drained adm adv_E_eq solves show ?thesis by blast
qed
```

74 The transitive bhb chain through three correct processes

We expose the two-hop $\rightarrow_B$ chain from p_b's send through p_a's relay to p_c's target. This is the substantive content of the multihop demo beyond the 1-message case: a causal dependency that crosses two messages and three processes is correctly captured by the bhb relation.

```
lemma multi_send_to_recv_at_pa_is_message_order:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "message_order (multi_H p_a p_b p_c)
           (Send p_b 1 p_a 1) (Receive p_a 1 p_b 1)"
  using ab ac bc unfolding message_order_def
  by (simp add: multi_H_events)

lemma multi_recv_to_send_at_pa_is_program_order:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "program_order (multi_H p_a p_b p_c)
           (Receive p_a 1 p_b 1) (Send p_a 2 p_c 2)"
proof -
  let ?H = "multi_H p_a p_b p_c"
  have list_eq: "?H p_a = [Receive p_a 1 p_b 1, Send p_a 2 p_c 2]"
    using ab ac by (rule multi_H_at_p_a)
  have len: "length (?H p_a) = 2" by (simp add: list_eq)
  have e0: "(?H p_a) ! 0 = Receive p_a 1 p_b 1" by (simp add: list_eq)
  have e1: "(?H p_a) ! 1 = Send p_a 2 p_c 2" by (simp add: list_eq)
  have "(0 :: nat) < 1" by simp
  moreover have "(1 :: nat) < length (?H p_a)" using len by simp
  ultimately show ?thesis
    using e0 e1 unfolding program_order_def by blast
qed
```

```
lemma multi_send_to_recv_at_pc_is_message_order:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "message_order (multi_H p_a p_b p_c)
           (Send p_a 2 p_c 2) (Receive p_c 1 p_a 2)"
  using ab ac bc unfolding message_order_def
  by (simp add: multi_H_events)

lemma multi_recv_to_internal_at_pc_is_program_order:
  assumes ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "program_order (multi_H p_a p_b p_c)
           (Receive p_c 1 p_a 2) (Internal p_c 2)"
proof -
  let ?H = "multi_H p_a p_b p_c"
  have list_eq: "?H p_c = [Receive p_c 1 p_a 2, Internal p_c 2]"
    using ac bc by (rule multi_H_at_p_c)
  have len: "length (?H p_c) = 2" by (simp add: list_eq)
  have e0: "(?H p_c) ! 0 = Receive p_c 1 p_a 2" by (simp add: list_eq)
  have e1: "(?H p_c) ! 1 = Internal p_c 2" by (simp add: list_eq)
  have "(0 :: nat) < 1" by simp
  moreover have "(1 :: nat) < length (?H p_c)" using len by simp
  ultimately show ?thesis
    using e0 e1 unfolding program_order_def by blast
qed
```

The full bhb chain: `Send p_b 1 p_a 1` bhb-precedes `Internal p_c 2` through four hops crossing three processes.

```
theorem multi_bhb_chain:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc: "p_c ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "bhb correct (multi_H p_a p_b p_c)
           (Send p_b 1 p_a 1) (Internal p_c 2)"
proof -
  let ?H = "multi_H p_a p_b p_c"
  let ?e1 = "Send p_b 1 p_a 1"
  let ?e2 = "Receive p_a 1 p_b 1"
  let ?e3 = "Send p_a 2 p_c 2"
  let ?e4 = "Receive p_c 1 p_a 2"
  let ?e5 = "Internal p_c 2"

  have proc_e1: "proc_of ?e1 = p_b" by simp
  have proc_e2: "proc_of ?e2 = p_a" by simp
  have proc_e3: "proc_of ?e3 = p_a" by simp
  have proc_e4: "proc_of ?e4 = p_c" by simp
  have proc_e5: "proc_of ?e5 = p_c" by simp

  have step12: "bhb_step correct ?H ?e1 ?e2"
    using pa pb proc_e1 proc_e2
          multi_send_to_recv_at_pa_is_message_order[OF ab ac bc]
    unfolding bhb_step_def by simp
  have step23: "bhb_step correct ?H ?e2 ?e3"
    using pa proc_e2 proc_e3
```

```
        multi_recv_to_send_at_pa_is_program_order[OF ab ac bc]
    unfolding bhb_step_def by simp
  have step34: "bhb_step correct ?H ?e3 ?e4"
    using pa pc proc_e3 proc_e4
        multi_send_to_recv_at_pc_is_message_order[OF ab ac bc]
    unfolding bhb_step_def by simp
  have step45: "bhb_step correct ?H ?e4 ?e5"
    using pc proc_e4 proc_e5
        multi_recv_to_internal_at_pc_is_program_order[OF ac bc]
    unfolding bhb_step_def by simp

  have t12: "(bhb_step correct ?H)^++ ?e1 ?e2" using step12 by blast
  have t23: "(bhb_step correct ?H)^++ ?e2 ?e3" using step23 by blast
  have t34: "(bhb_step correct ?H)^++ ?e3 ?e4" using step34 by blast
  have t45: "(bhb_step correct ?H)^++ ?e4 ?e5" using step45 by blast
  have t13: "(bhb_step correct ?H)^++ ?e1 ?e3" using t12 t23 by (rule tranclp_trans)
  have t14: "(bhb_step correct ?H)^++ ?e1 ?e4" using t13 t34 by (rule tranclp_trans)
  have "(bhb_step correct ?H)^++ ?e1 ?e5" using t14 t45 by (rule tranclp_trans)
  thus ?thesis unfolding bhb_def .
qed

end — context byzantineSystem

end


theory T6_With_Byzantine
  imports T6_Concrete
begin

context byzantineSystem
begin
```

75 Three-process scenario with a Byzantine bystander

The target history. Two correct processes `p_a`, `p_b` exchanging one message, plus a Byzantine process `p_c` running an internal event during the exchange.

```
definition byz_demo_H :: "'p ⇒ 'p ⇒ 'p ⇒ 'p history" where
  "byz_demo_H p_a p_b p_c =
     (λp. if p = p_a then [Receive p_a 1 p_b 0, Internal p_a 2]
          else if p = p_b then [Send p_b 1 p_a 0]
          else if p = p_c then [Internal p_c 1]
          else [])"

definition byz_demo_adv :: "'p ⇒ 'p ⇒ 'p ⇒ 'p adversary" where
  "byz_demo_adv p_a p_b p_c =
     (| adv_E = byz_demo_H p_a p_b p_c,
       adv_e_star = Internal p_a 2,
       adv_i = p_a |)"

lemma byz_demo_H_at_p_a:
```

```
  assumes "p_a ≠ p_b" "p_a ≠ p_c"
  shows "byz_demo_H p_a p_b p_c p_a = [Receive p_a 1 p_b 0, Internal p_a 2]"
  using assms by (simp add: byz_demo_H_def)

lemma byz_demo_H_at_p_b:
  assumes "p_a ≠ p_b" "p_b ≠ p_c"
  shows "byz_demo_H p_a p_b p_c p_b = [Send p_b 1 p_a 0]"
  using assms by (simp add: byz_demo_H_def)

lemma byz_demo_H_at_p_c:
  assumes "p_a ≠ p_c" "p_b ≠ p_c"
  shows "byz_demo_H p_a p_b p_c p_c = [Internal p_c 1]"
  using assms by (simp add: byz_demo_H_def)

lemma byz_demo_H_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b" "p ≠ p_c"
  shows "byz_demo_H p_a p_b p_c p = []"
  using assms by (simp add: byz_demo_H_def)

lemma byz_demo_H_events:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "events_of (byz_demo_H p_a p_b p_c) =
           {Receive p_a 1 p_b 0, Internal p_a 2,
            Send p_b 1 p_a 0, Internal p_c 1}"
proof -
  have "events_of (byz_demo_H p_a p_b p_c)
          = (⋃p. set (byz_demo_H p_a p_b p_c p))"
    by (simp add: events_of_def)
  also have "... = set (byz_demo_H p_a p_b p_c p_a)
                   ∪ set (byz_demo_H p_a p_b p_c p_b)
                   ∪ set (byz_demo_H p_a p_b p_c p_c)
                   ∪ (⋃p ∈ -{p_a, p_b, p_c}. set (byz_demo_H p_a p_b p_c p))"
    by auto
  also have "set (byz_demo_H p_a p_b p_c p_a)
                = {Receive p_a 1 p_b 0, Internal p_a 2}"
    using ab ac by (simp add: byz_demo_H_at_p_a)
  moreover have "set (byz_demo_H p_a p_b p_c p_b) = {Send p_b 1 p_a 0}"
    using ab bc by (simp add: byz_demo_H_at_p_b)
  moreover have "set (byz_demo_H p_a p_b p_c p_c) = {Internal p_c 1}"
    using ac bc by (simp add: byz_demo_H_at_p_c)
  moreover have "(⋃p ∈ -{p_a, p_b, p_c}.
                       set (byz_demo_H p_a p_b p_c p)) = {}"
    by (auto simp: byz_demo_H_elsewhere)
  ultimately show ?thesis by auto
qed
```

76 Four intermediate configurations along the run

From `init_config` we trace four `run_step` transitions:

- `byz_cfg1`: after `step_send p_b 1 p_a 0`

- byz_cfg2: after step_byzantine (Internal p_c 1)
- byz_cfg3: after step_recv p_a 1 p_b 0
- byz_cfg4: after step_internal p_a 2 (drained final)

```
definition byz_cfg1 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "byz_cfg1 p_a p_b p_c =
     (| cfg_hist = (λp. if p = p_b then [Send p_b 1 p_a 0] else []),
       cfg_inflight = empty_inflight ∪# {# (p_b, p_a, 0) } |)"

definition byz_cfg2 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "byz_cfg2 p_a p_b p_c =
     (| cfg_hist = (λp. if p = p_b then [Send p_b 1 p_a 0]
                        else if p = p_c then [Internal p_c 1]
                        else []),
       cfg_inflight = empty_inflight ∪# {# (p_b, p_a, 0) } |)"

definition byz_cfg3 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "byz_cfg3 p_a p_b p_c =
     (| cfg_hist = (λp. if p = p_a then [Receive p_a 1 p_b 0]
                        else if p = p_b then [Send p_b 1 p_a 0]
                        else if p = p_c then [Internal p_c 1]
                        else []),
       cfg_inflight = empty_inflight |)"

definition byz_cfg4 :: "'p ⇒ 'p ⇒ 'p ⇒ 'p config" where
  "byz_cfg4 p_a p_b p_c =
     (| cfg_hist = byz_demo_H p_a p_b p_c,
       cfg_inflight = empty_inflight |)"
```

77 The four run_step transitions

```
lemma byz_step_1_send:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and ab: "p_a ≠ p_b"
  shows "run_step init_config (byz_cfg1 p_a p_b p_c)"
proof -
  let ?cfg0 = "init_config"
  have hist_pb: "cfg_hist ?cfg0 p_b = []"
    by (simp add: init_config_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg0 p_b))"
    using hist_pb by simp
  have hist_eq: "cfg_hist (byz_cfg1 p_a p_b p_c)
                 = (cfg_hist ?cfg0)
                     (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 0])"
    by (rule ext) (simp add: byz_cfg1_def init_config_def)
  have inflight_eq: "cfg_inflight (byz_cfg1 p_a p_b p_c)
                       = cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 0) }"
    by (simp add: byz_cfg1_def init_config_def)
  have cfg_eq: "byz_cfg1 p_a p_b p_c
                 = ?cfg0 (| cfg_hist :=
```

```
                              (cfg_hist ?cfg0)
                                (p_b := cfg_hist ?cfg0 p_b @ [Send p_b 1 p_a 0]),
                             cfg_inflight :=
                              cfg_inflight ?cfg0 ∪# {# (p_b, p_a, 0) } ⦆"
    using hist_eq inflight_eq by (simp add: byz_cfg1_def init_config_def)
  show ?thesis
    by (rule run_step.step_send[OF pb pa n_eq cfg_eq])
qed

lemma byz_step_2_byzantine:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (byz_cfg1 p_a p_b p_c) (byz_cfg2 p_a p_b p_c)"
proof -
  let ?cfg1 = "byz_cfg1 p_a p_b p_c"
  let ?ev = "Internal p_c 1"
  have hist_pc: "cfg_hist ?cfg1 p_c = []"
    using bc by (simp add: byz_cfg1_def)
  have proc_ev: "proc_of ?ev = p_c" by simp
  have seq_ev: "seq_of ?ev = Suc (length (cfg_hist ?cfg1 p_c))"
    using hist_pc by simp
  have hist_eq: "cfg_hist (byz_cfg2 p_a p_b p_c)
                 = (cfg_hist ?cfg1)(p_c := cfg_hist ?cfg1 p_c @ [?ev])"
    using bc ac hist_pc
    by (rule_tac ext) (simp add: byz_cfg1_def byz_cfg2_def)
  have inflight_eq: "cfg_inflight (byz_cfg2 p_a p_b p_c)
                       = cfg_inflight ?cfg1"
    by (simp add: byz_cfg1_def byz_cfg2_def)
  have cfg_eq: "byz_cfg2 p_a p_b p_c
                 = ?cfg1 ⦇ cfg_hist :=
                             (cfg_hist ?cfg1)(p_c := cfg_hist ?cfg1 p_c @ [?ev])
⦈"
    using hist_eq inflight_eq by (simp add: byz_cfg2_def)
  show ?thesis
    by (rule run_step.step_byzantine[OF pc_byz proc_ev seq_ev cfg_eq])
qed

lemma byz_step_3_recv:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (byz_cfg2 p_a p_b p_c) (byz_cfg3 p_a p_b p_c)"
proof -
  let ?cfg2 = "byz_cfg2 p_a p_b p_c"
  have buf_in: "(p_b, p_a, 0) ∈# cfg_inflight ?cfg2"
    by (simp add: byz_cfg2_def empty_inflight_def)
  have hist_pa: "cfg_hist ?cfg2 p_a = []"
    using ab ac by (simp add: byz_cfg2_def)
  have n_eq: "(1 :: nat) = Suc (length (cfg_hist ?cfg2 p_a))"
    using hist_pa by simp
  have hist_eq: "cfg_hist (byz_cfg3 p_a p_b p_c)
                 = (cfg_hist ?cfg2)
                     (p_a := cfg_hist ?cfg2 p_a @ [Receive p_a 1 p_b 0])"
```

```
    using ab ac hist_pa
    by (rule_tac ext) (simp add: byz_cfg2_def byz_cfg3_def)
  have inflight_eq: "cfg_inflight (byz_cfg3 p_a p_b p_c)
                       = cfg_inflight ?cfg2 -# (p_b, p_a, 0)"
    by (rule ext) (simp add: byz_cfg2_def byz_cfg3_def empty_inflight_def)
  have cfg_eq: "byz_cfg3 p_a p_b p_c
                  = ?cfg2 (| cfg_hist :=
                               (cfg_hist ?cfg2)
                                 (p_a := cfg_hist ?cfg2 p_a @ [Receive p_a 1 p_b 0]),
                             cfg_inflight :=
                               cfg_inflight ?cfg2 -# (p_b, p_a, 0) |)"
    using hist_eq inflight_eq by (simp add: byz_cfg3_def)
  show ?thesis
    by (rule run_step.step_recv[OF pa pb buf_in n_eq cfg_eq])
qed

lemma byz_step_4_internal:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run_step (byz_cfg3 p_a p_b p_c) (byz_cfg4 p_a p_b p_c)"
proof -
  let ?cfg3 = "byz_cfg3 p_a p_b p_c"
  have hist_pa: "cfg_hist ?cfg3 p_a = [Receive p_a 1 p_b 0]"
    using ab ac by (simp add: byz_cfg3_def)
  have n_eq: "(2 :: nat) = Suc (length (cfg_hist ?cfg3 p_a))"
    using hist_pa by simp
  have hist_eq: "cfg_hist (byz_cfg4 p_a p_b p_c)
                   = (cfg_hist ?cfg3)
                       (p_a := cfg_hist ?cfg3 p_a @ [Internal p_a 2])"
    using ab ac bc hist_pa
    by (rule_tac ext) (simp add: byz_cfg3_def byz_cfg4_def byz_demo_H_def)
  have inflight_eq: "cfg_inflight (byz_cfg4 p_a p_b p_c)
                       = cfg_inflight ?cfg3"
    by (simp add: byz_cfg3_def byz_cfg4_def)
  have cfg_eq: "byz_cfg4 p_a p_b p_c
                  = ?cfg3 (| cfg_hist :=
                               (cfg_hist ?cfg3)
                                 (p_a := cfg_hist ?cfg3 p_a @ [Internal p_a 2]) |)"
    using hist_eq inflight_eq by (simp add: byz_cfg4_def)
  show ?thesis
    by (rule run_step.step_internal[OF pa n_eq cfg_eq])
qed
```

78 The byz-presence run is fair, drained, and produces byz_demo_H

```
lemma byz_run:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct" and pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "run (byz_cfg4 p_a p_b p_c)"
proof -
```

```
  have r0: "run init_config" by simp
  have s1: "run_step init_config (byz_cfg1 p_a p_b p_c)"
    by (rule byz_step_1_send[OF pa pb ab])
  have r1: "run (byz_cfg1 p_a p_b p_c)" by (rule run_extend[OF r0 s1])
  have s2: "run_step (byz_cfg1 p_a p_b p_c) (byz_cfg2 p_a p_b p_c)"
    by (rule byz_step_2_byzantine[OF pa pb pc_byz ab ac bc])
  have r2: "run (byz_cfg2 p_a p_b p_c)" by (rule run_extend[OF r1 s2])
  have s3: "run_step (byz_cfg2 p_a p_b p_c) (byz_cfg3 p_a p_b p_c)"
    by (rule byz_step_3_recv[OF pa pb ab ac bc])
  have r3: "run (byz_cfg3 p_a p_b p_c)" by (rule run_extend[OF r2 s3])
  have s4: "run_step (byz_cfg3 p_a p_b p_c) (byz_cfg4 p_a p_b p_c)"
    by (rule byz_step_4_internal[OF pa pb ab ac bc])
  show ?thesis by (rule run_extend[OF r3 s4])
qed

lemma byz_drained:
  shows "cfg_inflight (byz_cfg4 p_a p_b p_c) = empty_inflight"
  by (simp add: byz_cfg4_def)

lemma byz_cfg_hist_eq:
  shows "cfg_hist (byz_cfg4 p_a p_b p_c) = byz_demo_H p_a p_b p_c"
  by (simp add: byz_cfg4_def)
```

79 Well-formedness and adversary admissibility

```
lemma byz_wf_local_p_a:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c"
  shows "wf_history_local p_a (byz_demo_H p_a p_b p_c p_a)"
proof -
  let ?L = "[Receive p_a 1 p_b 0, Internal p_a 2]"
  have list_eq: "byz_demo_H p_a p_b p_c p_a = ?L"
    using ab ac by (rule byz_demo_H_at_p_a)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_a" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma byz_wf_local_p_b:
  assumes ab: "p_a ≠ p_b" and bc: "p_b ≠ p_c"
  shows "wf_history_local p_b (byz_demo_H p_a p_b p_c p_b)"
proof -
  let ?L = "[Send p_b 1 p_a 0]"
  have list_eq: "byz_demo_H p_a p_b p_c p_b = ?L"
    using ab bc by (rule byz_demo_H_at_p_b)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_b" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
```

```
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0" by auto
    thus "seq_of (?L ! k) = Suc k" by simp
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma byz_wf_local_p_c:
  assumes ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "wf_history_local p_c (byz_demo_H p_a p_b p_c p_c)"
proof -
  let ?L = "[Internal p_c 1]"
  have list_eq: "byz_demo_H p_a p_b p_c p_c = ?L"
    using ac bc by (rule byz_demo_H_at_p_c)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_c" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0" by auto
    thus "seq_of (?L ! k) = Suc k" by simp
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma byz_wf_local_elsewhere:
  assumes "p ≠ p_a" "p ≠ p_b" "p ≠ p_c"
  shows "wf_history_local p (byz_demo_H p_a p_b p_c p)"
  using assms by (simp add: byz_demo_H_elsewhere wf_history_local_def)

lemma byz_wf_history:
  assumes ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "wf_history (byz_demo_H p_a p_b p_c)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (byz_demo_H p_a p_b p_c p)"
  proof (cases "p = p_a")
    case True
    thus ?thesis using ab ac byz_wf_local_p_a by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_b")
      case True
      thus ?thesis using ab bc byz_wf_local_p_b by simp
    next
      case False
      show ?thesis
      proof (cases "p = p_c")
        case True
```

```
      thus ?thesis using ac bc byz_wf_local_p_c by simp
    next
      case False
      with ‹p ≠ p_a› ‹p ≠ p_b› show ?thesis
        by (rule byz_wf_local_elsewhere)
    qed
  qed
qed

lemma byz_adv_admissible:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "adversary_admissible correct (byz_demo_adv p_a p_b p_c)"
proof -
  have wf: "wf_history (byz_demo_H p_a p_b p_c)"
    using ab ac bc by (rule byz_wf_history)
  have i_corr: "adv_i (byz_demo_adv p_a p_b p_c) ∈ correct"
    using pa by (simp add: byz_demo_adv_def)
  have proc_eq:
    "proc_of (adv_e_star (byz_demo_adv p_a p_b p_c))
       = adv_i (byz_demo_adv p_a p_b p_c)"
    by (simp add: byz_demo_adv_def)
  have target_in:
    "adv_e_star (byz_demo_adv p_a p_b p_c)
        ∈ events_of (adv_E (byz_demo_adv p_a p_b p_c))"
    using ab ac bc by (simp add: byz_demo_adv_def byz_demo_H_events)
  show ?thesis
    using wf i_corr proc_eq target_in
    by (simp add: adversary_admissible_def byz_demo_adv_def)
qed
```

80 The naive algorithm solves CD_B despite the Byzantine bystander

Headline theorem: with a Byzantine process p_c actively producing a local internal event during the run, the naive algorithm *still* solves CD_B at the correct adversary's target. This is the operational core of T6's robustness claim.

```
theorem T6_with_byzantine_demo:
  assumes pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "valid_B correct (adv_E (byz_demo_adv p_a p_b p_c))
                (fst (naive_cd_B_alg
                       (recv_from_history (adv_i (byz_demo_adv p_a p_b p_c))
                                          (adv_E (byz_demo_adv p_a p_b p_c)))
                       (adv_i (byz_demo_adv p_a p_b p_c))
                       (adv_e_star (byz_demo_adv p_a p_b p_c))))
                (adv_e_star (byz_demo_adv p_a p_b p_c))"
proof -
```

```
  let ?cfg = "byz_cfg4 p_a p_b p_c"
  let ?adv = "byz_demo_adv p_a p_b p_c"
  have run_cfg: "run ?cfg"
    by (rule byz_run[OF pa pb pc_byz ab ac bc])
  have drained: "cfg_inflight ?cfg = empty_inflight" by (rule byz_drained)
  have adm: "adversary_admissible correct ?adv"
    by (rule byz_adv_admissible[OF pa pb ab ac bc])
  have adv_E_eq: "adv_E ?adv = cfg_hist ?cfg"
    using byz_cfg_hist_eq[of p_a p_b p_c] by (simp add: byz_demo_adv_def)
  show ?thesis
    by (rule fair_drained_run_solves_CD_B_unicast[OF run_cfg drained adm adv_E_eq])
qed
```

Existential witness: T6 holds in the presence of a live Byzantine process whenever the locale supplies two distinct correct processes plus a distinct Byzantine process.

```
theorem T6_with_byzantine_witnessed:
  assumes pieces:
    "∃p_a p_b p_c. p_a ∈ correct ∧ p_b ∈ correct ∧ p_c ∈ byzantine
                  ∧ p_a ≠ p_b ∧ p_a ≠ p_c ∧ p_b ≠ p_c"
  shows "∃adv cfg.
           run cfg
         ∧ cfg_inflight cfg = empty_inflight
         ∧ adversary_admissible correct adv
         ∧ adv_E adv = cfg_hist cfg
         ∧ valid_B correct (adv_E adv)
                   (fst (naive_cd_B_alg
                          (recv_from_history (adv_i adv) (adv_E adv))
                          (adv_i adv) (adv_e_star adv)))
                   (adv_e_star adv)"
proof -
  obtain p_a p_b p_c
    where pa: "p_a ∈ correct" and pb: "p_b ∈ correct"
      and pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
    using pieces by blast
  let ?cfg = "byz_cfg4 p_a p_b p_c"
  let ?adv = "byz_demo_adv p_a p_b p_c"
  have run_cfg: "run ?cfg" by (rule byz_run[OF pa pb pc_byz ab ac bc])
  have drained: "cfg_inflight ?cfg = empty_inflight" by (rule byz_drained)
  have adm: "adversary_admissible correct ?adv"
    by (rule byz_adv_admissible[OF pa pb ab ac bc])
  have adv_E_eq: "adv_E ?adv = cfg_hist ?cfg"
    using byz_cfg_hist_eq[of p_a p_b p_c] by (simp add: byz_demo_adv_def)
  have solves: "valid_B correct (adv_E ?adv)
                       (fst (naive_cd_B_alg
                              (recv_from_history (adv_i ?adv) (adv_E ?adv))
                              (adv_i ?adv) (adv_e_star ?adv)))
                       (adv_e_star ?adv)"
    by (rule T6_with_byzantine_demo[OF pa pb pc_byz ab ac bc])
  from run_cfg drained adm adv_E_eq solves show ?thesis by blast
qed
```

81 The Byzantine's internal event is not on any bhb chain

The paper-faithful sanity check: the Byzantine's local event `Internal p_c 1` is *not* on any bhb chain to the adversary's target (or any other correct event), because `bhb` steps require both endpoints to be in `correct` and `p_c ∈ byzantine`.

Concretely, `Internal p_c 1` has `proc_of = p_c`, which is in `byzantine`; hence by `bhb ?C ?H ?e ?e' ⟹ proc_of ?e ∈ ?C ∧ proc_of ?e' ∈ ?C` no bhb chain ends at this event (or starts from it).

```
lemma byzantine_event_not_on_bhb_chain_left:
  assumes pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "¬ bhb correct (byz_demo_H p_a p_b p_c) (Internal p_c 1) e'"
proof
  assume bhb_chain: "bhb correct (byz_demo_H p_a p_b p_c) (Internal p_c 1) e'"
  hence "proc_of (Internal p_c 1) ∈ correct"
    using bhb_proc_of_endpoints by blast
  hence "p_c ∈ correct" by simp
  thus False using pc_byz partition_disj by blast
qed

lemma byzantine_event_not_on_bhb_chain_right:
  assumes pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "¬ bhb correct (byz_demo_H p_a p_b p_c) e (Internal p_c 1)"
proof
  assume bhb_chain: "bhb correct (byz_demo_H p_a p_b p_c) e (Internal p_c 1)"
  hence "proc_of (Internal p_c 1) ∈ correct"
    using bhb_proc_of_endpoints by blast
  hence "p_c ∈ correct" by simp
  thus False using pc_byz partition_disj by blast
qed
```

In particular: the Byzantine event neither hb-before-precedes nor hb-after-succeeds the adversary's target event `Internal p_a 2` under `bhb_eval`. The algorithm correctly does not include the Byzantine event in any bhb-relation to the target.

```
theorem bhb_eval_byzantine_unrelated_to_target:
  assumes pc_byz: "p_c ∈ byzantine"
      and ab: "p_a ≠ p_b" and ac: "p_a ≠ p_c" and bc: "p_b ≠ p_c"
  shows "¬ bhb_eval correct (byz_demo_H p_a p_b p_c)
                (Internal p_c 1) (Internal p_a 2)"
    and "¬ bhb_eval correct (byz_demo_H p_a p_b p_c)
                (Internal p_a 2) (Internal p_c 1)"
proof -
  show "¬ bhb_eval correct (byz_demo_H p_a p_b p_c)
                 (Internal p_c 1) (Internal p_a 2)"
    using byzantine_event_not_on_bhb_chain_left[OF pc_byz ab ac bc]
    by (simp add: bhb_eval_def)
  show "¬ bhb_eval correct (byz_demo_H p_a p_b p_c)
                 (Internal p_a 2) (Internal p_c 1)"
    using byzantine_event_not_on_bhb_chain_right[OF pc_byz ab ac bc]
    by (simp add: bhb_eval_def)
```

qed

end — context `byzantineSystem`

end

theory `CD_vs_Consensus`
  imports
    `Impossibility`
    `Foundation_Vacuity`
    `FLP_Consensus`
    `CD_B_Algorithm`
begin

82 Theorem 15: in a Byzantine setting, CD is harder than Consensus

Paper, Section 5.1:

> "Theorem 15. In an asynchronous system with Byzantine failures, `CD` $\neg\preceq$ `Consensus` and the CD problem is harder than Consensus."

The paper's proof first exhibits an oracle (process-identification) that is sufficient to solve Consensus, then revisits Theorem 1 to argue that even with that oracle CD still admits false negatives. Hence "Consensus-solvability does not imply CD-solvability".

Our formalisation. At the abstraction level of this development, the abstract predicate `solves_Consensus` is trivially satisfiable by a pure-HOL function (see `∃alg. solves_Consensus ?C alg` in `Foundation_Vacuity.thy`, a regression diagnostic that retains the witness). CD on the other hand is unsolvable by Theorem 1. Conjoining these two facts already gives the formal content of Theorem 15:

> `solves_Consensus`-witness exists, but no `produces_valid_F`-witness does, so the reduction ``Consensus-solver ⟶ CD-solver'' is False.`

A note on idiom. The paper uses the formal symbol $\preceq$ for the "reduces to" relation: `X` $\preceq$ `Y` means "a Y-solver yields an X-solver". Under that reading, `CD` $\preceq$ `Consensus` unfolds to "every Consensus-solver yields some CD-solver". In HOL: `(∃ a. solves_Consensus correct a) ⟶ (∃ a. produces_valid_F correct a)`. Theorem 15 negates that.

context `byzantineSystem`
begin

theorem `CD_unsolvable_but_Consensus_solvable:`
  assumes `byz_ne: "byzantine ≠ {}"`
    and `cor_ne: "correct ≠ {}"`
    and `fin_cd:`
      `"∀cd_alg. produces_valid_F correct cd_alg ⟶`
        `(∀p_i_in. finite (events_of`
                    `(fst (cd_alg p_i_in (Internal p_i_in 2)))))"`

```
  shows "(∃cons_alg. solves_Consensus correct cons_alg)
       ∧ ¬ (∃cd_alg. produces_valid_F correct cd_alg)"
proof
  show "∃cons_alg. solves_Consensus correct cons_alg"
    by (rule exists_consensus_alg)
  show "¬ (∃cd_alg. produces_valid_F correct cd_alg)"
    by (rule no_produces_valid_F[OF byz_ne cor_ne fin_cd])
qed
```

theorem `CD_not_reducible_to_Consensus:`
— Paper's "`CD` $\neg\preceq$ `Consensus`" in Byzantine setting. Reading `X` $\preceq$ `Y` as "Y-solver yields X-solver", `CD` $\preceq$ `Consensus` would be `(∃a. solves_Consensus correct a)` $\longrightarrow$ `(∃a. produces_valid_F correct a)`; we negate that.

```
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ ((∃cons_alg. solves_Consensus correct cons_alg) ⟶
            (∃cd_alg. produces_valid_F correct cd_alg))"
proof -
  from CD_unsolvable_but_Consensus_solvable[OF byz_ne cor_ne fin_cd]
  show ?thesis by blast
qed
```

theorem `CD_harder_than_Consensus:`
— Paper's headline conclusion of Theorem 15: "the CD problem is harder than Consensus". In our setting this is the conjunction of (i) Consensus is abstractly solvable, (ii) CD is unsolvable, (iii) hence no reduction `CD` $\preceq$ `Consensus` exists.

```
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                             (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "(∃cons_alg. solves_Consensus correct cons_alg)
       ∧ ¬ (∃cd_alg. produces_valid_F correct cd_alg)
       ∧ ¬ ((∃cons_alg. solves_Consensus correct cons_alg)
             ⟶ (∃cd_alg. produces_valid_F correct cd_alg))"
  using CD_unsolvable_but_Consensus_solvable[OF byz_ne cor_ne fin_cd]
        CD_not_reducible_to_Consensus[OF byz_ne cor_ne fin_cd]
  by blast
```

end — context `byzantineSystem`

83 Theorem 16: in a crash-failure setting, Consensus is harder than CD

Paper, Section 5.1:

> "Theorem 16. In an asynchronous system with crash failures, CD is solvable

but Consensus is not solvable; thus `Consensus` $\neg\preceq$ `CD` and `CD` $\preceq$ `Consensus`."

The paper's proof has two parts:

1. "To solve CD does not require identifying the crashed processes; their (correct) execution histories can be faithfully transmitted to other processes (transitively) via the execution messages ..." – i.e., CD is solvable in the crash-failure model.
2. "Solving Consensus in the crash failure model is impossible by the FLP impossibility result."

Half (2) – formalised. The FLP impossibility on the asynchronous-distributed model of the AFP entry is the proven theorem ¬ `flp_consensus_solvable ?transFn ?sendsFn ?startFn` in `FLP_Consensus.thy`. We re-export it below.

Half (1) – formalised against the `cd_alg_with_recv` *signature of* `CD_B_Algorithm.thy`. The bare `cd_solver` signature (`'p ⇒ 'p event ⇒ 'p history × bool`) gives the algorithm only the query indices `i` and `e_star`, with no way to "collect" messages. The paper's "transitive propagation via execution messages" argument requires a signature in which the algorithm has an input channel. The `cd_alg_with_recv` signature of `CD_B_Algorithm.thy` provides exactly such a channel: an algorithm of that signature additionally takes a per-peer reported history `recv` – "what `p_i` has been told about each other process's execution".

In the crash-failure model, no process lies; the only failure mode is to stop sending and stop executing. Hence every reported history the algorithm receives is faithful to the true execution `adv_E adv`, and "transitive propagation" (Section 5.1 of the paper) guarantees that all causally-relevant events reach every correct process. We model this abstractly as `recv = adv_E adv` pointwise: the algorithm has perfect information about each process's actual history.

Under this assumption, the naive algorithm `naive_cd_B_alg` from `CD_B_Algorithm.thy` – which simply outputs `F := recv` – trivially produces valid F: because `recv = E` the `valid` predicate reduces to `valid E E e_star`, which holds at every event by reflexivity of `hb_eval`.

theorem `T16_Consensus_unsolvable_part:`
— One half of Theorem 16: in the AFP entry's asynchronous-distributed model, FLP-style consensus is unsolvable. See `FLP_Consensus.thy` for the proof against AFP's ⟦`flpPseudoConsensus ?trans ?sends ?start; ⋀fe ft. asynchronousSystem.fairInfiniteExecution ?trans ?sends ?start fe ft ⟹ flpSystem.terminationFLP ?trans ?sends ?start fe ft; ∀i c. flpSystem.validity ?trans ?sends ?start i c; ∀i c. flpSystem.agreementInit ?trans ?sends ?start i c`⟧ ⟹ `False`.
shows "¬ `flp_consensus_solvable transFn sendsFn startFn`"
by `(rule flp_consensus_unsolvable)`

83.1 The CD-solvable half

Predicate: an algorithm of type `'p cd_alg_with_recv` satisfies the CD problem under crash failures (modelled abstractly as "every report is faithful") if, on every admissible adversary `adv` and every `recv` that pointwise matches `adv_E adv`, the algorithm's collected history is `valid` (in the plain `valid` sense, with respect to the actual execution).

Deviation: "every report is faithful" is the strongest form of the crash-model report-faithfulness assumption. The paper's "transitive propagation via execution messages"

technically only guarantees that events in the *causal past* of `e_star` are propagated, not necessarily *every* event at *every* process. We adopt the stronger pointwise-equality reading because (i) it is strictly easier to discharge (subsumes the causal-past-only version) and (ii) the paper's stated conclusion "CD is solvable" under crash failures is what we need to back up the `Consensus` $\neg\preceq$ `CD` reasoning that motivates Theorem 16.

```
definition produces_valid_F_recv ::
  "'p set ⇒ 'p cd_alg_with_recv ⇒ bool" where
  "produces_valid_F_recv P alg ⟷
     (∀adv recv.
        adversary_admissible P adv ⟶
        wf_history recv ⟶
        recv = adv_E adv ⟶
          (let (F', _) = alg recv (adv_i adv) (adv_e_star adv) in
             valid (adv_E adv) F' (adv_e_star adv)))"

context byzantineSystem
begin
```

The naive algorithm `naive_cd_B_alg` satisfies the crash-CD specification: when `recv = adv_E adv` the output `F := recv` equals `adv_E adv`, so `valid` reduces to `valid E E e_star`, which holds at every event.

```
lemma valid_self [simp]:
  shows "valid E E e_star"
  by (simp add: valid_def)

lemma naive_cd_B_alg_solves_CD_under_crash:
  shows "produces_valid_F_recv correct naive_cd_B_alg"
proof (unfold produces_valid_F_recv_def, intro allI impI)
  fix adv :: "'p adversary" and recv :: "'p ⇒ 'p history_local"
  assume adm:    "adversary_admissible correct adv"
     and wfR:    "wf_history recv"
     and rec_eq: "recv = adv_E adv"

  have valid_at: "valid (adv_E adv) recv (adv_e_star adv)"
    using rec_eq by simp

  show "let (F', _) = naive_cd_B_alg recv (adv_i adv) (adv_e_star adv) in
          valid (adv_E adv) F' (adv_e_star adv)"
    using valid_at by (simp add: naive_cd_B_alg_def Let_def)
qed

theorem T16_CD_solvable_under_crash_part:
```

— The constructive half of Theorem 16: in the crash-failure model, CD is solvable. Under the `cd_alg_with_recv` signature, the naive algorithm `F := recv` works whenever the report is faithful to the true execution – which is the abstract content of the paper's "transitive propagation via execution messages" claim.

```
  shows "∃alg. produces_valid_F_recv correct alg"
  using naive_cd_B_alg_solves_CD_under_crash by blast
```

83.2 Full Theorem 16

Paper Theorem 16 as a single statement: under crash failures, CD is solvable but Consensus is not. The asymmetry justifies the paper's twin conclusions `Consensus` $\neg\preceq$ `CD` (a CD solver does not yield a Consensus solver, because Consensus is impossible) and `CD` $\preceq$ `Consensus` (a Consensus solver yields a CD solver – this direction is the harder one in the paper's narrative, and is not directly mechanised here because it would require committing to a specific algorithm-from-Consensus construction).

```
theorem T16_full:
  shows "(∃alg. produces_valid_F_recv correct alg)
       ∧ ¬ flp_consensus_solvable transFn sendsFn startFn"
proof
  show "∃alg. produces_valid_F_recv correct alg"
    by (rule T16_CD_solvable_under_crash_part)
  show "¬ flp_consensus_solvable transFn sendsFn startFn"
    by (rule T16_Consensus_unsolvable_part)
qed
```

`Consensus` $\neg\preceq$ `CD` under crash failures (the headline conclusion of Theorem 16 in the paper, reading `X` $\preceq$ `Y` as "Y-solver yields X-solver"): a CD-solver in our richer signature exists, but no Consensus-solver in the FLP sense does, so the implication "crash-CD-solver $\longrightarrow$ Consensus-solver" is False.

```
theorem T16_Consensus_not_reducible_to_CD_under_crash:
  shows "¬ ((∃alg. produces_valid_F_recv correct alg)
             ⟶ flp_consensus_solvable transFn sendsFn startFn)"
proof -
  from T16_full
  show ?thesis by blast
qed

end — context byzantineSystem

end


theory CO
  imports Impossibility
begin
```

84 CO problem: definitions

Paper, Section 5.2, Definition 10:

> "The causal ordering problem `CO(E, F, m_2)` at a correct process `p_r` that is the receiver of message `m_2` is to devise an algorithm that collects the execution history `E` as `F` at `p_r` such that `valid(F) = 1` for `e_r^* ∈ T(E)` the receive event of `m_2`, with the algorithm using `F` to decide `CO_Deliv(m_2)`."

We mechanise this by reusing the CD admissibility predicate and restricting the target event to be a receive event. The "deliverability decision" is subsumed by `valid`

`(adv_E adv) F e_star`: knowing whether `F` is HB-faithful to `E` at the receive event of `m_2` is exactly the information needed to settle whether all causally prior messages have been received.

```
definition co_admissible :: "'p set ⇒ 'p adversary ⇒ bool" where
  "co_admissible C adv ⟷
     adversary_admissible C adv ∧ is_receive (adv_e_star adv)"

definition produces_valid_F_CO ::
  "'p set ⇒ 'p cd_solver ⇒ bool" where
  "produces_valid_F_CO C alg ⟷
     (∀adv. co_admissible C adv ⟶
        (let (F', _) = alg (adv_i adv) (adv_e_star adv) in
           valid (adv_E adv) F' (adv_e_star adv)))"

definition CO_solvable :: "comm_mode ⇒ 'p set ⇒ bool" where
  "CO_solvable m C ⟷ (∃alg. produces_valid_F_CO C alg)"
```

85 Forward reduction (T17): `CD` ⟶ `CO`

Paper, Section 5.2 (proof sketch of Theorem 17, the "`CO` ⪯ `CD`" direction):

> "To solve `CO(E, F, m_2)` at `p_r`, invoke `CD(E, F, e_r^*)` locally where `e_r^*` is the receive event of `m_2`; the BB-style output gives enough information to settle `CO_Deliv(m_2)`."

In our function-level abstraction this is immediate: a CD solver is correct for *every* admissible adversary; receive-event-target adversaries are a strict subset.

```
lemma produces_valid_F_imp_produces_valid_F_CO:
  assumes "produces_valid_F C alg"
  shows   "produces_valid_F_CO C alg"
  using assms
  by (auto simp: produces_valid_F_def produces_valid_F_CO_def
                 co_admissible_def)

lemma CD_solvable_imp_CO_solvable:
  assumes "CD_solvable m C"
  shows   "CO_solvable m C"
  using assms produces_valid_F_imp_produces_valid_F_CO
  by (auto simp: CD_solvable_def CO_solvable_def)

context process_partition
begin
```

86 Theorem 18a: CO is subject to false negatives

Paper, Theorem 18 (Section 5.2):

> "CO is subject to FN and FP in the Byzantine model."

The paper derives this as a corollary of the interreducibility (Theorem 17) and the FN/FP results for CD (Theorems 1/2 [42]). We discharge it directly with the same fresh-nat construction used for Theorem 1, adapted so the target is a receive event.

Our construction. Given an algorithm `alg`, pick a correct process `p_i` and a Byzantine process `p_b`. Use a two-message scenario:

- the *target* is the receive event of a fixed message `m_2 = 0` from `p_b` to `p_i`: namely `e_star = Receive p_i 2 p_b 0`;
- the *FN witness* is an earlier send-receive pair using a fresh message identifier `m_1 = fresh_nat F`.

Concretely:

$$E_{p_b} = [\text{Send } p_b\ 1\ p_i\ m_1,\ \text{Send } p_b\ 2\ p_i\ 0]$$
$$E_{p_i} = [\text{Receive } p_i\ 1\ p_b\ m_1,\ \text{Receive } p_i\ 2\ p_b\ 0]$$

The fresh `m_1 = fresh_nat F` guarantees the send `Send p_b 1 p_i m_1` – which has a happened-before chain to `e_star` in `E` – is absent from `events_of F`, producing a false negative.

```
definition co_fn_E :: "'p ⇒ 'p ⇒ nat ⇒ 'p history" where
  "co_fn_E p_i p_b m_1 ≡
     (λp. if p = p_b
          then [Send p_b 1 p_i m_1, Send p_b 2 p_i 0]
          else if p = p_i
          then [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]
          else [])"

definition co_fn_adv :: "'p ⇒ 'p ⇒ nat ⇒ 'p adversary" where
  "co_fn_adv p_i p_b m_1 ≡
     (| adv_E = co_fn_E p_i p_b m_1,
       adv_e_star = Receive p_i 2 p_b 0,
       adv_i = p_i |)"

lemma co_fn_E_at_pb [simp]:
  assumes "p_b ≠ p_i"
  shows "co_fn_E p_i p_b m_1 p_b
           = [Send p_b 1 p_i m_1, Send p_b 2 p_i 0]"
  using assms by (simp add: co_fn_E_def)

lemma co_fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "co_fn_E p_i p_b m_1 p_i
           = [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
  using assms by (auto simp: co_fn_E_def)

lemma co_fn_E_elsewhere:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "co_fn_E p_i p_b m_1 p = []"
  using assms by (simp add: co_fn_E_def)
```

```
lemma co_fn_E_events:
  assumes "p_b ≠ p_i"
  shows "events_of (co_fn_E p_i p_b m_1)
           = {Send p_b 1 p_i m_1, Send p_b 2 p_i 0,
              Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0}"
proof -
  have "events_of (co_fn_E p_i p_b m_1)
          = (⋃p. set (co_fn_E p_i p_b m_1 p))"
    by (simp add: events_of_def)
  also have "... = set (co_fn_E p_i p_b m_1 p_b)
                   ∪ set (co_fn_E p_i p_b m_1 p_i)
                   ∪ (⋃p ∈ -{p_b, p_i}. set (co_fn_E p_i p_b m_1 p))"
    by auto
  also have "set (co_fn_E p_i p_b m_1 p_b)
               = {Send p_b 1 p_i m_1, Send p_b 2 p_i 0}"
    using assms by simp
  moreover have "set (co_fn_E p_i p_b m_1 p_i)
                  = {Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0}"
    using assms by (simp add: co_fn_E_at_pi)
  moreover have "(⋃p ∈ -{p_b, p_i}. set (co_fn_E p_i p_b m_1 p)) = {}"
    by (auto simp: co_fn_E_elsewhere)
  ultimately show ?thesis by auto
qed

lemma wf_history_local_pb_in_co_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_b (co_fn_E p_i p_b m_1 p_b)"
proof -
  let ?L = "[Send p_b 1 p_i m_1, Send p_b 2 p_i 0]"
  have list_eq: "co_fn_E p_i p_b m_1 p_b = ?L"
    using assms by simp
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_b" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_pi_in_co_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (co_fn_E p_i p_b m_1 p_i)"
proof -
  let ?L = "[Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
  have list_eq: "co_fn_E p_i p_b m_1 p_i = ?L"
    using assms by (rule co_fn_E_at_pi)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_i" by simp
  have seq_ok: "∀k < length ?L. seq_of (?L ! k) = Suc k"
  proof (intro allI impI)
```

```
    fix k assume "k < length ?L"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?L ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_co_fn_E:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "wf_history_local p (co_fn_E p_i p_b m_1 p)"
  using assms by (simp add: co_fn_E_elsewhere wf_history_local_def)

lemma wf_history_co_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history (co_fn_E p_i p_b m_1)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (co_fn_E p_i p_b m_1 p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_co_fn_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms wf_history_local_pi_in_co_fn_E by simp
    next
      case False
      with ‹p ≠ p_b› show ?thesis
        by (rule wf_history_local_elsewhere_in_co_fn_E)
    qed
  qed
qed

lemma message_order_co_fn_E_send1_receive1:
  assumes "p_b ≠ p_i"
  shows "message_order (co_fn_E p_i p_b m_1)
           (Send p_b 1 p_i m_1) (Receive p_i 1 p_b m_1)"
  using assms unfolding message_order_def
  by (simp add: co_fn_E_events)

lemma program_order_co_fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "program_order (co_fn_E p_i p_b m_1)
           (Receive p_i 1 p_b m_1) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_E p_i p_b m_1"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
    using assms by (rule co_fn_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
```

```
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m_1" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Receive p_i 2 p_b 0" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis
    using e0 e1 unfolding program_order_def by blast
qed

lemma hb_send1_to_estar_in_co_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb (co_fn_E p_i p_b m_1)
          (Send p_b 1 p_i m_1) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_E p_i p_b m_1"
  let ?s = "Send p_b 1 p_i m_1"
  let ?r = "Receive p_i 1 p_b m_1"
  let ?es = "Receive p_i 2 p_b 0"
  have step1: "hb_step ?H ?s ?r"
    using message_order_co_fn_E_send1_receive1[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?r ?es"
    using program_order_co_fn_E_at_pi[OF assms]
    by (simp add: hb_step_def)
  have t1: "(hb_step ?H)^++ ?s ?r" using step1 by blast
  have t2: "(hb_step ?H)^++ ?r ?es" using step2 by blast
  have "(hb_step ?H)^++ ?s ?es" using t1 t2 by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)
qed

lemma hb_eval_send1_to_estar_in_co_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb_eval (co_fn_E p_i p_b m_1)
          (Send p_b 1 p_i m_1) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_E p_i p_b m_1"
  have e1: "Send p_b 1 p_i m_1 ∈ events_of ?H"
    using assms by (simp add: co_fn_E_events)
  have e2: "Receive p_i 2 p_b 0 ∈ events_of ?H"
    using assms by (simp add: co_fn_E_events)
  have h: "hb ?H (Send p_b 1 p_i m_1) (Receive p_i 2 p_b 0)"
    using assms by (rule hb_send1_to_estar_in_co_fn_E)
  show ?thesis using e1 e2 h by (simp add: hb_eval_def)
qed

lemma adv_admissible_co_fn_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i"
  shows "adversary_admissible correct (co_fn_adv p_i p_b m_1)"
proof -
  have a: "wf_history (co_fn_E p_i p_b m_1)"
    using assms(2) by (rule wf_history_co_fn_E)
  have b: "p_i ∈ correct" by (rule assms(1))
  have c: "proc_of (Receive p_i 2 p_b 0) = p_i" by simp
```

```
  have d: "Receive p_i 2 p_b 0 ∈ events_of (co_fn_E p_i p_b m_1)"
    using assms(2) by (simp add: co_fn_E_events)
  show ?thesis using a b c d
    by (simp add: adversary_admissible_def co_fn_adv_def)
qed

lemma co_admissible_co_fn_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i"
  shows "co_admissible correct (co_fn_adv p_i p_b m_1)"
proof -
  have adm: "adversary_admissible correct (co_fn_adv p_i p_b m_1)"
    using assms by (rule adv_admissible_co_fn_adv)
  have rcv: "is_receive (adv_e_star (co_fn_adv p_i p_b m_1))"
    by (simp add: co_fn_adv_def)
  show ?thesis using adm rcv by (simp add: co_admissible_def)
qed

theorem CO_FN_unavoidable:
  assumes byz_cor_distinct:
                "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
      and fin_F:  "∀p_i_in p_b_in.
                     finite (events_of
                               (fst (alg p_i_in (Receive p_i_in 2 p_b_in 0))))"
  shows "∃adv. co_admissible correct adv ∧
              false_negative (adv_E adv)
                             (fst (alg (adv_i adv) (adv_e_star adv)))
                             (adv_e_star adv)"
proof -
  obtain p_i p_b where
      pi_cor: "p_i ∈ correct" and pb_byz: "p_b ∈ byzantine"
      and dist: "p_b ≠ p_i"
    using byz_cor_distinct by blast

  let ?e_star = "Receive p_i 2 p_b 0"
  let ?F = "fst (alg p_i ?e_star)"
  let ?m_1 = "fresh_nat ?F"
  let ?adv = "co_fn_adv p_i p_b ?m_1"
  let ?s = "Send p_b 1 p_i ?m_1"

  have finF: "finite (events_of ?F)" using fin_F by blast

  have adm: "co_admissible correct ?adv"
    using pi_cor dist by (rule co_admissible_co_fn_adv)

  have witness_in_E: "?s ∈ events_of (adv_E ?adv)"
    using dist by (simp add: co_fn_adv_def co_fn_E_events)
  have hbE: "hb_eval (adv_E ?adv) ?s (adv_e_star ?adv)"
  proof -
    have "hb_eval (co_fn_E p_i p_b ?m_1) ?s (Receive p_i 2 p_b 0)"
      by (rule hb_eval_send1_to_estar_in_co_fn_E[OF dist])
    thus ?thesis by (simp add: co_fn_adv_def)
  qed
```

```
  have witness_not_in_F: "?s ∉ events_of ?F"
    using finF Send_at_fresh_nat_not_in_F[of ?F] by blast
  have not_hbF: "¬ hb_eval ?F ?s (adv_e_star ?adv)"
    using witness_not_in_F by (simp add: hb_eval_def co_fn_adv_def)

  from witness_in_E hbE not_hbF
  have "∃e ∈ events_of (adv_E ?adv) ∪ events_of ?F.
            hb_eval (adv_E ?adv) e (adv_e_star ?adv) ∧
            ¬ hb_eval ?F e (adv_e_star ?adv)"
    by blast
  hence FN: "false_negative (adv_E ?adv) ?F (adv_e_star ?adv)"
    by (simp add: false_negative_def)

  have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
    by (simp add: co_fn_adv_def)

  from adm FN F_eq show ?thesis by metis
qed
```

87 Theorem 18b: CO has FN or FP for internal-event witnesses

Paper, Theorem 18 (the FN/FP corollary, internal-event side – by the same logic as the paper's T2):

> "CO is subject to FN and FP in the Byzantine model." In particular, for any internal event `e_h^x` at a Byzantine process `p_h`, neither FN nor FP can be prevented when settling `e_h^x → e_r^*` at the receiving correct process `p_r`.

Construction. Strengthen the T18a construction with a chain of internal events at `p_b` before its sends. The witness is the `k`-th internal event at `p_b`, with `k = fresh_nat F` chosen so it is absent from `events_of F`.

```
definition co_fn_internal_E ::
  "'p ⇒ 'p ⇒ nat ⇒ nat ⇒ 'p history" where
  "co_fn_internal_E p_i p_b k m_1 ≡
     (λp. if p = p_b
          then (map (Internal p_b) [1..<Suc k])
                @ [Send p_b (Suc k) p_i m_1,
                   Send p_b (Suc (Suc k)) p_i 0]
          else if p = p_i
          then [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]
          else [])"

definition co_fn_internal_adv ::
  "'p ⇒ 'p ⇒ nat ⇒ nat ⇒ 'p adversary" where
  "co_fn_internal_adv p_i p_b k m_1 ≡
     (| adv_E = co_fn_internal_E p_i p_b k m_1,
       adv_e_star = Receive p_i 2 p_b 0,
       adv_i = p_i |)"

lemma co_fn_internal_E_at_pb [simp]:
```

```
  assumes "p_b ≠ p_i"
  shows "co_fn_internal_E p_i p_b k m_1 p_b
           = (map (Internal p_b) [1..<Suc k])
              @ [Send p_b (Suc k) p_i m_1,
                 Send p_b (Suc (Suc k)) p_i 0]"
  using assms by (simp add: co_fn_internal_E_def)

lemma co_fn_internal_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "co_fn_internal_E p_i p_b k m_1 p_i
           = [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
  using assms by (auto simp: co_fn_internal_E_def)

lemma co_fn_internal_E_elsewhere:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "co_fn_internal_E p_i p_b k m_1 p = []"
  using assms by (simp add: co_fn_internal_E_def)

lemma length_co_fn_internal_E_pb:
  assumes "p_b ≠ p_i"
  shows "length (co_fn_internal_E p_i p_b k m_1 p_b) = Suc (Suc k)"
  using assms by simp

lemma nth_co_fn_internal_E_pb_lt:
  assumes "p_b ≠ p_i" "j < k"
  shows "co_fn_internal_E p_i p_b k m_1 p_b ! j = Internal p_b (Suc j)"
proof -
  let ?xs = "map (Internal p_b) [1..<Suc k]"
  let ?tail = "[Send p_b (Suc k) p_i m_1, Send p_b (Suc (Suc k)) p_i 0]"
  let ?L = "?xs @ ?tail"
  have list_eq: "co_fn_internal_E p_i p_b k m_1 p_b = ?L"
    using assms(1) by simp
  have len_xs: "length ?xs = k"
    using assms(2) by simp
  with assms(2) have j_lt_len: "j < length ?xs" by simp
  have j_lt_upt: "j < length [1..<Suc k]"
    using assms(2) by simp
  have step1: "?L ! j = ?xs ! j"
    using j_lt_len by (simp add: nth_append)
  have step2: "?xs ! j = Internal p_b ([1..<Suc k] ! j)"
    using j_lt_upt by (rule nth_map)
  have step3: "[1..<Suc k] ! j = Suc j"
  proof -
    have "[1..<Suc k] ! j = 1 + j"
      using assms(2) by (intro nth_upt) simp
    thus ?thesis by simp
  qed
  from step1 step2 step3 list_eq show ?thesis by simp
qed

lemma nth_co_fn_internal_E_pb_at_k:
  assumes "p_b ≠ p_i"
```

```
  shows "co_fn_internal_E p_i p_b k m_1 p_b ! k = Send p_b (Suc k) p_i m_1"
  using assms by (simp add: nth_append)

lemma nth_co_fn_internal_E_pb_at_Sk:
  assumes "p_b ≠ p_i"
  shows "co_fn_internal_E p_i p_b k m_1 p_b ! Suc k
         = Send p_b (Suc (Suc k)) p_i 0"
  using assms by (simp add: nth_append)

lemma set_co_fn_internal_E_pb:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "set (co_fn_internal_E p_i p_b k m_1 p_b)
          = (Internal p_b ‘ {1..k})
              ∪ {Send p_b (Suc k) p_i m_1,
                 Send p_b (Suc (Suc k)) p_i 0}"
proof -
  have set_map: "set (map (Internal p_b) [1..<Suc k]) = Internal p_b ‘ {1..<Suc k}"
    by auto
  have rng_eq: "{1..<Suc k} = {1..k}" by auto
  have "set (co_fn_internal_E p_i p_b k m_1 p_b)
        = set (map (Internal p_b) [1..<Suc k])
            ∪ {Send p_b (Suc k) p_i m_1, Send p_b (Suc (Suc k)) p_i 0}"
    using assms(1) by simp
  also have "... = (Internal p_b ‘ {1..k})
                   ∪ {Send p_b (Suc k) p_i m_1,
                      Send p_b (Suc (Suc k)) p_i 0}"
    using set_map rng_eq by simp
  finally show ?thesis .
qed

lemma co_fn_internal_E_events:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "events_of (co_fn_internal_E p_i p_b k m_1) =
          (Internal p_b ‘ {1..k})
          ∪ {Send p_b (Suc k) p_i m_1,
             Send p_b (Suc (Suc k)) p_i 0,
             Receive p_i 1 p_b m_1,
             Receive p_i 2 p_b 0}"
proof -
  have "events_of (co_fn_internal_E p_i p_b k m_1)
          = (⋃p. set (co_fn_internal_E p_i p_b k m_1 p))"
    by (simp add: events_of_def)
  also have "... = set (co_fn_internal_E p_i p_b k m_1 p_b)
                   ∪ set (co_fn_internal_E p_i p_b k m_1 p_i)
                   ∪ (⋃p ∈ -{p_b, p_i}.
                           set (co_fn_internal_E p_i p_b k m_1 p))"
    by auto
  also have "(⋃p ∈ -{p_b, p_i}. set (co_fn_internal_E p_i p_b k m_1 p)) = {}"
    by (auto simp: co_fn_internal_E_elsewhere)
  moreover have "set (co_fn_internal_E p_i p_b k m_1 p_b)
                   = (Internal p_b ‘ {1..k})
                       ∪ {Send p_b (Suc k) p_i m_1,
```

```
                        Send p_b (Suc (Suc k)) p_i 0}"
    using assms by (rule set_co_fn_internal_E_pb)
  moreover have "set (co_fn_internal_E p_i p_b k m_1 p_i)
                  = {Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0}"
    using assms(1) by (simp add: co_fn_internal_E_at_pi)
  ultimately show ?thesis by auto
qed

lemma wf_history_local_pb_in_co_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "wf_history_local p_b (co_fn_internal_E p_i p_b k m_1 p_b)"
proof -
  let ?L = "co_fn_internal_E p_i p_b k m_1 p_b"
  have len_L: "length ?L = Suc (Suc k)"
    using assms(1) by (rule length_co_fn_internal_E_pb)

  have all_proc: "∀e ∈ set ?L. proc_of e = p_b"
  proof
    fix e assume "e ∈ set ?L"
    hence "e ∈ (Internal p_b ‘ {1..k})
              ∪ {Send p_b (Suc k) p_i m_1,
                 Send p_b (Suc (Suc k)) p_i 0}"
      using set_co_fn_internal_E_pb[OF assms] by simp
    thus "proc_of e = p_b" by (auto simp: image_iff)
  qed

  have all_seq: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)
    fix j assume "j < length ?L"
    hence j_lt: "j < Suc (Suc k)" using len_L by simp
    show "seq_of (?L ! j) = Suc j"
    proof (cases "j < k")
      case True
      thus ?thesis using nth_co_fn_internal_E_pb_lt[OF assms(1)] by simp
    next
      case False
      with j_lt have "j = k ∨ j = Suc k" by auto
      thus ?thesis
        using nth_co_fn_internal_E_pb_at_k[OF assms(1)]
              nth_co_fn_internal_E_pb_at_Sk[OF assms(1)]
        by auto
    qed
  qed

  show ?thesis using all_proc all_seq
    unfolding wf_history_local_def by blast
qed

lemma wf_history_local_pi_in_co_fn_internal_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (co_fn_internal_E p_i p_b k m_1 p_i)"
proof -
```

```
  let ?L = "[Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
  have list_eq: "co_fn_internal_E p_i p_b k m_1 p_i = ?L"
    using assms by (rule co_fn_internal_E_at_pi)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_i" by simp
  have seq_ok: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)
    fix j assume "j < length ?L"
    hence "j = 0 ∨ j = 1" by auto
    thus "seq_of (?L ! j) = Suc j" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_co_fn_internal_E:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "wf_history_local p (co_fn_internal_E p_i p_b k m_1 p)"
  using assms by (simp add: co_fn_internal_E_elsewhere wf_history_local_def)

lemma wf_history_co_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "wf_history (co_fn_internal_E p_i p_b k m_1)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (co_fn_internal_E p_i p_b k m_1 p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_co_fn_internal_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms(1) wf_history_local_pi_in_co_fn_internal_E by simp
    next
      case False
      with ‹p ≠ p_b› show ?thesis
        by (rule wf_history_local_elsewhere_in_co_fn_internal_E)
    qed
  qed
qed

lemma program_order_internal_to_send1_in_co_fn_internal:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "program_order (co_fn_internal_E p_i p_b k m_1)
           (Internal p_b k) (Send p_b (Suc k) p_i m_1)"
proof -
  let ?H = "co_fn_internal_E p_i p_b k m_1"
  have len: "length (?H p_b) = Suc (Suc k)"
    using assms(1) by (rule length_co_fn_internal_E_pb)
  from assms(2) have km1_lt_k: "k - 1 < k" by simp
  have at_km1: "(?H p_b) ! (k - 1) = Internal p_b (Suc (k - 1))"
```

```
    using assms(1) km1_lt_k nth_co_fn_internal_E_pb_lt by simp
  hence at_km1': "(?H p_b) ! (k - 1) = Internal p_b k"
    using assms(2) by simp
  have at_k: "(?H p_b) ! k = Send p_b (Suc k) p_i m_1"
    using assms(1) by (rule nth_co_fn_internal_E_pb_at_k)
  have idx1: "k - 1 < k" using assms(2) by simp
  have idx2: "k < length (?H p_b)" using len by simp
  show ?thesis using at_km1' at_k idx1 idx2
    unfolding program_order_def by blast
qed

lemma program_order_receive_to_estar_in_co_fn_internal:
  assumes "p_b ≠ p_i"
  shows "program_order (co_fn_internal_E p_i p_b k m_1)
           (Receive p_i 1 p_b m_1) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_internal_E p_i p_b k m_1"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m_1, Receive p_i 2 p_b 0]"
    using assms by (rule co_fn_internal_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m_1" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Receive p_i 2 p_b 0" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis using e0 e1
    unfolding program_order_def by blast
qed

lemma message_order_send1_to_receive1_in_co_fn_internal:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "message_order (co_fn_internal_E p_i p_b k m_1)
           (Send p_b (Suc k) p_i m_1) (Receive p_i 1 p_b m_1)"
  using assms unfolding message_order_def
  by (simp add: co_fn_internal_E_events)

lemma hb_internal_to_estar_in_co_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "hb (co_fn_internal_E p_i p_b k m_1)
           (Internal p_b k) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_internal_E p_i p_b k m_1"
  let ?ih = "Internal p_b k"
  let ?s = "Send p_b (Suc k) p_i m_1"
  let ?r = "Receive p_i 1 p_b m_1"
  let ?es = "Receive p_i 2 p_b 0"
  have step1: "hb_step ?H ?ih ?s"
    using program_order_internal_to_send1_in_co_fn_internal[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?s ?r"
    using message_order_send1_to_receive1_in_co_fn_internal[OF assms]
    by (simp add: hb_step_def)
  have step3: "hb_step ?H ?r ?es"
```

```
    using program_order_receive_to_estar_in_co_fn_internal[OF assms(1)]
    by (simp add: hb_step_def)
  have t1: "(hb_step ?H)^++ ?ih ?s" using step1 by blast
  have t2: "(hb_step ?H)^++ ?s ?r" using step2 by blast
  have t3: "(hb_step ?H)^++ ?r ?es" using step3 by blast
  have "(hb_step ?H)^++ ?ih ?r" using t1 t2 by (rule tranclp_trans)
  hence "(hb_step ?H)^++ ?ih ?es" using t3 by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)
qed

lemma hb_eval_internal_to_estar_in_co_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "hb_eval (co_fn_internal_E p_i p_b k m_1)
           (Internal p_b k) (Receive p_i 2 p_b 0)"
proof -
  let ?H = "co_fn_internal_E p_i p_b k m_1"
  have e_ih: "Internal p_b k ∈ events_of ?H"
    using assms by (simp add: co_fn_internal_E_events)
  have e_es: "Receive p_i 2 p_b 0 ∈ events_of ?H"
    using assms by (simp add: co_fn_internal_E_events)
  have h: "hb ?H (Internal p_b k) (Receive p_i 2 p_b 0)"
    using assms by (rule hb_internal_to_estar_in_co_fn_internal_E)
  show ?thesis using e_ih e_es h by (simp add: hb_eval_def)
qed

lemma adv_admissible_co_fn_internal_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i" "k ≥ 1"
  shows "adversary_admissible correct (co_fn_internal_adv p_i p_b k m_1)"
proof -
  have a: "wf_history (co_fn_internal_E p_i p_b k m_1)"
    using assms(2) assms(3) by (rule wf_history_co_fn_internal_E)
  have b: "p_i ∈ correct" by (rule assms(1))
  have c: "proc_of (Receive p_i 2 p_b 0) = p_i" by simp
  have d: "Receive p_i 2 p_b 0 ∈ events_of (co_fn_internal_E p_i p_b k m_1)"
    using assms(2) assms(3) by (simp add: co_fn_internal_E_events)
  show ?thesis using a b c d
    by (simp add: adversary_admissible_def co_fn_internal_adv_def)
qed

lemma co_admissible_co_fn_internal_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i" "k ≥ 1"
  shows "co_admissible correct (co_fn_internal_adv p_i p_b k m_1)"
proof -
  have adm: "adversary_admissible correct (co_fn_internal_adv p_i p_b k m_1)"
    using assms by (rule adv_admissible_co_fn_internal_adv)
  have rcv: "is_receive (adv_e_star (co_fn_internal_adv p_i p_b k m_1))"
    by (simp add: co_fn_internal_adv_def)
  show ?thesis using adm rcv by (simp add: co_admissible_def)
qed

theorem CO_FN_or_FP_unavoidable_internal:
  assumes byz_cor_distinct:
```

```
                "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
    and fin_F:  "∀p_i_in p_b_in.
                  finite (events_of
                           (fst (alg p_i_in (Receive p_i_in 2 p_b_in 0))))"
  shows "∃adv e_h. co_admissible correct adv ∧
                  (∃p n. e_h = Internal p n) ∧
                  ((hb_eval (adv_E adv) e_h (adv_e_star adv) ∧
                    ¬ hb_eval (fst (alg (adv_i adv) (adv_e_star adv)))
                              e_h (adv_e_star adv))
                   ∨
                   (¬ hb_eval (adv_E adv) e_h (adv_e_star adv) ∧
                    hb_eval (fst (alg (adv_i adv) (adv_e_star adv)))
                            e_h (adv_e_star adv)))"
proof -
  obtain p_i p_b where
      pi_cor: "p_i ∈ correct" and pb_byz: "p_b ∈ byzantine"
      and dist: "p_b ≠ p_i"
    using byz_cor_distinct by blast

  let ?e_star = "Receive p_i 2 p_b 0"
  let ?F = "fst (alg p_i ?e_star)"
  let ?k = "fresh_nat ?F"
  let ?m_1 = "fresh_nat ?F"
  let ?adv = "co_fn_internal_adv p_i p_b ?k ?m_1"
  let ?ih = "Internal p_b ?k"

  have finF: "finite (events_of ?F)" using fin_F by blast

  have k_pos: "?k ≥ 1"
    by (simp add: fresh_nat_def Suc_leI)

  have adm: "co_admissible correct ?adv"
    using pi_cor dist k_pos by (rule co_admissible_co_fn_internal_adv)

  have witness_in_E: "?ih ∈ events_of (adv_E ?adv)"
    using dist k_pos
    by (auto simp: co_fn_internal_adv_def co_fn_internal_E_events)
  have hbE: "hb_eval (adv_E ?adv) ?ih (adv_e_star ?adv)"
  proof -
    have "hb_eval (co_fn_internal_E p_i p_b ?k ?m_1) ?ih (Receive p_i 2 p_b 0)"
      by (rule hb_eval_internal_to_estar_in_co_fn_internal_E[OF dist k_pos])
    thus ?thesis by (simp add: co_fn_internal_adv_def)
  qed
  have witness_not_in_F: "?ih ∉ events_of ?F"
    using finF Internal_at_fresh_nat_not_in_F[of ?F] by blast
  have not_hbF: "¬ hb_eval ?F ?ih (adv_e_star ?adv)"
    using witness_not_in_F by (simp add: hb_eval_def co_fn_internal_adv_def)

  have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
    by (simp add: co_fn_internal_adv_def)

  have e_h_internal: "∃p n. ?ih = Internal p n" by blast
```

```
  from adm e_h_internal hbE not_hbF F_eq witness_in_E
  show ?thesis by metis
qed

end — context process_partition
```

88 Theorems 17 and impossibility of CO

```
context byzantineSystem
begin
```

The direct CO-impossibility lemma, mirroring `no_produces_valid_F` in `Impossibility.thy`: given a finiteness side condition on every candidate CO solver's output history at the Theorem 18a target shape, Theorem 18a directly contradicts the existence of a CO-valid algorithm.

```
lemma no_produces_valid_F_CO:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                        (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "¬ (∃co_alg. produces_valid_F_CO correct co_alg)"
proof
  assume "∃co_alg. produces_valid_F_CO correct co_alg"
  then obtain co_alg
    where val_F: "produces_valid_F_CO correct co_alg" by blast

  have byz_cor_distinct:
    "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
    by (rule byz_cor_distinct_of_ne[OF byz_ne cor_ne])

  have fin_F:
    "∀p_i_in p_b_in.
        finite (events_of
                  (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0))))"
    using fin_co val_F by blast

  obtain adv where
    adm: "co_admissible correct adv"
    and FN: "false_negative (adv_E adv)
                            (fst (co_alg (adv_i adv) (adv_e_star adv)))
                            (adv_e_star adv)"
    using CO_FN_unavoidable[where alg = co_alg,
                            OF byz_cor_distinct fin_F]
    by blast

  have valid_at:
    "valid (adv_E adv)
```

```
          (fst (co_alg (adv_i adv) (adv_e_star adv)))
          (adv_e_star adv)"
    using val_F adm
    by (auto simp: produces_valid_F_CO_def Let_def split: prod.split)
  hence "¬ false_negative (adv_E adv)
                          (fst (co_alg (adv_i adv) (adv_e_star adv)))
                          (adv_e_star adv)"
    using valid_iff_no_FP_FN by blast
  with FN show False by contradiction
qed
```

89 CO impossibility: unicast / broadcast / multicast

```
theorem CO_impossible_unicast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                       (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "¬ CO_solvable Unicast correct"
  using no_produces_valid_F_CO[OF byz_ne cor_ne fin_co]
  by (auto simp: CO_solvable_def)

theorem CO_impossible_broadcast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                       (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "¬ CO_solvable Broadcast correct"
  using no_produces_valid_F_CO[OF byz_ne cor_ne fin_co]
  by (auto simp: CO_solvable_def)

theorem CO_impossible_multicast:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                       (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "¬ CO_solvable Multicast correct"
  using no_produces_valid_F_CO[OF byz_ne cor_ne fin_co]
  by (auto simp: CO_solvable_def)
```

90 Theorem 17: CO and CD are interreducible in the Byzantine model

Paper, Theorem 17 (Section 5.2):

> "CO and CD are interreducible in the Byzantine model."

The forward direction (`CD ⟶ CO`) is `CD_solvable_imp_CO_solvable` above, proven outside any locale – a CD solver is automatically a CO solver (CO admissibility is a strict restriction of CD admissibility).

The reverse direction (`CO ⟶ CD`) under Byzantine premises holds vacuously, because both sides are impossible: CO is impossible by `CO_impossible_unicast`/etc. above (¬ `CO_solvable`); CD is impossible by `CD_impossible_unicast`/etc. in `Impossibility.thy` (¬ `CD_solvable`). Hence `CO_solvable ⟷ CD_solvable`: both False.

Deviation: the paper sketches a constructive reverse reduction ("the reverse direction is similar"). In our function- level abstraction this would require synthesising a CD solver from a CO solver by extending every CD adversary with a self-receive event after its target – doable, but substantial. The vacuous route through the (independently proven) impossibility theorems yields the same final interreducibility statement.

```
theorem T17_CO_interreducible_with_CD:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                              (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                        (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "CO_solvable m correct ⟷ CD_solvable m correct"
proof -
  have no_cd: "¬ CD_solvable m correct"
  proof (cases m)
    case Unicast
    thus ?thesis
      using CD_impossible_unicast[OF byz_ne cor_ne fin_cd] by simp
  next
    case Broadcast
    thus ?thesis
      using CD_impossible_broadcast[OF byz_ne cor_ne fin_cd] by simp
  next
    case Multicast
    thus ?thesis
      using CD_impossible_multicast[OF byz_ne cor_ne fin_cd] by simp
  qed
  have no_co: "¬ CO_solvable m correct"
  proof (cases m)
    case Unicast
    thus ?thesis
```

```
      using CO_impossible_unicast[OF byz_ne cor_ne fin_co] by simp
  next
    case Broadcast
    thus ?thesis
      using CO_impossible_broadcast[OF byz_ne cor_ne fin_co] by simp
  next
    case Multicast
    thus ?thesis
      using CO_impossible_multicast[OF byz_ne cor_ne fin_co] by simp
  qed
  from no_cd no_co show ?thesis by blast
qed
```

Summary corollary: CO is impossible under all three communication modes (one theorem statement, all three modes).

```
theorem CO_impossible_all_modes:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_co:
        "∀co_alg. produces_valid_F_CO correct co_alg ⟶
           (∀p_i_in p_b_in.
              finite (events_of
                        (fst (co_alg p_i_in (Receive p_i_in 2 p_b_in 0)))))"
  shows "¬ CO_solvable Unicast   correct"
    and "¬ CO_solvable Broadcast correct"
    and "¬ CO_solvable Multicast correct"
proof -
  show "¬ CO_solvable Unicast correct"
    by (rule CO_impossible_unicast[OF byz_ne cor_ne fin_co])
  show "¬ CO_solvable Broadcast correct"
    by (rule CO_impossible_broadcast[OF byz_ne cor_ne fin_co])
  show "¬ CO_solvable Multicast correct"
    by (rule CO_impossible_multicast[OF byz_ne cor_ne fin_co])
qed

end — context byzantineSystem

end


theory CD_with_Crypto
  imports CD_B_Algorithm Impossibility
begin

context byzantineSystem
begin
```

91 Theorem 9: CD impossible, multicast + crypto

Paper, Theorem 9 (Section 4.4.1):

> "It is impossible to solve causality determination (Definition 5) as specified

by CD(E, F, e_i^*) in an asynchronous multicast- based message passing system with one or more Byzantine processes even when using cryptography."

Paper's proof: routes through Theorem 1's FN attack. "A Byzantine p_g can delete from F_g information about a multicast of m ... despite doing broadcasts using the BRB layer." The cryptographic envelope around m (group encryption plus BRB-broadcast of the ciphertext) does not prevent a Byzantine from omitting the receive event from its own reported history.

Our mechanisation: direct corollary of ⟦byzantine ≠ {}; correct ≠ {}; ∀cd_alg. produces_valid_F correct cd_alg ⟶ (∀p_i_in. finite (events_of (fst (cd_alg p_i_in (Internal p_i_in 2)))))⟧ ⟹ ¬ CD_solvable Multicast correct. Cryptography does not alter the correctness condition valid nor the adversary model, so the same fresh-id construction defeats every candidate crypto-augmented algorithm.

```
theorem T9_CD_impossible_multicast_with_crypto:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                              (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Multicast correct"
  by (rule CD_impossible_multicast[OF byz_ne cor_ne fin_cd])
```

92 Theorem 10: CD impossible, unicast + crypto

Paper, Theorem 10 (Section 4.4.1):

> "It is impossible to solve causality determination (Definition 5) as specified by CD(E, F, e_i^*) in an asynchronous unicast- based message passing system with one or more Byzantine processes even when using cryptography."

Paper's proof: "The proof of Theorem 9 carries over identically where each multicast group consists of two processes – the sender and the receiver." The paper notes additionally that "false positives can be prevented only if the semantics of the message content of a message do not matter". At our abstraction level the headline conclusion is the unconditional impossibility, mirroring the table entry.

```
theorem T10_CD_impossible_unicast_with_crypto:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                              (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Unicast correct"
  by (rule CD_impossible_unicast[OF byz_ne cor_ne fin_cd])
```

93 Theorem 11: CD impossible, broadcast + crypto

Paper, Theorem 11 (Section 4.4.1):

> "It is impossible to solve causality determination (Definition 5) as specified by `CD(E, F, e_i^*)` in an asynchronous broadcast- based message passing system with one or more Byzantine processes even when using cryptography."

Paper's proof: "The proof of Theorem 4 [...] carries over mostly identically with the two observations that (1) false positives can be prevented even without cryptography, and (2) false negatives cannot be prevented due to Theorem 1 whose proof is independent of whether or not cryptography is used."

The FN side of the disjunction is the headline conclusion, and is exactly ⟦`byzantine ≠ {}; correct ≠ {}; ∀cd_alg. produces_valid_F correct cd_alg ⟶ (∀p_i_in. finite (events_of (fst (cd_alg p_i_in (Internal p_i_in 2)))))`⟧ ⟹ ¬ `CD_solvable Broadcast correct`.

```
theorem T11_CD_impossible_broadcast_with_crypto:
  assumes byz_ne: "byzantine ≠ {}"
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                              (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Broadcast correct"
  by (rule CD_impossible_broadcast[OF byz_ne cor_ne fin_cd])
```

94 Theorem 12: CD_B possible, unicast + crypto

Paper, Theorem 12 (Section 4.5):

> "It is possible to solve causality determination (Definition 6) as specified by `CD_B(E, F, e_i^*)`, now defined in terms of the →_B relation, in an asynchronous unicast-based message passing system with one or more Byzantine processes when using cryptography."

Paper's argument: for the BHB version, "only all true causal dependencies are faithfully transmitted" along a causal path through correct processes. With or without cryptography, the →_B chain reaches every correct receiver of every causally preceding message.

Our mechanisation: direct corollary of `CD_B_solvable_with_recv Unicast correct`. The abstract correctness predicate `produces_valid_F_B_recv` is mode-agnostic; the operational discharge of `correct_reporting` is the only place where the choice "BRU vs. crypto" would matter, and the operational layer is below our abstraction.

```
theorem T12_CD_B_solvable_unicast_with_crypto:
  shows "CD_B_solvable_with_recv Unicast correct"
  by (rule CD_B_solvable_unicast)
```

95 Theorem 13: CD_B possible, broadcast + crypto

Paper, Theorem 13 (Section 4.5):

> "It is possible to solve causality determination (Definition 6) as specified by `CD_B(E, F, e_i^*)`, now defined in terms of the $\rightarrow_B$ relation, in an asynchronous broadcast-based message passing system with one or more Byzantine processes when using cryptography."

Our mechanisation: direct corollary of `CD_B_solvable_with_recv Broadcast correct`. Same reasoning as T12.

```
theorem T13_CD_B_solvable_broadcast_with_crypto:
  shows "CD_B_solvable_with_recv Broadcast correct"
  by (rule CD_B_solvable_broadcast)
```

96 Theorem 14: CD_B possible, multicast + crypto

Paper, Theorem 14 (Section 4.5):

> "It is possible to solve causality determination (Definition 6) as specified by `CD_B(E, F, e_i^*)`, now defined in terms of the $\rightarrow_B$ relation, in an asynchronous multicast-based message passing system with one or more Byzantine processes when using cryptography."

The genuinely new content of Section 4.4. Without cryptography $[\![\exists$ `p_i p_c. p_i ∈ correct ∧ p_c ∈ correct ∧ p_c ≠ p_i; ∀alg. produces_valid_F_B_recv_strong correct alg ⟶ (∀recv p_i_in. finite (events_of (fst (alg recv p_i_in (Internal p_i_in 2)))))` $]\!] \Longrightarrow \nexists$ `alg. produces_valid_F_B_recv_strong correct alg` (the recv-strong predicate) showed multicast was unsolvable because BRM is unachievable (`t_G < |G|/3` would require identifying the Byzantine processes within each multicast group). *With* cryptography, the paper argues that group encryption plus BRB-broadcast of the ciphertext, augmented by recursive hash histories, achieves `correct_reporting` for multicast.

Our mechanisation: the abstract correctness predicate is the same `produces_valid_F_B_recv` used for T6/T7/T12/T13. At this abstraction level T14 is therefore a direct application of $\exists$`alg. produces_valid_F_B_recv correct alg` with the multicast mode tag. The operational role of cryptography – discharging `correct_reporting` *under multicast* – is the only substantive content that differs between T8 (impossible without crypto) and T14 (possible with crypto), and that substantive content lives below our abstraction (the same way BRU/BCB live below for T6/T7).

```
theorem T14_CD_B_solvable_multicast_with_crypto:
  shows "CD_B_solvable_with_recv Multicast correct"
  unfolding CD_B_solvable_with_recv_def
  by (rule CD_B_solvable_under_correct_reporting)
```

97 Summary corollary: all six crypto theorems together

One statement combining T9-T14. The Byzantine premises (`byz_ne`, `cor_ne`, `fin_cd`) are required for the impossibility halves (T9/T10/T11); the possibility halves (T12/T13/T14) are unconditional.

```
theorem crypto_theorems_summary:
  assumes byz_ne: "byzantine ≠ {}"
```

```
      and cor_ne: "correct ≠ {}"
      and fin_cd:
        "∀cd_alg. produces_valid_F correct cd_alg ⟶
           (∀p_i_in. finite (events_of
                              (fst (cd_alg p_i_in (Internal p_i_in 2)))))"
  shows "¬ CD_solvable Multicast correct"   — T9
    and "¬ CD_solvable Unicast correct"     — T10
    and "¬ CD_solvable Broadcast correct"   — T11
    and "CD_B_solvable_with_recv Unicast correct"     — T12
    and "CD_B_solvable_with_recv Broadcast correct"   — T13
    and "CD_B_solvable_with_recv Multicast correct"   — T14
proof -
  show "¬ CD_solvable Multicast correct"
    by (rule T9_CD_impossible_multicast_with_crypto[OF byz_ne cor_ne fin_cd])
  show "¬ CD_solvable Unicast correct"
    by (rule T10_CD_impossible_unicast_with_crypto[OF byz_ne cor_ne fin_cd])
  show "¬ CD_solvable Broadcast correct"
    by (rule T11_CD_impossible_broadcast_with_crypto[OF byz_ne cor_ne fin_cd])
  show "CD_B_solvable_with_recv Unicast correct"
    by (rule T12_CD_B_solvable_unicast_with_crypto)
  show "CD_B_solvable_with_recv Broadcast correct"
    by (rule T13_CD_B_solvable_broadcast_with_crypto)
  show "CD_B_solvable_with_recv Multicast correct"
    by (rule T14_CD_B_solvable_multicast_with_crypto)
qed

end — context byzantineSystem

end
```

References

[1] B. Bisping, P.-D. Brodmann, T. Jungnickel, C. Rickmann, H. Seidler, A. Stüber, A. Wilhelm-Weidner, K. Peters, and U. Nestmann. A constructive proof for FLP. Archive of Formal Proofs, 2025. Formal proof development. Originally published 2016; last updated 2025-03. Cited year matches the AFP snapshot used in this development.

[2] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[3] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[4] A. Misra and A. D. Kshemkalyani. Byzantine-tolerant detection of causality: There is no holy grail. *Parallel Computing*, 124:103136, 2025.